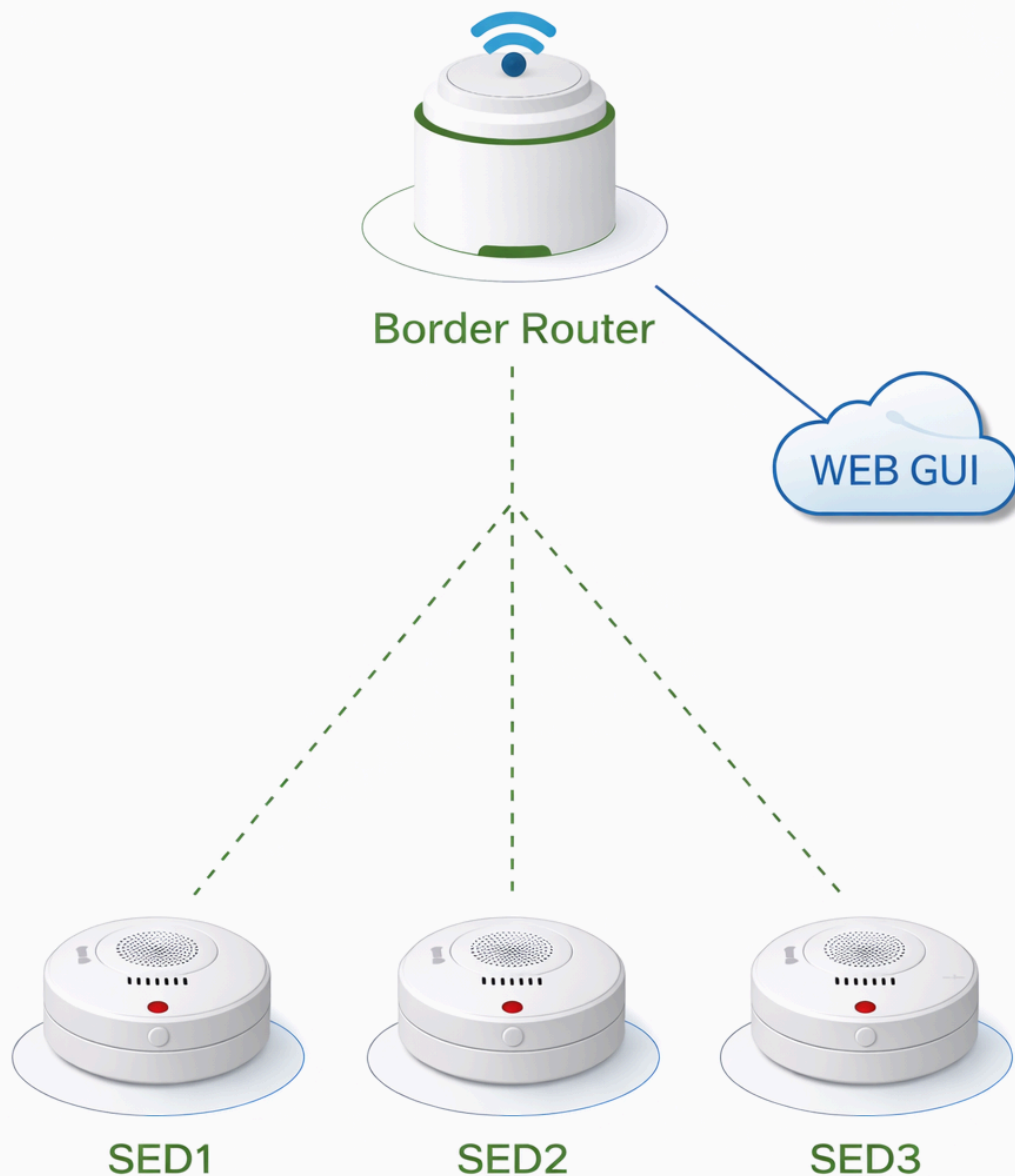




4. Semesters Eksamens projekt, It-teknolog, EAAA



Vejleder: Caspar Facius
Studerende: Oliver Hviid og Henrik B. Bruun
Afleveringsdato: 19.12.2025
Antal tegn: 82744
Antal sider: 34,47



Abstract

This project investigates a prototype wireless fire safety concept based on OpenThread and microcontroller technology, using retrofitting of pre-2020 buildings as the contextual frame for interpreting relevant BR18 fire safety requirements.

Instead of implementing a finished smoke alarm product, the work focuses on a simulated smoke detector in a Thread mesh network, where fire events are triggered via button input and siren behaviour is represented by on-board LEDs and status in a web-based interface.

The prototype consists of an ESP32-based Border Router (ESP32-S3 and ESP32-H2) connected to a small Thread network of three nRF54L15-based sleep end-devices, which act as battery-powered alarm nodes with configurable poll periods.

System behaviour is evaluated against a requirement-driven test plan covering four key areas: wireless range (line-of-sight, indoor and mesh-assisted communication using RSSI and PDR metrics), security (AES-128 encrypted Thread traffic, rejection of unauthorized join attempts and HTTPS-protected access with logging of security-relevant events), WebGUI functionality (central status overview, battery monitoring, alarm test and hush control) and energy consumption (current measurements under different operating and polling conditions to estimate suitability for low-power operation).

The results demonstrate that a Thread-based topology with a Border Router and sleep end-devices can provide robust communication, integrated monitoring and a realistic path towards an energy-efficient, upgradeable fire safety system for existing buildings, while also highlighting the technical considerations that must be addressed in future iterations with real sensors and certified hardware.



Indholdsfortegnelse

1. Projektbeskrivelse.....	5
1.1 Problemformulering.....	5
1.2 Formål.....	5
1.3 Afgrænsning.....	5
1.4 Metode.....	5
1.5 Kravspecifikationer.....	6
2. Indledende research.....	7
3. Indledning.....	9
4. Projektmanagement.....	10
5. Design.....	11
5.1 Blokdigram.....	11
5.2 WEB-GUI design.....	13
5.2.1 Battery request.....	13
5.2.2 Alarm Test.....	14
5.2.3 Hush alarm.....	14
6. Software development.....	15
6.1 Openthread.....	15
6.2 Zephyr.....	17
6.3 FreeRtos/IDF.....	18
6.4 DTLS - Datagram transport layer security.....	19
6.5 Embedded software hierarki diagram.....	20
6.5.1 Border router.....	21
6.5.2 Sleep end-device.....	27
7. Projekt test.....	33
7.1 Robusthedstest: K-01.....	34
7.1.1 Robusthedstest 1 - Rækkevidde fri linje.....	34
7.1.2 Robusthedstest 2 - Rækkevidde indendørs forhindringstest.....	36
7.1.3 Robusthedstest 3 – Mesh-hop simuleringstest.....	37
7.2 Sikkerhed: K-02.....	40
7.2.1 Sikkerhedstest 1 - Krypterings Verifikation.....	40
7.2.2 Sikkerhedstest 2 - Uautoriseret tilslutningsforsøg.....	41
7.2.3 Sikkerhedstest 3 - HTTPS.....	43
7.3 WEB-GUI: K-03.....	45
7.3.1 WEB-GUI enhedstest.....	45
7.3.2 Alarm test.....	47
7.3.3 Brandalarms test.....	48
7.4 Batterilevetid: K-04.....	49
7.4.1 Batteritest - Strømmåling.....	49
7.4.2 Strømforbrugs test.....	51
7.4.3 Polling period test.....	53
7.5 Produkt bevis.....	54



8. Perspektivering.....	55
8.1 PCB.....	55
8.2 Sikkerhed ved fysisk manipulation.....	55
8.3 Monitorering af enheders forbindelse.....	55
8.4 Genopladelige batterier.....	55
9. Konklusion.....	56
10. Bilag.....	57
10.1 Figurliste.....	57
10.2 Referenceliste.....	59
10.3 Bygningsreglement.....	62
10.4 Detaljerede kravspecifikationer:.....	63
10.5 O-QPSK Modulation og DSSS i IEEE 802.15.4.....	67
10.6 AES.....	69
10.7 Opstart af Border Router.....	72
10.8 Kode SED.....	76
10.8.1 main.c.....	76
10.8.2 network.c.....	76
10.8.3 udp.c.....	78
10.8.4 fire.c.....	82
10.8.5 alarm_test.c.....	85
10.8.6 adc.c.....	86
10.9 Kode Border router.....	86
10.9.1 esp_br_web.c.....	86
10.9.2 udp_br_func.c.....	94
10.9.3 index.html.....	104
10.9.4 device.html.....	110
10.9.5 style.css.....	112
10.10 OTNS og Wireshark.....	118
10.11 Bellman-Ford Algoritmen og table routing.....	126
10.12 Opsætning af sniffer og indstilling af dekryptering:.....	127
10.13 Angrebsmetoder på AES.....	134
10.14 Konvertering til HTTPS.....	135
10.15 OTNS tabeller.....	138



1. Projektbeskrivelse

Mange bygninger opført før 2020 er ikke udstyret med brandsikringssystemer, der lever op til BR18's brandkrav - se bilag 10.3. Dette udgør en potentiel sikkerhedsrisiko for både private husholdninger og virksomheder. Samtidig kan traditionelle brandsikringsløsninger være dyre og besværlige at eftermontere. Der er derfor behov for en fleksibel, energieffektiv og brugervenlig løsning, som kan opgradere sikkerheden i ældre bygninger uden større indgreb.

1.1 Problemformulering

Hvordan kan et brandalarmsystem, baseret på Thread-kommunikation, designes og demonstreres som en mulig løsning til eksisterende bygninger opført før 2020, så det understøtter centrale krav til pålidelig alarmering og driftssikker kommunikation i overensstemmelse med relevante dele af BR18, samtidig med at der tages højde for brugervenlighed, energieffektivitet og øget tryghed for brugeren?

- Hvilke funktioner er nødvendige for at systemet kan betragtes som sikkert og pålideligt i en nødsituation?
- Hvordan kan enhedernes brugerflade gøres intuitive for brugere?
- Hvordan kan systemet give feedback til brugeren, så trygheden øges i hverdagen?
- Hvordan kan energieffektivitet opnås gennem hardwarevalg og brug af OpenThread's funktioner?
- Hvordan påvirker energiforbruget systemets levetid og vedligeholdelsesbehov?

1.2 Formål

Projektets formål er at udvikle og demonstrere et simuleret trådløst brandsikringssystem baseret på OpenThread. Ved hjælp af udviklingsboards illustreres, hvordan et netværk af simulerede brandalarmer kan kommunikere pålideligt, sikkert, brugervenligt og energieffektivt mellem hinanden. Løsningen er en teknologisk prototype, som demonstrerer potentialet for anvendelse i både private hjem og erhvervsbygninger.

1.3 Afgrænsning

Projektet fokuserer på embedded softwareudvikling, trådløs kommunikation og systemets funktionelle demonstration. Projektet omfatter ikke certificering, færdigproducerede enheder, fast forsyning til enheder eller økonomisk rentabilitet, men sigter mod at give et teknologisk grundlag, der kan videreudvikles.

1.4 Metode

I projektet anvendes Nordic Semiconductor udviklingsboards nRF54L15 development kit samt ESP32-baserede Border Routers (S3 og H2) til at simulere et OpenThread-mesh-netværk. Hvert udviklingsboard har en rolle i denne simulerede brandalarm, hvor en trykknop kan udløse en brandhændelse, og LED'er bruges til at illustrere den visuelle alarm.



Når en alarm aktiveres, sender det pågældende board en besked til netværkets leder, som derefter distribuerer beskeden videre til de øvrige noder i netværket. Dette sikrer, at alle enheder gradvist aktiveres og indgår i alarmsituationen. På grund af forskudte polling rates blandt sleep end-devices (SED) vil aktiveringen ske med små variationer i tid, men fortsat inden for et kort interval, hvilket illustrerer synkroniseringsmekanismen i et trådløst mesh-netværk.

Metoden omfatter både hardware og software til at simulere og demonstrere en brandhændelse. Hardwaredelen består af udviklingsboards og knapper/LED'er, mens softwaredelen håndterer kommunikationen via OpenThread-protokollen, koordineringen gennem netværkslederen samt styringen af alarmsekvensen. På den måde kan systemets reaktionsevne, robusthed og skalerbarhed evalueres under realistiske demonstrationsbetingelser.

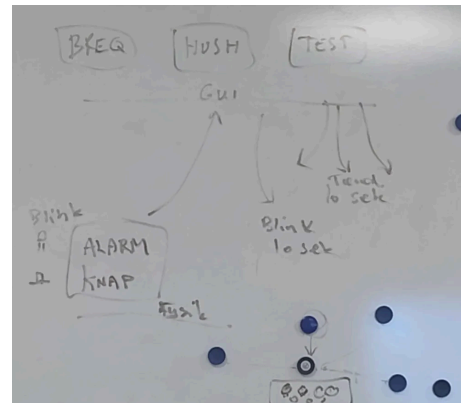
1.5 Kravspecifikationer

Kravspecifikationen er struktureret omkring fire centrale testområder, som tilsammen dækker systemets væsentligste egenskaber. Tabellen nedenfor giver et samlet overblik over disse områder og deres relation til det underliggende Thread mesh-netværk. Sammen viser de, hvordan kravene fokuserer på at dokumentere rækkevidde, sikkerhed, brugergrænseflade og batterilevetid i systemet. Detaljerede kravspecifikationer se bilag 10.4.

Krav-område	Fokusområde	Relation til Thread	Overordnet formål	Verifikation
Robusthed (K-01)	Trådløs kommunikation, signal-stabilitet og mesh routing	Fysisk kommunikation og mesh routing	At dokumentere systemets dækningsområde og mesh logik	Bestået
Sikkerhed (K-02)	Kryptering, autentificering og dataintegritet	Beskyttelse af kommunikation	At dokumentere beskytte af dataudveksling mod uautoriseret adgang	Bestået
WEB-GUI (K-03)	Central monitorering og kontrol	Brugergrænseflade	At dokumentere overblik og styring	Bestået
Batterilevetid (K-04)	Energiforbrug og driftstid	Monitorering af netværks-helbred	At dokumentere vedligeholdelses kadence og batterilevetid	Bestået

2. Indledende research

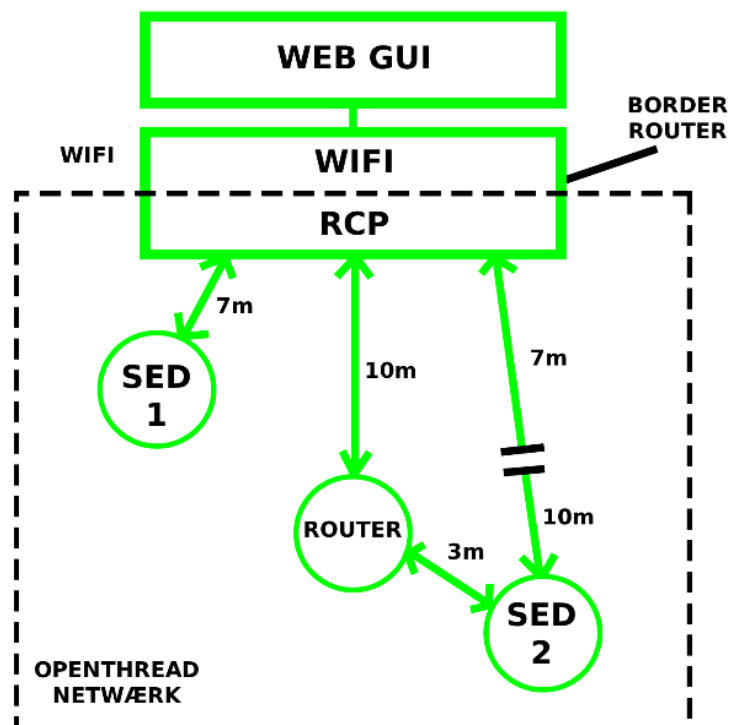
Projektets indledende fase omfattede en grundig undersøgelse af relevante kommunikations-protokoller med fokus på Thread og Matter. Formålet var at identificere, hvordan en evt. implementering af begge kunne se ud. Analysen viste, at Matter-protokollen stiller betydelige krav til integration og certificering, hvilket ville øge projektets kompleksitet væsentligt. Erfaringen viste derfor, at et eksklusivt fokus på Thread var mest realistisk for et stabilt udviklingsforløb med fokus på kommunikationsformen.



Figur 1: De første tanker debatteres

Fokus blev derfor udelukkende på Thread, som er en mesh-protokol, som muliggør lavenergi og solid kommunikation mellem enheder. En væsentlig del af arbejdet bestod i at forstå protokollens opbygning og etablere udviklingsmiljøer i nRF Connect SDK til Nordic-enheder samt ESP-IDF til implementering af en Thread Border Router. Valget faldt på at anvende et sample og så videre udvikle på dette, da dette projekt ville omhandle integration af to RTOS stacks og det blev anerkendt at en opbygning af disse fra bunden ville kræve en erfaring og længere tids arbejde med ZephyrRTOS og FreeRTOS, som er ESP-IDF real time operativsystem.

I den indledende fase blev det antaget, at enhedernes Thread-roller (Leader/Router/End Device) skulle defineres og styres eksplicit fra Border Routeren som en del af gateway-logikken. Senere i forløbet blev det afklaret, at rollefordeling og routing i Thread i stedet håndteres autonomt i mesh-netværket. Netværket etablerer selv ruter og vælger roller dynamisk baseret på link-kvalitet og afstandsmetrikker, via mekanismer der følger principper svarende til Bellman-Ford distance-vektor routing. På den baggrund blev det tydeligt, at Border Routerens primære funktion er at forbinde Thread-netværket til IP/LAN, mens selve rollefordelingen i Thread ikke kræver manuel konfiguration.

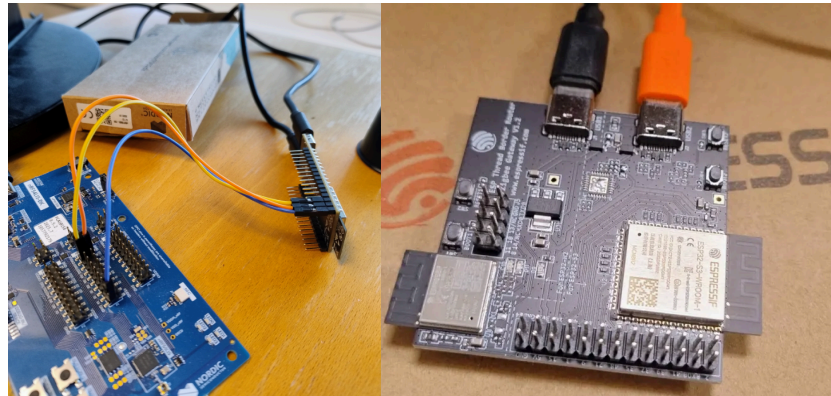


Figur 2: Første udkast til simulatoren

Opsætning og monitorering af en router kan på et senere tidspunkt indgå som en del af testforløb, hvor korrekt afstand, placering og opsætning af et Full Thread Device (FTD) verificeres. En sådan opsætning blev forsøgt med succes senere i forløbet, men blev ikke dokumenteret i rapporten ud over en simulering.

Den oprindelige hardware design tanke var at anvende en ESP32-C6 med en nRF-baseret Radio Co-Processor, men denne opsætning blev henlagt efter længere tids forsøg på grund af mangelfuld dokumentation og kompatibilitetsproblemer, særligt i spinel

kommunikation mellem Wi-Fi enheden og RCP på borderrouteren. Erfaringerne førte til valget af en ESP32-S3-H2, som udgør en veldokumenteret og stabil løsning med integreret understøttelse af Thread og Wi-Fi.

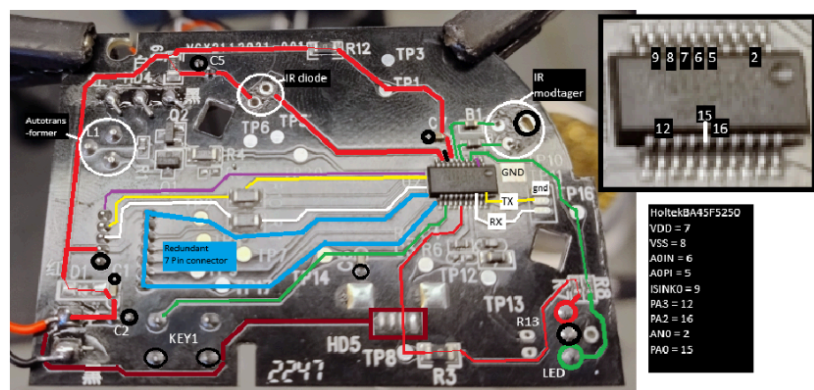


Figur 3: Border Router ESP32C6/nRF52840 vs ESP32S3H2

Parallelt blev et eksisterende alarmsystem analyseret for at kortlægge dens fysiske opbygning og funktionsprincipper.

Undersøgelserne omfattede komponenter som piezo elementer, LED-indikatorer og mikrocontrollere, herunder signalveje fra Holtek- til Tuya-baserede kredsløb. Analysen viste, at

nRF54L15 rummer potentiale til på sigt at fungere som den eneste hardwarekomponent i en alarmenhed. For at afgøre, om chippen faktisk kan bære hele løsningen selv, kræves dog langt mere detaljerede undersøgelser og praktiske tests. Det arbejde ligger uden for rammen af dette projekt, men perspektivet er teknisk interessant og peger på en mulig retning for fremtidige iterationer.



Figur 4: Kredsløbsanalyse af færdiglavet alarm

Den samlede researchfase bidrog til at skabe et teknisk grundlag og en klar afgrænsning af projektets omfang. Fravalget af Matter muliggjorde et fokuseret udviklingsforløb, hvor ressourcerne blev rettet mod at opnå en dybere forståelse af Thread-protokollen og fokus ændrede sig til udelukkende at indeholde kommunikationen i et simuleret Thread-Mesh netværk for en alarm, for senere at kunne implementere andre funktionelle enheder som sensorer, kameraer og andre IoT-enheder.



3. Indledning

Udviklingen inden for trådløs IoT-kommunikation har de seneste år gjort det muligt at tænke brandsikring på nye måder, særligt i ældre bygninger hvor traditionelle installationer kan være upålidelige eller teknisk vanskelige at eftermontere. Thread-protokollen og moderne microcontroller teknologi åbner op for en mere fleksibel tilgang, hvor batteridrevne enheder kan indgå i et selvorganiserende mesh-netværk med lavt energiforbrug og høj robusthed.

Dette projekt tager afsæt i netop denne mulighed: at undersøge, hvordan et trådløst, energieffektivt og skalerbart brandsikringssystem kan konstrueres på basis af OpenThread og udviklingsboards fra Nordic Semiconductor og Espressif. Målet er ikke at designe en færdig røgalarmsløsning, men at udvikle og demonstrere en teknisk prototype, der illustrerer de centrale mekanismer, som et fremtidigt system vil være afhængigt af – mesh-kommunikation, sikkerhed, web-baseret monitorering og lavenergidrift.

Indledningsvist sættes fokus på problemstillingen omkring brandsikring i bygninger opført før 2020, som i mange tilfælde ikke er opdateret til nutidens krav i BR18. Dette danner rammen for, hvorfor en trådløs og let-installerbar løsning kan være relevant, og hvordan Thread-protokollens egenskaber giver et potentielt match mellem teknologiske muligheder og praktiske behov.

Den prototype, der udvikles i projektet, består af en ESP32-baseret Border Router samt tre nRF54L15-enheder konfigureret som sleep end-devices. Systemet demonstrerer grundlæggende funktioner som alarmhåndtering, netværksrouting, statusmonitorering og krypteret kommunikation. For at kunne evaluere løsningen er udviklingen koblet til en kravspecifikation og tilhørende tests, der undersøger robusthed, sikkerhed, web-interface og energiforbrug – de fire nøgleområder, som kan være afgørende for et trådløst brandsikringssystem i praksis.



4. Projektmanagement

I projektet er der anvendt en enkel form for projektstyring med udgangspunkt i en minimal WBS, som har fungeret som en overordnet ramme for struktur og opgavestyring. Fremgangsmåden har dog i høj grad været præget af en fleksibel tilgang, hvor projektet er blevet opdelt i mindre dele, der hver især er blevet behandlet med fokus på test, vidensindsamling og konklusion, før næste del blev påbegyndt. Udviklingsprocessen har fulgt en iterativ metode, hvor der løbende er blevet bygget videre på funktionaliteten efter hver vellykket test.

Da projektet primært består af software og kun i begrænset omfang involverer hardware, har fokus været rettet mod udvikling og afprøvning i to forskellige softwaremiljøer. Gruppen bestod af to personer, og der har derfor ikke været en fast projektlederrolle. Beslutninger er i stedet truffet i fællesskab gennem løbende dialog og samarbejde. Arbejdet er foregået gennem fysiske møder, hvor opgaver er blevet drøftet og fordelt, og der er løbende blevet tilføjet opgaver og status i gruppens Excel-baserede WBS.

Udviklingen har været baseret på løbende test og tilpasning, hvor hver ny funktion blev afprøvet og justeret, inden næste del blev implementeret. For at sikre fremdrift og prioritering har gruppen arbejdet ud fra princippet "need to have" og "nice to have", hvor hovedfokus har været at løse den primære problemstilling, mens ekstra funktioner kun blev tilføjet, hvis der var tid og ressourcer til det. Enkelte funktioner er blevet testet separat uden for det samlede system for at lette fejlretning og integration.

Opgave	Prioritet	Status	Startdato	Slutdato	Noter
Statusmøde	Normal	Udført	20/10/2025	20/10/2025	Rapportstruktur/opsætning
Rapportskrivning	Normal	I gang	20/10/2025	05/12/2025	General rapportskrivning
Finpudsning	Lav	Endnu ikke startet	05/12/2025	18/12/2025	Finpudsning af rapport
Statusmøde	Normal	Udført	27/10/2025	27/10/2025	Inddeling af arbejdsopgaver
BR18 afsnit	Normal	I gang	03/11/2025	03/11/2025	Kort afsnit omkring BR 18
Sikkerheds afsnit	Normal	I gang	23/10/2025		DTLS, AES128, HTTPS
Funktionstest	Høj	I gang	20/10/2025		Afprøvning af funktioner
Statusmøde	Normal	Udført			Vidensdeling

Figur 5: WBS struktureret omkring den itative proces

https://docs.google.com/spreadsheets/d/1QlwkvuJKa43RE_CNyxv6lv_lzqzgedPSDzCAXq6-mM/edit?usp=sharing

5. Design

Design afsnittet danner grundlaget for den efterfølgende udvikling, test og implementering af det simulerede røgalarmsystem. Her præsenteres de overordnede designovervejelser, som ligger til grund for den endelige kommunikationssimulator. Afsnittet indeholder et blokdiagram med kommunikationsbeskrivelse og beskrivelser af de tekniske og funktionelle dele af WEB-GUI.

Formålet med designafsnittet er at give et billede af udviklingstanker og valg, der er truffet i projektet inden implementeringen, for at give et fundament, inden der påbegyndes fysiske tests og arbejdet hen mod et færdigt produkt.

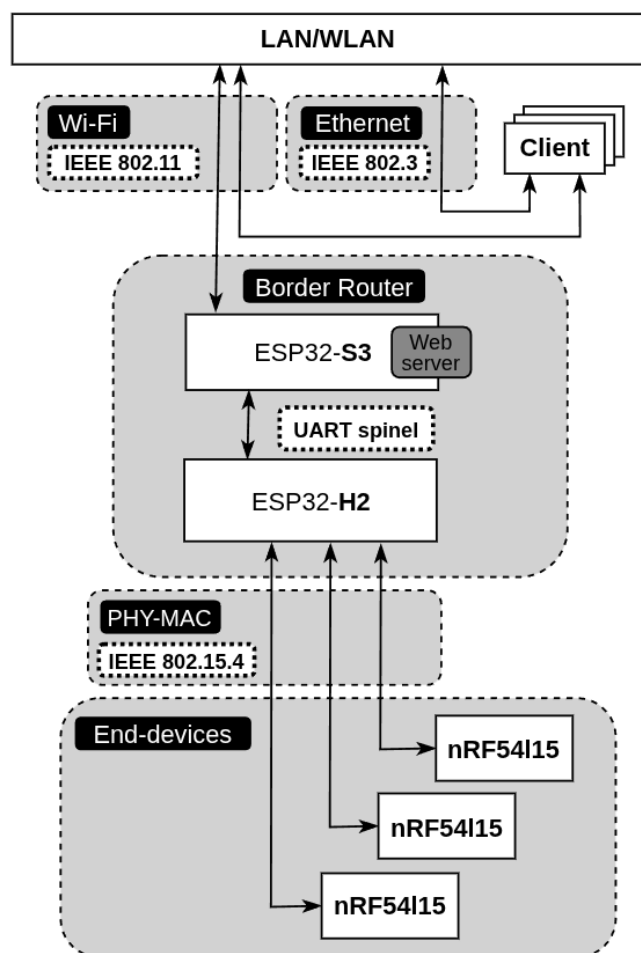
5.1 Blokdiagram

Thread-protokollen i simulatoren beskrives her for at give et indblik i logikken bag kommunikationsformen og et tydeligere overblik over de processer, den gennemgår i kommunikationen. Valget af udelukkende denne protokol blev, som tidligere nævnt, truffet for at reducere kompleksiteten og sikre en mere dybdegående forståelse.

Simulatoren består af en Thread-Border-Router baseret på ESP32-S3 og ESP32-H2, som forbinder et lokalt Thread-mesh-netværk og end-devices bestående af tre nRF54l15. Arkitekturen er opbygget som en kommunikationskæde, hvor hver komponent udfører en afgrænset funktion i dataoverførslen mellem end-devices og det private netværk.

Wi-Fi-forbindelsen fungerer som backhaul-link mellem Thread-netværket og LAN/WLAN. Forbindelsen muliggør adgang til den integrerede webserver på S3. Wi-Fi kommunikationen er baseret på IEEE 802.11¹ og understøtter både IPv4 og IPv6. Endvidere kan LAN/WLAN tilgås via IEEE 802.3² ethernet, som client.

S3 fungerer som gateway og hostprocessor for Thread Border Router. Den håndterer IP-trafik, NAT64, præfikshåndtering og routing mellem



Figur 6: Blokdiagram af Thread simulator

¹ Geeksforgeeks (23.07.2025)

² Cisco - Ethernet (IEEE 802.3) (20.08.2012)



Wi-Fi og Thread³. S3 kører OpenThread og styrer samspillet med H2. Enheden fungerer som netværksmæssig grænseflade mellem høj-båndbredde Wi-Fi og lav-energi Thread. Derudover indeholder S3 simulatorens HTTPS webserver.

S3 og H2 kommunikerer via et UART-interface⁴, hvor spinel-protokollen anvendes til styring og dataudveksling. Forbindelsen opererer ved 115.200 baud, 8N1 uden flow-control⁵. Spinel definerer de kommandoer og egenskaber, som hosten S3 kan læse og skrive i H2, f.eks. kanalvalg, netværksnøgle og dataframes. Kommunikationen er rammebaseret, hvor UART overfører binære spinel-pakker. Dette interface gør det muligt for S3 at styre radiooperationer og modtage trådløse datapakker fra Thread-netværket.

H2 fungerer som en dedikeret RCP i systemet og håndterer det fysiske lag (PHY) og datalink-laget (MAC) for IEEE 802.15.4. Enheden kører OpenThread RCP-firmware, som implementerer radio- og MAC-funktionerne. H2 står for håndtering af Neighbor Discovery - MLE, Routing - RLOC16, kryptering via AES-128, samt datagram-transport - UDP/IPv6. H2 udgør Threads radio-motor, som sender og modtager trådløse frames fra netværkets end-devices.

Den trådløse kommunikation mellem H2 og nRF54L15, bygger på IEEE 802.15.4. Standarden opererer i 2,4-GHz-båndet og anvender Offset QPSK-modulation - Offset Quadrature Phase Shift Keying kombineret med DSSS - Direct Sequence Spread Spectrum - se bilag 10.5, hvilket giver lavt strømforbrug og høj robusthed mod interferens. IEEE 802.15.4 tilbyder 16 kanaler og en datarate på 250 kbps. Som PHY/MAC-laget leverer det rammestruktur, kanaladgang via CSMA-CA og kvitteringsmekanismer (ACK), hvilket udgør det tekniske fundament, som Thread-protokollen bygger videre på. nRF54L15 fungerer som end-device i Thread. Enheden udveksler applikationsdata via OpenThread og UDP/IPv6 med AES-128 kryptering. Den kommunikerer trådløst med Border Routeren gennem 802.15.4-radioen og indgår i mesh-topologien som en lav-energi sensor- eller aktuator-enhed. end-devices benytter Thread-protokollens MLE - mesh link establishment til at opretholde forbindelse og synkronisering med Border Routeren.

³ **ESP Thread Border Router SDK - 3.0** (02.04.2025)

⁴ **ESP H2 - OpenThread** (23.04.2023) og **Hui, Jonathan** (18.01.2019)

⁵ **Geelen, Pieter** (11.07.2025) - Key Characteristics of IEEE 802.15.4

5.2 WEB-GUI design

For at sikre høj brugervenlighed i systemet er alle centrale funktioner tilgængelige gennem et webbaseret GUI. Brugerfladen giver mulighed for at overvåge enhedernes status, udføre tests og sende kommandoer til netværket. WEB-GUI fungerer som et kontrolpanel, hvor alle handlinger initieres via en HTTP-forespørgsel til border routeren. Border Routeren omsætter HTTP-forespørgslerne til UDP-pakker, som distribueres til de enkelte end-devices.

Kommunikationen er baseret på et request/response-princip: Når brugeren aktiverer en funktion i GUI, sendes en HTTP GET-anmodning til border routeren. Denne videregiver en tilhørende UDP-pakke til en eller flere enheder. Hver enhed udfører derefter den relevante funktion og returnerer en statusbesked til border routeren, der derefter opdaterer GUI. På denne måde sikres det, at kommandoer er modtaget og udført, samtidig med at status altid afspejler den aktuelle tilstand i systemet.

GUI indeholder flere funktioner, hver med en specifik rolle i forhold til systemets drift, overvågning og fejlfinding. Nedenfor gennemgås de vigtigste funktioner og deres formål.

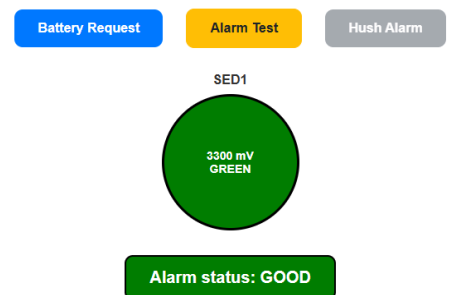
5.2.1 Battery request

Da systemet er designet til at indgå i batteridrevne røgalarmer, er det afgørende, at der løbende kan monitoreres batterispænding for at sikre rettidig udskiftning. Battery Request-funktionen håndterer denne opgave. Når brugeren aktiverer Battery Request i WEB-GUI, sendes der en HTTP GET-forespørgsel til border routeren, som efterfølgende udsender en UDP-pakke til målrettede enheder.

Når en røgalarms-enhed modtager Battery Request-pakken, foretager den en måling af batteriets spænding via microcontrollerens ADC-indgang. Når målingen er fuldført, returnerer enheden den målte spænding samt relevante status informationer til border routeren via UDP. Herefter opdateres GUI, så visuelle indikatorer i form af farvede cirkler afspejler den nye batteristatus.

Hvis batteriet fjernes eller er afkoblet, vil ADC typisk registrere 0 mV eller en meget lav spænding, hvilket vil resultere i, at GUI viser en rød indikator med status "No battery". Dette giver brugeren en tydelig feedback om, at enheden ikke længere har forsyningsspænding.

Thread Network Monitoring Dashboard



Figur 7: WEB GUI

```
UDP_BR: Battery request sent to child RLOC16=0x7c04
UDP_BR: Battery request sent to child RLOC16=0x7c04
UDP_BR: UDP received: { "voltage": 3300, "status": "GREEN", "mleid": "fdc2:205f:9288:9e0a:51dc:e4a0:5886:bca2", "rloc16": "0x7c04", "extaddr": "f4:ce:36:25:51:9f:81:dc" }
UDP_BR: Updated SED1: 3300 mV, GREEN, MLEID=fdc2:205f:9288:9e0a:51dc:e4a0:5886:bca2, RLOC16=0x7c04, ExtAddr=f4:ce:36:25:51:9f:81:dc, LastSeen=1515870811
UDP_BR: Battery request sent to child RLOC16=0x7c04
```

Figur 8: Border Router sender et payload - BREQ



5.2.2 Alarm Test

Alarm Test-funktionen gør det muligt at udføre en funktionstest af en enhed uden at udløse en fuld alarmtilstand. Når brugeren trykker på Alarm Test i GUI, genererer systemet en HTTP GET-forespørgsel til border routeren, som sender en test-kommando via UDP til alle enheder.

Ved modtagelse af test-kommandoen aktiverer enheden en indikatorlampe (eller anden visuel markør), som lyser i 10 sekunder. Dette repræsenterer en simuleret alarm, men uden lyd giver eller andre kritiske funktioner. Testperioden på 10 sekunder sikrer, at enheden fungerer korrekt, samtidig med at testen er kortvarig og ikke forstyrrer omgivelserne unødigt.

Denne funktion er primært tiltænkt installations- og vedligeholdelsesfasen, hvor teknikere hurtigt skal kunne verificere, at enhederne responderer korrekt på netværkskommandoer.

5.2.3 Hush alarm

Hush Alarm-funktionen bruges til at stoppe en aktiv alarm på en enhed. Funktionen er relevant i situationer, hvor en alarm er blevet udløst fejlagtigt, eksempelvis som følge af madlavning eller røg uden reel brandfare. I sådanne tilfælde giver hush-funktionen brugeren mulighed for at deaktivere alarmen direkte via GUI.

Knappen er kun aktiv, når GUI har registreret, at mindst én enhed befinder sig i en aktiv alarmtilstand. Når brugeren aktiverer hush-funktionen, sender GUI en HTTP GET-forespørgsel til border routeren, som udsender en hush-kommando via UDP til alle enheder med aktiv alarmstatus. Under udførelsen skifter knappen til en ventetilstand, hvor systemet afventer bekræftelse fra alle berørte enheder om, at alarmen er stoppet.

Det er vigtigt at bemærke, at Hush Alarm ikke påvirker Alarm Test-funktionen, da test-alarmer automatisk stopper efter 10 sekunder og derfor ikke kræver manuel afbrydelse.

6. Software development

Dette afsnit introducerer det tekniske grundlag for projektets kommunikationsmodel og beskriver, hvordan Thread-protokollen og de underliggende softwarelag realiserer systemets funktionalitet. Først præsenteres den logiske opbygning af et Thread-netværk, herunder principperne for mesh-kommunikation, rollefordeling og håndtering af lavenergi-enheder. Formålet er at etablere en forståelsesramme for de mekanismer, som gør det muligt for enhederne at kommunikere robust og energieffektivt uden en central router.

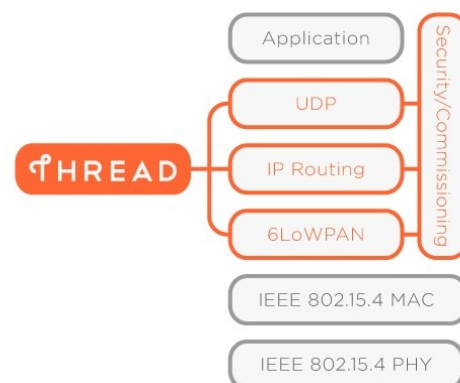
Herefter gennemgås den tilhørende softwarestack, bestående af OpenThread, Zephyr og ESP-IDF/FreeRTOS. Disse lag udgør tilsammen den operationelle platform, der muliggør både etablering af Thread-netværket og integrationen mellem nRF-baserede sensorenheder og ESP32-baseret Border Router. Gennemgangen viser, hvordan netværkskommunikation, ressourcestyring og applikationsfunktioner fordeles mellem lagene, samt hvordan disse komponenter samarbejder for at skabe et samlet, driftssikkert system.

Afsnittene 6.1 - 6.4 danner det metodiske og arkitektoniske udgangspunkt for de efterfølgende beskrivelser af det embedded software hierarki i afsnit 6.5.

6.1 OpenThread

OpenThread⁶ er en open source-implementering af Thread-protokollen, som er udviklet af Google. Thread er en lavenergi, trådløs mesh-netværksprotokol, designet specifikt til IoT-enheder. Protokollen gør det muligt for enheder at kommunikere direkte med hinanden uden behov for en central router, hvilket øger robustheden og pålideligheden af netværket. Hvis én enhed mister forbindelsen, kan data automatisk finde en alternativ rute gennem andre noder i netværket.

Tilgangen til OpenThread har været at bruge nogle af de smarte funktioner, der kommer med Thread protokollen så som rollefordeling, udp og sikkerhed. I et Thread netværk inddeles de enheder som der er en del af netværket i forskellige roller som har adgang til forskellige funktioner af Thread protokollen. De roller der er fokus på i dette projekt er leader, router og sleep end-devices.



Figur 9: OpenThread stack

Leader

Lederrollen fungerer som netværkets centrale styringsenhed. Den enhed, der opretter Thread-netværket, tildeles automatisk lederrollen og får dermed ansvaret for at administrere og koordinere netværket.

⁶ OpenThread Documentation - <https://openthread.io/>



For at kunne fungere som leder skal enheden være et Full Thread Device (FTD) – det vil sige, at den understøtter alle funktioner i Thread-protokollen. Et FTD har altid sin radio aktiv, så den kan modtage, videresende og styre al kommunikation i netværket.

Lederens opgaver omfatter blandt andet at tildele adresser, opretholde routing-tabeller, og holde styr på netværkets topologi, herunder hvilke enheder der fungerer som routere og end-devices. Der kan kun være én leder ad gangen i et Thread-netværk, men hvis lederen mister forbindelsen eller fejler, kan en router-enhed automatisk overtage lederrollen, så netværket fortsætter med at fungere uden afbrydelser.

Router

Router-rollen fungerer som en hjælpe rolle til lederrollen (Leader). Routerens primære opgave er at videresende og route data mellem andre enheder i netværket samt kommunikere med lederen. Router-rollen bruges til at udvide netværkets rækkevidde og udnytte fordelene ved mesh-netværk.

Man kan sammenligne routeren med en repeater, da den kan gentage og videresende beskeder, så signalet kan nå længere ud i netværket. Ud over at øge rækkevidden fungerer router-enheder som mellemlid i kommunikationen mellem end-devices og lederen.

Når en enhed skal sende en besked til lederen, vurderer den først, om den har direkte forbindelse. Hvis ikke, sendes beskeden via den nærmeste eller mest effektive router. Routeren sørger derefter for at føre beskeden videre mod lederen gennem det mest optimale kommunikationsled.

Fordelen ved et mesh-netværk er, at det øger driftsikkerheden og robustheden. Hvis en router-enhed går ned, kan trafikken automatisk omdirigeres gennem en anden router. Dermed kan kommunikationen fortsætte uden afbrydelser, hvilket gør netværket mere stabilt og pålideligt.

Sleep end-device

En end-enhed, også kaldet et end-device, er den mest simple enhedstype i et Thread-netværk. Den kommunikerer kun gennem en tilknyttet router og har ikke selv routing-ansvar. Det betyder, at slut-enheden ikke videresender data for andre enheder, men udelukkende sender og modtager sine egne beskeder via routeren.

End-devices anvendes ofte i lavenergi-applikationer, da de kan slukke deres radio, når der ikke skal kommunikeres, hvilket reducerer strømforbruget markant. De findes i to varianter:

- Full end-device (FED) – har sin radio tændt hele tiden og kan modtage beskeder når som helst.
- Minimal end-device (MED) eller Sleep end-device (SED) – slukker sin radio i perioder for at spare strøm og vågner i perioder for at sende eller modtage data.

End-devices er typisk sensorer, aktuatorer eller andre små IoT-enheder, der kun behøver at kommunikere med netværket lejlighedsvis. Denne rolle gør det muligt at opbygge energieffektive og fleksible mesh-netværk, hvor routere og lederen håndterer netværkets tunge trafik, mens end-devices fokuserer på deres specifikke funktion. Projektet fokuserer på

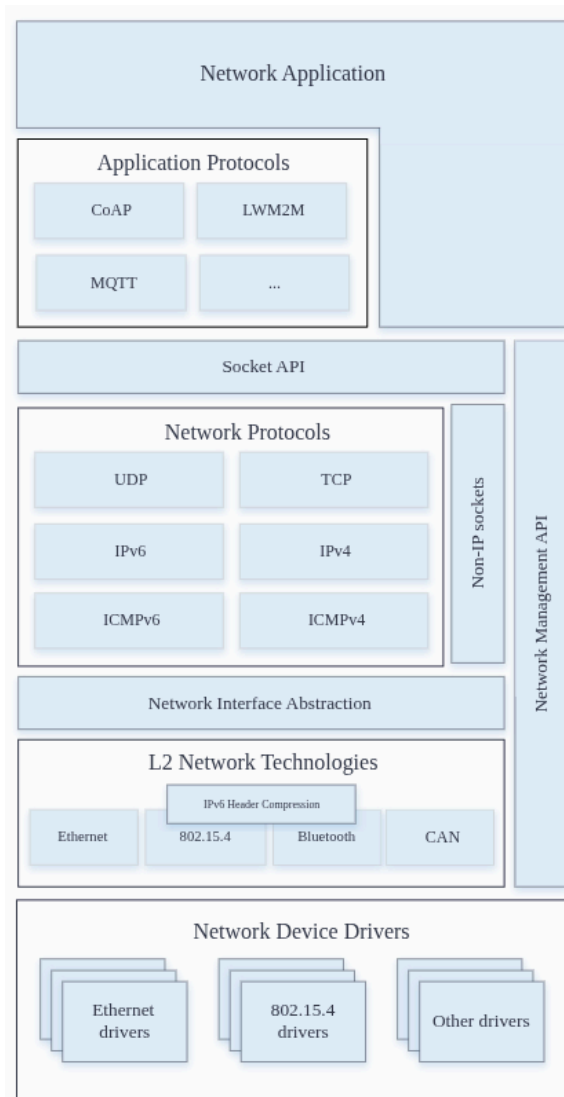
brugen af sleep end-devices, da disse er designet til at fungere som lavenergi-enheder. Enheden arbejder med en sleep-cyklus på 10 sekunder, hvilket betyder, at radioen er slukket i denne periode for at spare strøm. Efter 10 sekunder vågner enheden kortvarigt for at lytte, om der er indgående beskeder. Hvis der ikke modtages nogen data, går den straks tilbage i dvale. Denne metode sikrer et meget lavt energiforbrug, hvilket gør den ideel til batteridrevne IoT-enheder.

6.2 Zephyr

Zephyr⁷ er et open source realtimeoperativsystem (RTOS) udviklet til indlejrede systemer og IoT-enheder. Det bruges til at styre og koordinere funktionerne på en microcontroller, såsom trådstyring, hukommelse og kommunikation med hardware. Zephyr er designet til at være letvægts, energieffektivt og modulært, hvilket gør det velegnet til små enheder med begrænsede ressourcer.

Zephyr understøtter en lang række hardwareplatforme – herunder Nordic Semiconductors nRF-serie – og gør det muligt at integrere forskellige netværksprotokoller, som for eksempel OpenThread. I den sammenhæng fungerer Zephyr som det styresystem, der håndterer enhedens ressourcer, mens OpenThread varetager selve netværkskommunikationen.

I projektet anvendes Zephyr desuden til at oprette og håndtere UDP-sockets, som bruges til at sende og modtage data mellem enhederne i Thread-netværket. Ved hjælp af Zephyrs netværks-API'er kan systemet åbne UDP-porte, sende datapakker og lytte efter beskeder på tværs af netværket. Dette gør Zephyr til et central komponent i kommunikationen, da det binder applikationslaget sammen med OpenThread-netværket.



Figur 10: Zephyr stack

⁷Zephyr dokumentation

<https://docs.zephyrproject.org/latest/connectivity/networking/net-stack-architecture.html>

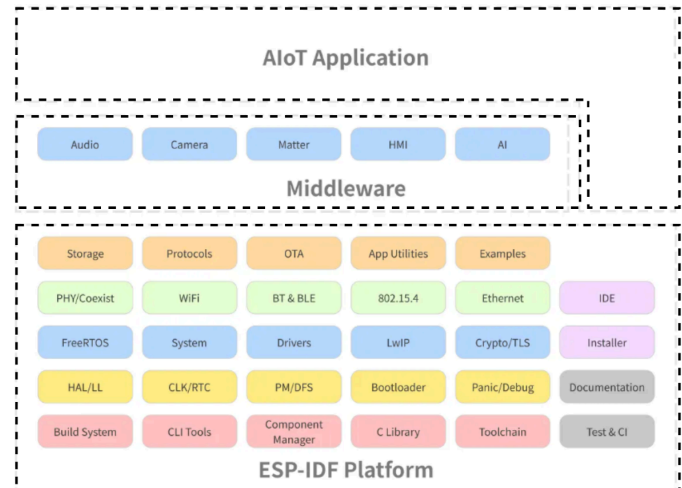
Kombinationen af Zephyr og OpenThread gør det muligt at opbygge et trådløst mesh-netværk, hvor Zephyr håndterer styringen af enheden og kommunikationen gennem UDP, mens OpenThread står for selve forbindelsen mellem enhederne.

6.3 FreeRtos/IDF

ESP-IDF⁸ (Espressif IoT Development Framework) er Espressifs officielle udviklingsmiljø og softwareplatform til deres microcontrollere. Frameworket bygger på FreeRTOS og indeholder en lang række biblioteker, drivere og netværksstakke, som gør det muligt at udvikle komplekse IoT-løsninger med både trådløs kommunikation og applikationslag.

I projektet anvendes ESP-IDF til at køre OpenThread Border Router implementeringen på et board bestående af en ESP32-H2 og en ESP32-S3. H2'en håndterer Thread-kommunikationen via OpenThread, mens S3'en fungerer som grænseflade til Wi-Fi og webserver. Denne kombination gør det muligt at forbinde Thread-netværket til et IP-baseret netværk og samtidig give adgang til en webbaseret brugerflade til monitorering og test.

ESP-IDF sørger i denne opsætning for at koordinere samspillet mellem de to controllere, styre tråde og netværksressourcer samt håndtere kommunikationen mellem Thread-netværket og webserveren. Frameworket gør det dermed muligt at opbygge en stabil og integreret Border Router-løsning, der kan forbinde og visualisere data fra Thread-enhederne.



Figur 11: ESP-IDF - FreeRTOS

⁸ FreeRtos/IDF dokumentation

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html



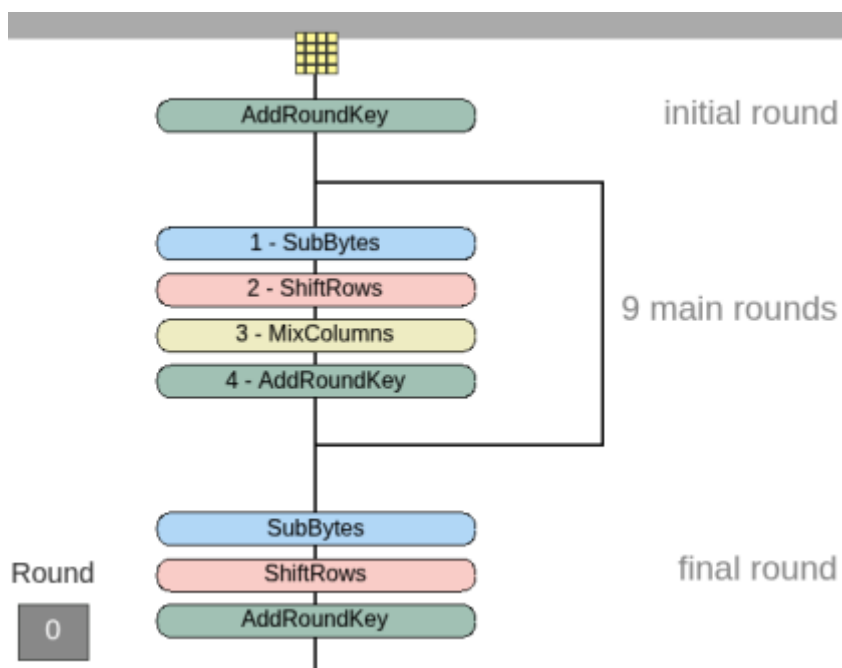
6.4 DTLS - Datagram transport layer security

For at beskytte kommunikationen mellem enhederne i Thread-netværket anvendes AES-128-baseret kryptering⁹ - Advanced Encryption Standard, som er en internationalt anerkendt og gennemtestet symmetrisk krypteringsmetode. AES-kryptering fungerer ved, at både afsender og modtager benytter den samme hemmelige nøgle til at kryptere og dekryptere data. Metoden behandler data i 128-bit blokke og anvender et nøglelængdeprincip, hvor sikkerhedsniveauet øges i takt med nøglens størrelse.

AES-128 anses som det mest anvendte kompromis mellem høj sikkerhed og lavt ressourceforbrug. Denne variant er velegnet til systemer som trådløse sensornetværk, hvor datamængden er begrænset, og hvor lav latenstid og energieffektivitet er afgørende. AES-128 giver en robust beskyttelse mod brute-force-angreb, da en 128-bit nøgle teoretisk har 2^{128} mulige kombinationer, hvilket gør et komplet gæt-angreb praktisk umuligt med moderne computerkraft.

Ved nærmere undersøgelse viser det sig, at AES kan brydes ved hjælp af kendte angrebsmetoder såsom power analyse, cache-timing, side-channel og elektromagnetisk analyse, ved at få adgang til fx en SED-enhed. AES kryptering er grundigt beskrevet i bilag 10.6. Det er derfor vigtigt at have dette for øje i udvikling og endelig produktion.

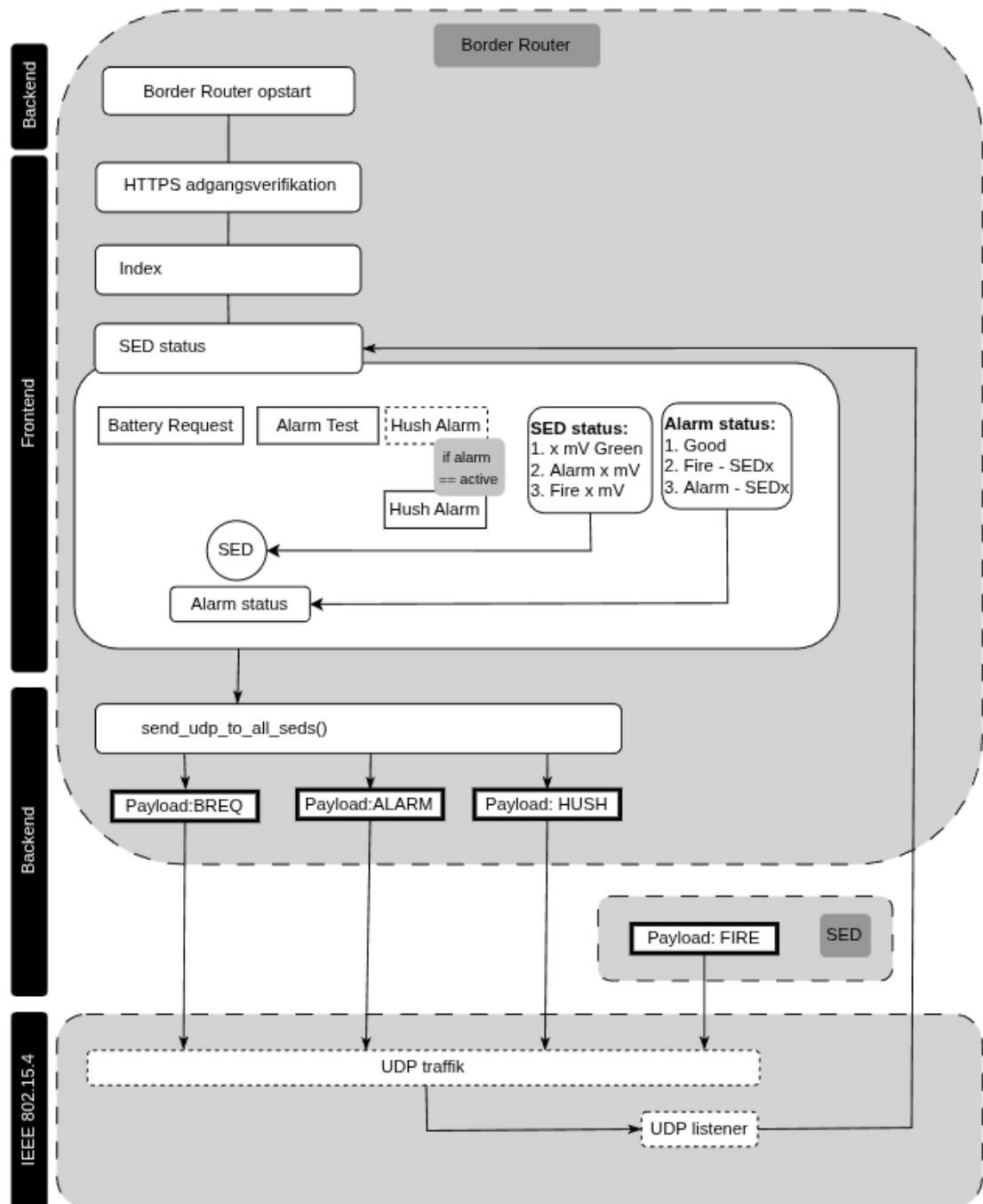
Beskrivelse af mulige angrebstyper se bilag 10.13



Figur 12: AES enkryptering

⁹ AES128 - Simulering "CrypTool-Online" <https://legacy.cryptool.org/en/cto/aes-animation>

6.5 Embedded software hierarki diagram



Opstartsflowet for Border Router, adgang til webserveren, rollefordelingen mellem komponenterne samt håndtering af advisering beskrives i bilag 10.7. Selve opstartssekvensen for Border Routeren er udeladt fra dette afsnit, da den bygger på Espressif ESP-IDF og FreeRTOS og er allerede implementeret i border routerens sample kode. Flowet er dog væsentligt for udviklere at forstå, da det danner grundlag for kommunikationen mellem RCP, NCP og den overliggende webserverlogik, samt for korrekt initialisering af Thread-netværket.



Diagrammet viser hele kommunikationsflowet mellem Frontend, Backend og IEEE 802.15.4 laget. *Frontend* er alt det, der lever inde i `esp_br_web.c` og de resource handlers der er deklareret. *HTTPS adgangsverifikation* Webserveren kører HTTPS og bruger servercert + key fra SPIFFS. Brugeren skal ind via SSL verificering før de kan få adgang.

Index er den almindelige hjemmeside, hvor alle knapper og statusfelter ligger.

SED Status viser status på alle sleep end-devices i WEB-GUI. Det bliver opdateret via UDP-svar, som webserveren modtager i baggrunden.

Figur 13: Embedded software hierakidiagram

Battery Request, Alarm Test og Hush Alarm er WEB-GUI knapper. Når der trykkes på en knap, kaldes en HTTP_GET-handler som igen kalder `send_udp_to_all_seds()` i backend.

Hush har en lille note: "if alarm == active". Den er altså kun relevant når mindst én SED har meldt ALARM.

Frontendens job er at modtage brugerinput, vise den nyeste status og sende HTTP GET requests til backend.

Backend – HTTPS-handlere og UDP-funktionen Backend består af `esp_br_web.c` - handlers og `udp_br_func.c` - d `send_udp_to_all_seds()`

Når en knap i frontend udløses, sker dette:

1. Handleren bliver aktiveret.
2. Handleren tjekker, at OpenThread-instansen kører.
3. Handleren kalder `send_udp_to_all_seds(instance)`.
4. `send_udp_to_all_seds()` bygger en UDP socket.
5. Den finder alle SEDs i netværket
6. Den sender den passende payload, f.eks:
 - BREQ (Battery Request)
 - ALARM (Alarm Test)
 - HUSH (Stop alarm)

Backendens job er at oversætte web-knap - UDP-kommando, udsende kommandoen til alle SEDs og Modtage data retur via UDP listeneren.

IEEE 802.15.4 - Bundlaget er selve Thread-netværket, der transporterer pakkerne mellem Border Router og SED. *UDP trafik*: Her sendes selve pakkerne på IPv6 over 802.15.4. *UDP listener*: Når SED'er svarer tilbage, fanger denne listener dem og sender resultatet videre til frontend for opdatering. Frontendens SED Status-felt opdateres altså kun når listeneren modtager data fra SED'erne.

Sleep end-devices – Modtager og udfører kommandoer. Selvom SED ikke er med i diagrammet, er flowet tydeligt:

1. BR sender UDP payload til SED.
2. SED vågner styret af poll period.



3. SED udp_server_thread() modtager pakken.
4. Beslutning træffes:
 - BREQ → Mål batteri → send JSON tilbage.
 - ALARM → Kør alarm_test_trigger().
 - HUSH → Stop alarm → send HUSH_ACK.
5. SED sender svar tilbage til Border Routerens UDP listener.

6.5.1 Border router

I projektet har vi valgt at tage udgangspunkt i den eksisterende ESP32 Border Router sample-kode frem for at udvikle en fuldstændig egen implementation. I den indledende fase blev der forsøgt at oprette en border router-løsning selvstændigt, men oplevelsen gav udfordringer med kompatibiliteten mellem ESP32-C6 og vores nRF-baserede enheder, som tidligere beskrevet, særligt i forhold til OpenThread-stakken og oprettelse af stabile forbindelser. For at sikre en robust og driftssikker løsning blev valget derfor at anvende det officielle sample som fundament, så fokus kunne rettes mod at implementere de funktionelle dele af systemet.

esp_br_web.c

I ESP32 OpenThread-samplet indgår en komplet webserver, som kan køre parallelt med border routerens funktioner. Webserveren håndteres af filen *esp_br_web.c*, som fungerer som indgangspunkt for serverlogikken. Filen er ansvarlig for håndtering af statiske filer, routing af HTTP-forespørgsler og behandling af GET-endpoints.

Da samplet indeholdt en række funktioner, vi ikke havde brug for, blev webserveren tilpasset til vores behov. Unødvendige endpoints, statiske ressourcer og template-funktioner blev fjernet, så kun de funktioner, som indgår direkte i vores system, blev bevaret. Til gengæld blev egne funktionskald integreret i serverstrukturen, så GUI kunne kommunikere direkte med systemets backend-funktionalitet.

Opsætning af resource handlers

Her oprettes de HTTP_GET-endpoints, som WEB-GUI gør brug af. Hvert endpoint kobles til en specifik handlerfunktion – eksempelvis til Battery Request, Device Info eller statusopdateringer.

```
// Funktionserklæring (prototype) for handleren,  
// som håndterer HTTP GET-forespørgsler til batterianmodning  
static esp_err_t battery_request_handler(httpd_req_t *req);  
  
// Funktionserklæring for handleren,  
// som håndterer HTTP GET-forespørgsler til batteristatus  
static esp_err_t battery_status_handler(httpd_req_t *req);  
  
// Funktionserklæring for handleren,  
// som håndterer HTTP-forespørgsler relateret til enheden  
static esp_err_t device_handler(httpd_req_t *req);  
  
// Opretter et array af URI-handlere til HTTP-serveren  
static httpd_uri_t s_resource_handlers[] = {  
  
    {
```



```
// URL-stien, som klienten skal kalde (fx http://ip/battery_request)
.uri = "/battery_request",

// HTTP-metoden der accepteres (GET-request)
.method = HTTP_GET,

// Funktion der kaldes, når URI'en og metoden matcher
.handler = battery_request_handler,

// Brugerdefineret kontekst (ikke brugt her, derfor NULL)
.user_ctx = NULL,
},
};
```

Disse resource handlers sikrer, at når brugeren aktiverer en knap i GUI, modtager webserveren en HTTP GET-forespørgsel, som derefter videresendes til den relevante funktion på border routeren.



Handler funktion eksempel

Handlerfunktionerne udgør forbindelsesleddet mellem webserveren og de funktioner, der udfører den reelle kommunikation i systemet. Hver handler validerer først, om OpenThread-instansen er aktiv. Hvis den ikke er tilgængelig, returneres en HTTP-fejl til klienten. Hvis instansen findes, kaldes den interne funktion, som fx sender UDP-beskeder ud til enhederne.

Efter udførelsen sender handleren et HTTP-svar tilbage i JSON-format, hvilket gør det nemt for WEB-GUI at opdatere brugergrænsefladen.

```
// HTTP-handler der håndterer en battery_request GET-forespørgsel
static esp_err_t battery_request_handler(httpd_req_t *req)
{
    // Henter den aktive OpenThread-instans fra ESP-IDF
    otInstance *instance = esp_openthread_get_instance();

    // Tjekker om OpenThread-instansen findes
    if (!instance) {
        // Sender en HTTP 500 fejl tilbage til klienten,
        // hvis OpenThread ikke er initialiseret korrekt
        httpd_resp_send_err(
            req,
            HTTPD_500_INTERNAL_SERVER_ERROR,
            "No OpenThread instance"
        );
        // Returnerer fejlstatus til HTTP-serveren
        return ESP_FAIL;
    }

    // Sender en UDP-besked til alle Sleep end-devices (SEDs)
    // via OpenThread-netværket
    send_udp_to_all_seds(instance);

    // Sætter HTTP-responsens content-type til JSON
    httpd_resp_set_type(req, "application/json");

    // Sender et JSON-svar tilbage til klienten
    // som bekræfter at battery request er afsendt
    httpd_resp_sendstr(
        req,
        "{\"message\": \"Battery request sent\"}"
    );

    // Returnerer OK-status til HTTP-serveren
    return ESP_OK;
}
```



udp_br_func.c

For at beskrive kommunikationen dybere har vi analyseret og dokumenteret funktionen *send_udp_to_all_seds()*, som har en central rolle i grænsefladen mellem border router og de trådløse enheder.

Funktionens formål er at sende en UDP-pakke til samtlige sleep end-devices (SED), der er forbundet til OpenThread-netværket. SEDs er lavstrømsenheder, som kun er vågne i korte perioder for at modtage beskeder, hvilket gør kommunikationen mere kompleks, da beskeder skal sendes i et format og tidspunkt, hvor enheden kan modtage dem.

Funktionens arbejdsproces kan beskrives i følgende trin:

1. **Validering af OpenThread-instans**

Hvis instansen ikke findes, logges der en fejl, og funktionen afsluttes.

2. **Oprettelse af UDP socket**

Der oprettes en IPv6-baseret UDP socket, som bruges til at udsende beskederne.

3. **Hentning af børnedata (child nodes)**

Systemet henter det maksimale antal tilladte child devices via OpenThread.

4. **Iteration gennem alle child devices**

For hver child node forsøges der at hente detaljer via *otThreadGetChildInfoByIndex()*.

5. **Filtrering til SEDs**

Kun enheder som **ikke** er Full Thread Devices (FTD) behandles videre.

6. **Hentning af IPv6-adresse**

Når en SED er identificeret, forsøges det at hente dens næste IPv6-adresse via *otThreadGetChildNextIp6Address()*.

7. **Afsendelse af UDP-pakke**

UDP-beskeden sendes til enhedens IPv6-adresse og portnummeret defineret som `UDP_PORT_SEND`.

8. **Logning af resultat**

Ved succes logges RLOC16-adressen, og ved fejl logges errno.

9. **Lukning af socket**

Når alle enheder er behandlet, lukkes socket'en for at frigive ressourcer.



Udp sende funktion

```
// Funktion der sender en UDP-besked til alle Sleep end-devices (SEDs)
void send_udp_to_all_seds(otInstance *instance)
{
    // Tjekker om OpenThread-instansen er gyldig
    if (!instance) {
        // Logger en fejl, hvis OpenThread ikke er initialiseret
        ESP_LOGE(TAG, "No OpenThread instance available");
        return;
    }

    // Opretter en IPv6 UDP-socket til afsendelse af data
    int sock_tx = socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);

    // Tjekker om socket-oprettelsen fejlede
    if (sock_tx < 0) {
        // Logger fejl med errno-kode
        ESP_LOGE(TAG, "TX socket create failed (errno=%d)", errno);
        return;
    }

    // Henter det maksimale antal child-enheder,
    // som Thread-netværket tillader
    uint16_t max_children = otThreadGetMaxAllowedChildren(instance);

    // Gennemløber alle child-pladser i Thread-netværket
    for (uint16_t i = 0; i < max_children; i++) {

        // Struktur til at gemme information om en child
        otChildInfo child;

        // Forsøger at hente child-information for index i
        if (otThreadGetChildInfoByIndex(instance, i, &child) != OT_ERROR_NONE) {
            // Springer videre, hvis der ikke findes en child på dette index
            continue;
        }

        // Tjekker om child IKKE er en Full Thread Device (FTD)
        // dvs. at den er en Sleep end-device (SED)
        if (!child.mFullThreadDevice) {

            // Initialiserer iterator til barnets IPv6-adresser
            otChildIp6AddressIterator iter = OT_CHILD_IP6_ADDRESS_ITERATOR_INIT;

            // Struktur til at gemme IPv6-adressen
            otIp6Address ip;

            // Variabel til fejlhåndtering fra OpenThread-funktioner
            otError err;

            // Henter næste IPv6-adresse for child-enheden
            err = otThreadGetChildNextIp6Address(instance, i, &iter, &ip);

            // Tjekker om der blev hentet en gyldig IPv6-adresse
            if (err == OT_ERROR_NONE) {
```



```
// Opretter destinationens socket-struktur (IPv6)
struct sockaddr_in6 dest = {0};

// Angiver at det er en IPv6-adresse
dest.sin6_family = AF_INET6;

// Sætter destinationsporten (konverteret til netværks-byteorden)
dest.sin6_port = htons(UDP_PORT_SEND);

// Kopierer barnets IPv6-adresse til destinationen
memcpy(&dest.sin6_addr, &ip, sizeof(ip));

// Sender UDP-payload til child-enheden
int ret = sendto(
    sock_tx,                // Afsender-socket
    UDP_PAYLOAD,            // Data der sendes
    strlen(UDP_PAYLOAD),    // Længde af data
    0,                      // Ingen flags
    (struct sockaddr *)&dest, // Destination
    sizeof(dest)            // Størrelse af adresse
);

// Tjekker om afsendelsen fejlede
if (ret < 0) {
    // Logger fejl ved afsendelse
    ESP_LOGE(TAG, "UDP send error (errno=%d)", errno);
} else {
    // Logger succesfuld afsendelse med child'ens RLOC16
    ESP_LOGI(
        TAG,
        "BREQ sent to child RLOC16=0x%04x",
        child.mRloc16
    );
}
}
}
}

// Lukker UDP-socketten efter alle beskeder er sendt
close(sock_tx);
}
```

Funktionen er essentiel, fordi den muliggør distribution af kommandoer til alle lavstrømsenheder på én gang. Dette er særligt vigtigt i et system, hvor flere enheder skal reagere samtidig på forespørgsler som fx batterimålinger, testbeskeder eller alarmrelaterede funktioner.



6.5.2 Sleep end-device

Denne del af systemet beskriver Sleep end-device (SED), som fungerer som en individuel brandalarm i Thread-netværket. Enheden er implementeret i Zephyr RTOS og kommunikerer trådløst via OpenThread-protokollen. Hver SED kan registrere en brand gennem en fysisk knap, sende en alarmbesked til Border Routeren og reagere på modtagne kommandoer som eksempelvis aktivering, test eller slukning af alarmen.

main.c

Formål
Initialiserer systemet, opretter Thread-netværksforbindelse og starter alle nødvendige moduler.
Input
Ingen direkte input (systemstart).
Output
Initialiseret Thread-netværk.
Kørende moduler (ADC, alarm_test, fire, UDP).
Ansvar
Overordnet styring af opstart og modulinitialisering.

I *main.c* påbegyndes systemet ved at hente en OpenThread-instans og etablere netværksforbindelsen gennem funktionen *configure_thread_network_leader()*

Når netværket er konfigureret, initialiseres en række moduler, som tilsammen udgør enhedens funktionelle kerne:

- **ADC-modulet**, som simulerer måling af batterispænding.
- **Alarm_test-modulet**, der håndterer testfunktionen og styrer LED-indikering.
- **Fire-modulet**, som står for den egentlige brandalarm og kommunikation af alarmsignaler.
- **UDP-modulet**, der opretter en lyttertråd til at modtage og behandle netværksbeskeder.

Denne opsætning sikrer, at SED'en hurtigt efter opstart både kan kommunikere og reagere på hændelser.

For koden se bilag 10.8.1



network.c

Formål
Konfigurerer og tilslutter enheden til et eksisterende Thread-netværk som Sleep end-device.
Input
Netværksparametre (Network Name, Key, PAN ID, Channel, m.fl.).
Output
Aktiv Thread-netværksforbindelse.
Enheden registreret som SED med defineret poll-interval.
Ansvar
Netværksopsætning og energibesparende konfiguration.

Funktionen `configure_thread_network_leader()` sørger for, at SED'en tilslutter sig det eksisterende Thread-netværk. Den nødvendige dataset-struktur udfyldes med netværksoplysninger:

For kode se bilag 10.8.2

SED konfigureres som en sleep end-device ved at deaktivere konstant modtagelse (`mRxOnWhenIdle = false`). Dette reducerer strømforbruget betydeligt. Derudover defineres et *poll interval*, som bestemmer, hvor ofte enheden vågner for at tjekke efter netværksbeskeder.



udp.c

Formål
Håndterer al UDP-baseret kommunikation mellem SED og Border Router.
Input
Indkommende UDP-beskeder på port 12345.
Output
Afsendelse af svar- og statusbeskeder (tekst/JSON).
Kald til interne moduler (alarm, test, status).
Ansvar
Fortolkning af kommandoer og videreformidling af handlinger.

Dette modul håndterer kommunikationen via UDP. Der startes en dedikeret lyttertråd (*udp_server_thread()*), som konstant overvåger port 12345 for nye beskeder. Når der modtages data, tolkes indholdet og den relevante handling udføres:

- **"ALARMTEST"** → udløser en testfunktion og tænder LED'en i en begrænset periode.
- **"ALARM_START"** → aktiverer den lokale alarm og sender en bekræftelse *"ALARM_ONLY"* tilbage.
- **"HUSH"** → stopper en aktiv alarm og kvitterer med *"HUSH_ACK"*.
- **"BREQ"** → anmoder om status, hvorefter enheden måler batterispænding, indsamler OpenThread-identifikatorer (Mleid, Rloc16, ExtAddr) og returnerer et JSON-objekt.

For kode se bilag 10.8.3

UDP-modulet fungerer dermed som forbindelsesleddet mellem SED'en og Border Routeren.



alarm_test.c

Formål
Simulerer en alarm for test og verifikation af systemets funktion.
Input
Kommando via UDP eller intern funktionskald.
Output
LED aktiveret i en fast periode (10 sekunder).
Ansvar
Funktionel test uden aktivering af reel alarm.

Dette modul anvendes til testformål. Når *alarm_test_trigger()* aktiveres, tændes LED'en i 10 sekunder som en simuleret alarmindikering.

For kode se bilag 10.8.5

Det gør det muligt at kontrollere systemets funktion uden at aktivere den egentlige alarm.



fire.c

Formål
Styrer den egentlige brandalarm og alarmindikering.
Input
Fysisk knap.
UDP-kommandoer (ALARM_START, HUSH).
Output
Blinkende LED-alarm.
UDP-besked til Border Router ved alarmstart.
Ansvar
Aktivering, styring og deaktivering af alarmtilstand.

I *fire.c* håndteres den primære brandalarm. LED'en initialiseres som visuel sirene, og en fysisk knap anvendes til at udløse alarmer. Når knappen trykkes, startes alarmer via *fire_start()*, og der sendes en UDP-besked "fire" til Border Routeren.

For kode se bilag 10.8.4

En separat tråd styrer LED-blinkmønstret, så alarmer er tydeligt synlig. Alarmer kan stoppes via et "HUSH"-signal eller ved at genstarte enheden. Funktioner som *fire_is_active()* anvendes til at afgøre, om alarmer allerede kører.



adc.c

Formål
Simulerer måling af batterispænding og status.
Input
Forespørgsel fra UDP-modulet.
Output
Fast spændingsværdi og batteristatus.
Ansvar
Understøtter test af statusrapportering uden fysisk hardware.

Dette modul simulerer batteriovervågning. Da der ikke benyttes en fysisk ADC i test opsætningen, returneres faste spændings- og statusværdier. Det muliggør test af kommunikationsflowet uden ekstra hardware. Grundlaget for dette var at brugen af ADC var forskellige fra board til board og det lykkedes ikke at få den til at virke på nRF54115

For kode se bilag 10.8.6

Samlet funktion

Disse moduler udgør tilsammen en komplet, selvstændig brandalarmenhed, der kan deltage i et trådløst Thread-netværk, kommunikere med Border Routeren og håndtere både alarmer, testfunktioner og statusforespørgsler.

Den modulære opbygning gør systemet fleksibelt, let at udvide og robust over for fejl – eksempelvis ved tilføjelse af sensorer, yderligere funktioner eller optimering af strømforbrug.



7. Projekt test

Formålet med dette afsnit er at dokumentere de centrale tests, der gennemføres for at vurdere, om projektet lever op til de selvvalgte kriterier og standarder. Testene skal ikke blot bekræfte, at systemet fungerer, men også skabe en faglig forståelse for dets styrker og begrænsninger, så en eventuel videreudvikling kan tage afsæt på et veldokumenteret grundlag.

De fire testområder - Robusthed, sikkerhed, WEB-GUI og batterilevetid - repræsenterer de mest væsentlige aspekter af et moderne Thread mesh-netværk. Samlet belyser de systemets evne til at opretholde stabil kommunikation, beskytte data og kryptering, sikre automatisk rollefordeling, tilbyde brugervenlig styring og optimere strømforbrug i trådløse enheder.

Testene udføres som demonstrationsforsøg, der dokumenterer systemets funktionalitet og robusthed inden for projektets rammer. Denne tilgang gør det muligt at validere både tekniske og anvendelsesorienterede egenskaber uden krav om fuld certificering, men med fokus på at skabe et realistisk og anvendeligt bevis på konceptets virkning. I det følgende præsenteres projektets kravspecifikationer, som danner grundlag for dokumentationen af systemets funktionalitet, sikkerhed og ydeevne.



7.1 Robusthedstest: K-01

For at verificere krav K-01 gennemføres en række praktiske tests, der tilsammen vurderer Thread-netværkets robusthed, kommunikationsstabilitet og rækkevidde under forskellige forhold. Testene fokuserer på måling af RSSI og PDR, samt på systemets evne til at opretholde eller genetablere forbindelse ved afstand, forstyrrelser eller afbrydelser.

Test	Formål	Metode	Forventet resultat
Robusthedstest 1 Rækkevidde fri linje	Bestemme maksimal stabil rækkevidde uden hindringer	Måling af RSSI og PDR ved stigende afstand (20, 30, 40 og 50 m)	PDR $\geq$ 95 %, RSSI $>$ -85 dBm op til 25 m
Robusthedstest 2 - Rækkevidde indendørs forhindring	Evaluer signalstyrke gennem vægge eller glas	Placer væg/glas mellem enheder og gentag måling	Forbindelse opretholdes, RSSI $>$ -85 dBm
Robusthedstest 3 - Mesh-hop simuleringstest	Verificere korrekt videresendelse gennem mellemnode	Test dataflow mellem noder via 1–2 hop	Datapakker videresendes uden tab

7.1.1 Robusthedstest 1 - Rækkevidde fri linje

Test Type: Funktionel

Metode: Måle RSSI, PDR og stabil forbindelse mellem Border Router og enhed ved stigende afstand (20,30,40 og 50 m).

Resultat: Forbindelse og $<$ 5 % pakkefejl op til 50 m.

Formål:

Formålet med testen er at bestemme den maksimale kommunikation rækkevidde mellem Borderrouter og End-device i et åbent miljø uden hindringer. Testen skal dokumentere, ved hvilken afstand systemet fortsat kan opretholde en pålidelig forbindelse i henhold til krav K-01.

Metode:

I forbindelse med testen vurderes kvaliteten af den trådløse kommunikation ved hjælp af to nøgleparametre: RSSI (Received Signal Strength Indicator)¹⁰ og PDR (Packet Delivery Ratio)¹¹. Disse parametre anvendes til at bestemme signalstyrke og pålidelighed i kommunikationen mellem thread-enhederne.

RSSI angiver den modtagne signalstyrke målt i dBm. Jo mindre negativt tallet er, desto stærkere er signalet. En typisk vurdering er, at værdierne over -80 dBm betegnes som gode, mens værdier under -90 dBm

Signal strength	RSSI (dBm)	Meaning
Excellent	-30 to -70 dBm	Your connection is strong and you shouldn't have any issues
Good	-71 to -80 dBm	Your connection is good enough for most activities.
Fair	-81 to -90 dBm	Your connection is fine for most tasks, but you may experience some issues.
Weak	Below -90 dBm	Your connection isn't strong enough for most activities.

Figur 14: dBm signal styrkemeter

¹⁰ Virgin Media Edit (18.03.2025)

¹¹ Rao, Venkateswara M. (Vol12, No 5 - 2021, p.631)

indikerer svagt signal og risiko for pakketab. I denne test anvendes RSSI til at beskrive, hvordan signalstyrken ændrer sig med stigende afstand mellem border routeren og end devices.

Metoden til måling af RSSI bestod i at aflæse værdien i neighbor table-registret. Her blev der søgt efter en forbindelse med en RSSI-værdi over -90 dBm for at udføre PDR-testen.

Tabellen nedenunder viser et eksempel på, hvordan neighbor table ser ud – i dette tilfælde med en RSSI på cirka -44 dBm:

```
> neighbor table
```

Role	RLoc16	Age	Avg RSSI	Last RSSI	R/D/N	Extended MAC	Version
C	0x3401	5	-45	-44	0 0 0	262d2ae1b60b2670	5

Figur 15: Neighbor table på Border router

PDR beskriver andelen af datapakker, der modtages korrekt i forhold til det samlede antal afsendte pakker. En høj PDR-værdi indikerer en stabil forbindelse. I denne test anses en værdi på 95% eller derover som tilfredsstillende ved afstande op til 50 meter.

Metoden PDR blev målt på, var ved at pinge enddevice ved at anvende denne kommando for at pinge 100 gange: `ping >IPv6< 100 100` for derefter at udregne det med:

$$\text{PDR} = r / s \times 100\% \quad (r = \text{received og } s = \text{sent})$$



Figur 16: Rækkevidde testmiljø

Kombinationen af RSSI og PDR målinger giver et samlet billede af kommunikationens kvalitet. Et højt RSSI niveau uden tilsvarende høj PDR kan indikere støj eller interferens i miljøet, mens et lavt RSSI niveau typisk hænger sammen med øget pakketab. Derfor anvendes begge måleparametre i testen for at vurdere systemets reelle ydeevne og robusthed i praksis.

Resultat:

Resultat ved 20 meter - RSSI -75 dBm god forbindelse - 100% PDR

```
100 packets transmitted, 100 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 47/158.200/369 ms.
Done
> |
```

Resultat ved 26 meter - RSSI -85 dBm rimelig forbindelse - 100% PDR

```
100 packets transmitted, 100 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 62/165.350/320 ms
Done
> |
```

Resultat ved 30 meter - RSSI -95 dBm svagt signal - 100% PDR

```
100 packets transmitted, 100 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 38/161.330/383 ms.  
Done  
> |
```

Resultat ved 50 meter - RSSI -100 dBm svagt signal - 100% PDR

```
100 packets transmitted, 100 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 38/161.330/383 ms.  
Done  
> |
```

Figur 17: Rækkevidde testresultat

Konklusion:

Testen dokumenterer, at Thread-netværket kan opretholde trådløs kommunikation inden for den forventede rækkevidde på 50 meter under optimale forhold. Resultaterne bekræfter, at systemets kommunikation lever op til krav K-01 ved fri linje uden forhindringer.

7.1.2 Robusthedstest 2 - Rækkevidde indendørs forhindringstest

Testtype: Funktionel / Robusthed

Metode: Placer væg eller glas mellem noder og observer RSSI-fald.

Forventning: Forbindelsen må ikke falde under -85 dBm ved ≤ 10 m.

Formål:

Formålet med testen er at evaluere, hvordan fysiske forhindringer som vægge og glas påvirker signalstyrken og kommunikationsstabiliteten mellem border router og enddevice. Testen skal give indsigt i, hvor robust forbindelsen er under mere realistiske indendørsforhold.

Metode:

Testen blev udført for at opnå et overordnet billede af Thread-kommunikationens rækkevidde og stabilitet i det aktuelle område. Et end device blev placeret i et køretøj og bevæget gennem forskellige zoner, mens border routeren forblev stationær. Under bevægelsen blev der løbende registreret RSSI-værdier og PDR for at observere, hvordan signalstyrke og datapålidelighed påvirkes af afstand, terræn og naturlige forhindringer i omgivelserne.

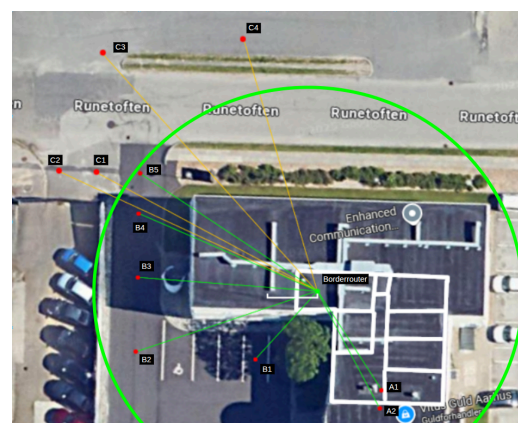


Figur 18: Rækkevidde end-device afstandsmetode

Resultat:

A1 og A2 er målinger foretaget uden køretøj: RSSI mellem -85dBm og -92dBm og afstanden er mellem ca 13 og 15 meter, gennem vægge og gulv.

B1 til B5 er målinger med forbindelse med køretøj: RSSI -65 dBm og -105 dBm. afstand mellem 8 og 25 meter.



C1 til C4 er målinger uden forbindelse med køretøj: RSSI kunne ikke måles, da “end-device” ikke fremgik af neighbor-table. Afstanden ca. 27-35 meter.

Konklusion:

Figur 19: Rækkevidde kort over indendørs test

Testen viser, at Thread-netværket kan opretholde stabil kommunikation gennem almindelige indendørsforhindringer uden markant signalforringelse. Resultaterne indikerer, at systemet er robust nok til drift i bygninger med typiske vægmateriale.

7.1.3 Robusthedstest 3 – Mesh-hop simuleringstest

Test Type: Funktionel/Integration

Metode: Analyse af routing, link quality og dataplan via OTNS og Wireshark

Forventning: Netværket etablerer og opretholder korrekt multi-hop routing baseret på Thread's afstandsvektor-logik

Formål:

Formålet med testen er at vurdere robustheden i Thread-netværkets mesh-routing under forhold, hvor direkte kommunikation mellem alle noder ikke er mulig. I stedet for udelukkende at verificere et enkelt hop scenario undersøger testen netværkets evne til at håndtere flere routere og end devices, etablere alternative ruter samt opretholde stabil datapakkevideresendelse ved varierende linkkvalitet.

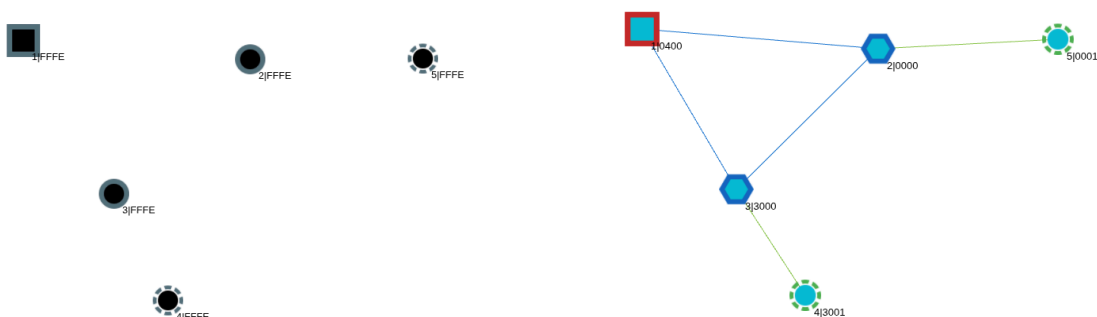
Metode:

Testen blev udført i OpenThread Network Simulator (OTNS) se bilag 10.10 og bestod af i alt fem Thread-noder:

- Node 1: Border Router (FTD)
- Node 2: Router A (FTD)
- Node 3: Router B (FTD)
- Node 4: End Device (SED)
- Node 5: End Device (SED)

Noderne blev tilføjet i CLI med følgende specs: Range på 10 meter.

```
add br    x 486 y 282 rr 100
add router x 784 y 307 rr 100
add router x 605 y 486 rr 100
add sed   x 676 y 628 rr 100
add sed   x 1011 y 306 rr 100
```



Figur 20: OTNS simulering automatisk logik opsætning

Da alle noder var placeret, blev noderne tændt og deres specs blev hentet med cmd !nodes.

```
> !nodes
id=1      type=br      extaddr=8afab88584311ee2  rloc16=0400
id=2      type=router  extaddr=2a0c5d75293c24f3  rloc16=0000
id=3      type=router  extaddr=f6ab38b3b00c4a4a  rloc16=3000
id=4      type=sed     extaddr=c23eb01f0e2fcd a0  rloc16=3001
id=5      type=sed     extaddr=9ed826ce3a23082f  rloc16=0001
```

Figur 21: OTNS noderes Rloc16 adresser

Derefter blev router-, neighbor- og child tabeller hentet - for fuld tabel se bilag 10.15

Resultat

For at demonstrere Thread netværkets robusthed og selvhelende egenskaber blev en aktiv router bevidst fjernet fra topologien. Da routeren ophørte med at svare på MLE-meddelelser, mistede tilknyttede end-devices deres parent-forbindelse og initierede automatisk en reattach procedure. sleep end-devices, mangler nu en parent og begynder at sende data requests ud for at få adgang til netværket igen.

300 0x3001	0x3000	IEEE 802.15.4	22 Data Request
301 0x3001	0x3000	IEEE 802.15.4	22 Data Request
302 0x3001	0x3000	IEEE 802.15.4	22 Data Request

Figur 22: Wireshark capture - End-device 3001 søger data, men får intet svar

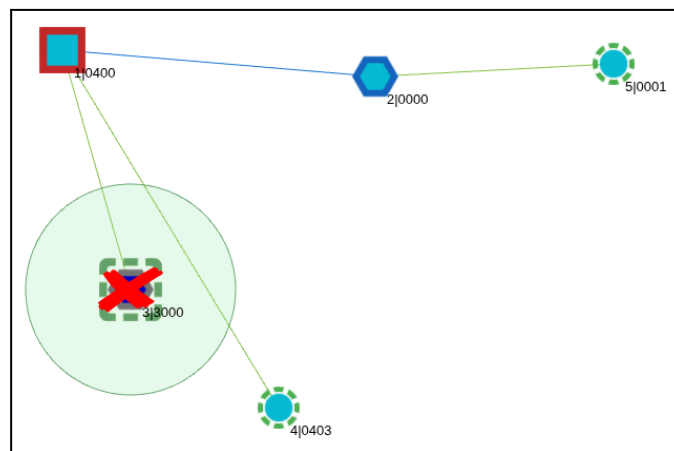
End-devices genfandt herefter en alternativ router med tilstrækkelig link quality og genetablerede forbindelsen uden manuel indgriben. Routingtabellerne i netværket blev opdateret dynamisk, og kommunikationen fortsatte via den resterende router. Testen bekræfter, at Thread-netværket understøtter selvhelende mesh-routing i overensstemmelse med protokollens design. Fuld wireshark kan ses i bilag 10.10.

348 fe80::c03e:b01f:e2f:cda0	ff02::2	MLE	63
349 fe80::280c:5d75:293c:24f3	fe80::c03e:b01f:e2f:cda0	MLE	117
350		IEEE 802.15.4	5 Ack
351 fe80::88fa:b885:8431:1ee2	fe80::c03e:b01f:e2f:cda0	MLE	117
352		IEEE 802.15.4	5 Ack
353 fe80::c03e:b01f:e2f:cda0	fe80::88fa:b885:8431:1ee2	MLE	121
354		IEEE 802.15.4	5 Ack
355 c2:3e:b0:1f:0e:2f:cd:a0	8a:fa:b8:85:84:31:1e:e2	IEEE 802.15.4	34 Data Request
356		IEEE 802.15.4	5 Ack
357 8a:fa:b8:85:84:31:1e:e2	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	126 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: c2:3e:b0:1f:0e:2f:cd:a0
358		IEEE 802.15.4	5 Ack
359 c2:3e:b0:1f:0e:2f:cd:a0	8a:fa:b8:85:84:31:1e:e2	IEEE 802.15.4	34 Data Request
360		IEEE 802.15.4	5 Ack
361 8a:fa:b8:85:84:31:1e:e2	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	121 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: c2:3e:b0:1f:0e:2f:cd:a0
362		IEEE 802.15.4	5 Ack
363 0x0403	0x0400	IEEE 802.15.4	34 Data, Src: 0x0403, Dst: 0x0400
364		IEEE 802.15.4	5 Ack
365 0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000

node 1> child table												
ID	RLOC16	Timeout	Age	LQ In	C_VN	R D N	Ver	CSL	QMsgCnt	Suprvsn	Extended MAC	
3	0x0403	240		3	3	89	0 0 1	5	0	0	129	c23eb01f0e2fcd a0

Figur 23: Wireshark capture - End-device finder Border router og får ny Rloc16 adresse

Denne konfiguration sikrede, at datapakker til visse end devices kun kunne leveres via multi-hop routing, samtidig med at der fandtes alternative ruter gennem begge routere



Figur 24: OTNS med slukket Router og ny rute til end-device med opdateret Rloc16

Konklusion

Testen viser, at Thread-netværket korrekt aktiverer multi-hop routing, når direkte rækkevidde ikke er mulig. Den Bellman-Ford-lignende afstandsvektorlogik - se Bilag 10.11 beregner ruten gennem mellemnode, og netværket opretholder stabil 2-hop kommunikation med høj PDR. Resultatet giver et realistisk billede af, hvordan et fysisk Thread-mesh forventes at fungere i praksis, og understøtter krav **K-01** om robust og selvhelende mesh-kommunikation.



7.2 Sikkerhed: K-02

For at verificere krav K-02 gennemføres en række analyser og praktiske tests, der tilsammen vurderer Thread-netværkets datasikkerhed og kryptering. Testene fokuserer på at dokumentere korrekt AES-128-kryptering, afvisning af uautoriserede tilslutningsforsøg, samt at systemet registrerer og logger sikkerhedshændelser pålideligt under drift.

Test	Formål	Metode	Forventet Resultat
Sikkerhedstest 1 Krypterings- verifikation	Bekræfte at nytte-data i Thread-trafik er krypteret	Wireshark/sniffer-analyse af IEEE 802.15.4 / Thread; kontrollér at nytte-data fremtræder krypteret	Kun krypterede datapakker observeres
Sikkerhedstest 2 Uautoriseret tilslutningsforsøg	Teste netværkets adgangskontrol mod rogue/joiner-enheder	<i>Uautoriseret tilslutning:</i> Forsøg at udføre join-flow fra uregistreret enhed uden nøgle	Adgang nægtes; ingen dataudveksling
Sikkerhedstest 3 HTTPS	Verificere at uautoriserede forsøg og sikkerhedshændelser logges korrekt	Gennemgang af Border Router-log: tid, enheds-ID, hændelsestype; korscheck mod testaktivitet	Uautoriserede forsøg registreres med tid og type; logentries kan spores til testaktiviteter

7.2.1 Sikkerhedstest 1 - Krypterings Verifikation

Test Type: Ikke-funktionel.

Metode: Verificér at al kommunikation sker med AES-128 via sniffer.

Forventning: Kun krypterede rammer er synlige.

Formål:

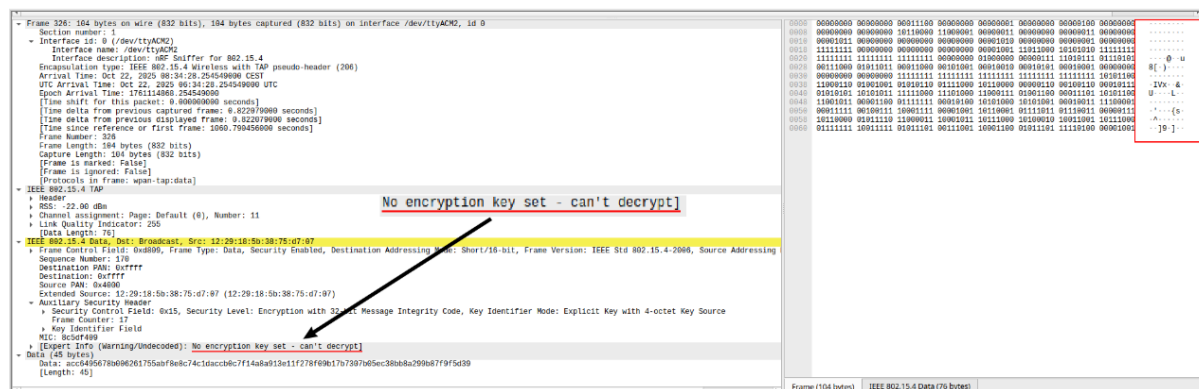
Formålet med testen er at verificere, at al kommunikation i Thread-netværket foregår krypteret via AES-128 og at ingen datapakker kan aflæses i klartekst - AES-128 forklaring, se bilag 10.6. Testen skal dokumentere, at kryptering i Thread-protokollen fungerer korrekt og beskytter kommunikationen mellem Border Router og end devices mod uautoriseret adgang.

Metode:

Trafikken blev analyseret i Wireshark ved hjælp af en Thread-sniffer under en aktiv DTLS-forbindelse mellem Border Routeren og et tilkoblet end device Opsætning af sniffer se bilag 10.12. Thread-protokollen anvender AES-128-baseret kryptering, hvor data opdeles i 128-bit blokke og behandles gennem 10 runder, hver med et nyt delnøglesæt afledt af den oprindelige nøgle.

Resultat:

Alle datapakker i Wireshark fremstår som uaf læselig krypteret trafik. Hver datapakke har en unik nonce-værdi, som forhindrer genbrug af nøgler. Den faktiske AES-128-nøgle udveksles aldrig i klartekst, men etableres via DTLS-handshake, der sikrer autentificering og sikker nøgleaftale mellem enhederne. Under analysen kunne det observeres, at selvom Wireshark registrerer UDP-trafik på de relevante Thread-porte, er data ulæselige uden korrekt nøgle.



Figur 25: Verificering af data kryptering efter wireshark sniffing

Konklusion:

Testen bekræfter, at Thread-netværket benytter korrekt AES-128-kryptering til at beskytte datakommunikationen. Alle observerede pakker fremstod krypterede, og ingen data kunne læses uden nøgle. Kommunikationen mellem Border Router og end devices er således sikret mod uautoriseret aflæsning og lever op til krav K-02 om datasikkerhed og kryptering.

7.2.2 Sikkerhedstest 2 - Uautoriseret tilslutningsforsøg

Test Type: Penetrationstest

Metode: Forsøg at forbinde uregistreret node uden netværksnøgle.

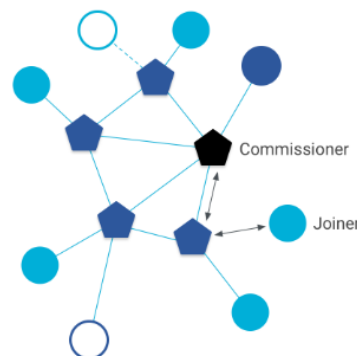
Resultat: Adgang nægttes, ingen dataudveksling. Border Routeren logger afvist join-forsøg.

Formål:

Formålet med testen er at verificere, at Thread-netværket afviser uautoriserede enheder, som forsøger at tilslutte sig uden gyldig netværksnøgle eller som joiner. Testen skal dokumentere, at autentificerings- og adgangskontrolmekanismerne i Thread effektivt forhindrer uønskede forbindelser og at hændelserne registreres i Border Routerens/logsystemets logfiler.

Metode:

Testen blev udført som et uautoriseret tilslutningsforsøg, hvor en enhed uden foruddefineret netværksnøgle forsøgte at etablere forbindelse til Thread-netværket. Til formålet benyttes en nRF5340 som rogue-enhed, startet på samme 802.15.4-kanal som det eksisterende netværk. Endvidere blev der testet at rogue-enheden ikke kunne joine uden korrekt commissioning credential eller registrering i Border Routerens whitelist. Under forsøget aktiveres join-processen fra den uautoriserede enhed (se figur x), mens Border Routerens systemlog og Wireshark-captures fra en nRF52840-sniffer indsamles for at dokumentere forløbet.



Figur 26: Commissioner/joiner netværkslogik

Resultat:

Her er rogue-enheden opsat til at forbinde til thread-mesh-netværket på kanal 11, men det mislykkedes, og Border Router afviste enheden.

Rogue enhed	Border Router
<pre>uart:~\$ *** Booting nRF Connect SDK v3.1.0-6c6e5b32496e *** *** Using Zephyr OS v4.1.99-1612683d4010 *** ot ifconfig up Done uart:~\$ ot channel 11 Done uart:~\$ ot panid 0x4000 Done uart:~\$ ot thread start Done</pre>	<pre>w(4737894) OPENTHREAD:[W] Mle-----: Failed to process UDP: Security w(4738644) OPENTHREAD:[W] Mle-----: Failed to process UDP: Security w(4739394) OPENTHREAD:[W] Mle-----: Failed to process UDP: Security w(4740644) OPENTHREAD:[W] Mle-----: Failed to process UDP: Security</pre>

Figur 27: Rogue enhed afvises af Border router ved forsøg på forbindelse via fælles kanal

I testforløbet blev join-forespørgsler forsøgt fra rogue-enheden gennem On-mesh commissioning metoden, men ingen session blev etableret, idet Border Routeren afviste forbindelsen.

Border Routeren har fået en commissioner joiner credential med J01NMESAFE og Rogue forsøger med standard J01NME, men det fejler og giver tilbagemelding på begge enheder:

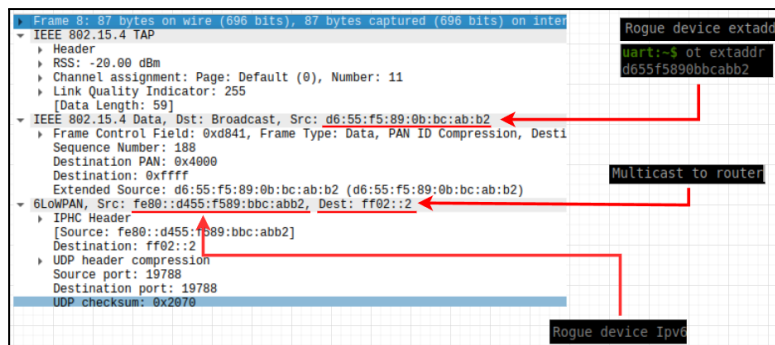
Rogue enhed	Border Router
<pre>uart:~\$ ot joiner start J01NME Done Join failed [Security]</pre>	<pre>Done > commissioner joiner add * J01NMESAFE > Commissioner: Joiner start d655f5890bbcabb2 Commissioner: Joiner end d655f5890bbcabb2 Commissioner: Joiner remove</pre>

Figur 28: Rogue enhed prøver commissioner/joiner logik

På netværksniveau viser Wireshark-capturen, at handshake-processen blev påbegyndt, men ikke fuldført. Dette indikerer, at der ikke fandt nogen nøglegenerering eller autentificering sted. Alle pakker med længden 87 og 89 stammer fra den rogue-enhed, der forsøger at sende multicast, men border routeren reagerer ikke på disse forespørgsler.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::1029:185b:3875:d707	ff02::1	MLE	95	Advertisement
2	9.880094	fe80::d455:f589:bbc:abb2	ff02::2	MLE	87	
3	12.649814	fe80::1029:185b:3875:d707	ff02::1	MLE	95	Advertisement
4	14.972598	fe80::d455:f589:bbc:abb2	ff02::2	MLE	87	
5	19.923289	fe80::d455:f589:bbc:abb2	ff02::2	MLE	87	
6	24.536010	fe80::d455:f589:bbc:abb2	ff02::2	MLE	87	
7	25.939582	fe80::1029:185b:3875:d707	ff02::1	MLE	95	Advertisement
8	29.935580	fe80::d655f5890bbcabb2	ff02::2	MLE	87	
9	34.855772	fe80::d455:f589:bbc:abb2	ff02::2	MLE	87	
10	36.989119	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
11	37.740099	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
12	39.219406	fe80::1029:185b:3875:d707	ff02::1	MLE	95	Advertisement
13	38.491091	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
14	39.744606	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
15	40.995844	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
16	42.245830	fe80::d455:f589:bbc:abb2	ff02::2	MLE	89	
17	43.511437	fe80::d455:f589:bbc:abb2	ff02::1	MLE	101	
18	44.073877	fe80::d455:f589:bbc:abb2	ff02::1	MLE	95	
19	46.196260	fe80::d455:f589:bbc:abb2	ff02::1	MLE	95	

Figur 29: Wireshark capture af join forsøg



Figur 30: Rogue forsøg på at multicast med Mle

Konklusion:

Testen bekræfter, at Thread-netværket konsekvent afviser enheder uden korrekt netværksnøgle eller joiner credentials. Der sker ingen uautoriseret kommunikation, og join-processen afbrydes, før nogle nøgler udveksles. Dette viser, at autentificeringsmekanismerne fungerer som tiltænkt, og at systemet samtidig er i stand til at registrere forsøg på uautoriseret adgang.

7.2.3 Sikkerhedstest 3 - HTTPS

Testtype: Funktionel / Monitorering

Metode: Forsøg på uautoriseret adgang til WEB-GUI uden gyldig HTTPS-session.

Resultat: Adgang nægtes, hændelsen af Border Router-systemets melding dokumenteres

Formål:

Formålet med testen er at verificere, at kommunikation mellem bruger og Border Router er beskyttet af HTTPS, og at systemet korrekt registrerer og tidsstempler forsøg på uautoriseret adgang til WEB-GUI. Testen skal påvise, at kun godkendte brugernavne kan tilgå administrationsportalen, samt at eventuelle adgangsforsøg uden gyldigt certifikat eller med ugyldige loginoplysninger registreres af border routeren.

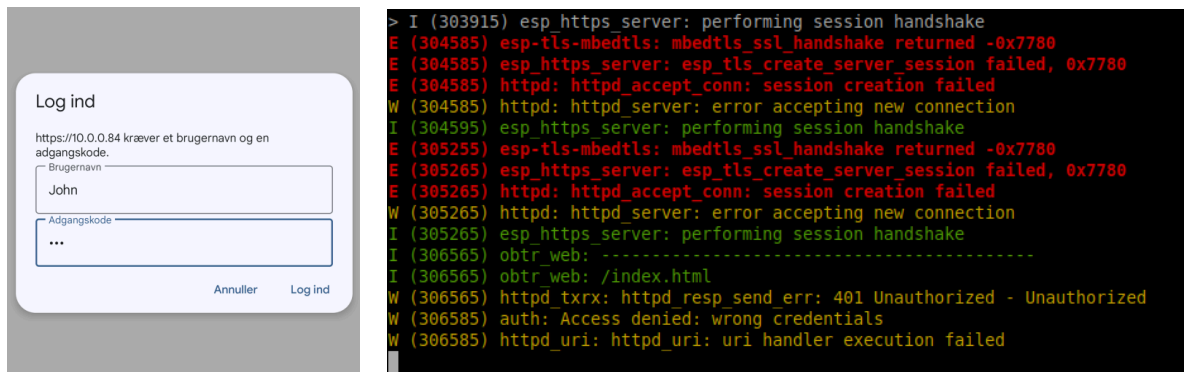
Metode:

Efter bekræftelsen af, at Thread-netværket korrekt afviser uautoriserede noder på netværksniveau, blev der udført en test af WEB-GUI-sikkerheden på Border Routeren. Testen bestod i at tilgå administrationsportalen både med og uden gyldig HTTPS-forbindelse for at kontrollere, om systemet nægter usikre forbindelser og håndterer fejlede loginforsøg korrekt. Opsætning af HTTPS se bilag 10.14

Først blev GUI åbnet via browserens HTTPS-protokol for at bekræfte, at forbindelsen anvender et gyldigt TLS-certifikat, og at der ikke tilbydes adgang via almindelig HTTP. Herefter blev der forsøgt adgang via HTTP (port 80) og med bevidst forkerte loginoplysninger under HTTPS for at udløse en sikkerhedsloghændelse. Under hele forløbet blev systemets logfiler overvåget for at registrere hændelser relateret til authentication failures, session-oprettelser og adgangsafvisninger.

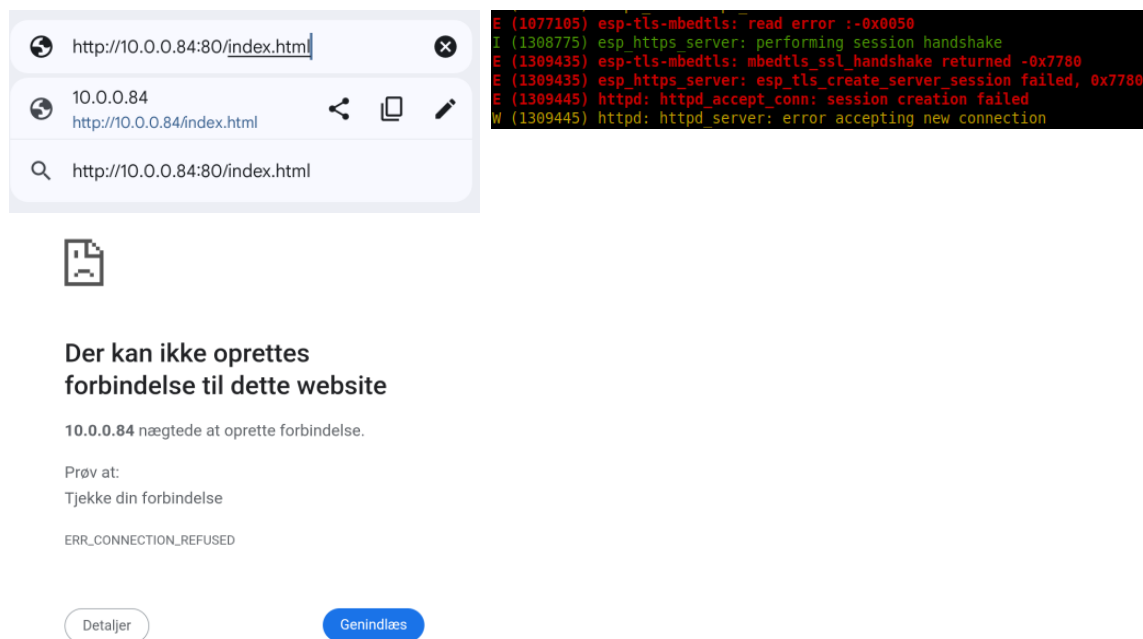
Resultat:

Ved forsøg på at komme ind på https webserveren blev forsøgt at logge på med John 123, det mislykkedes og tilbagemelding på borderrouteren var en 401 unauthorized attempt og access denied:



Figur 31: Forsøg på at logge på med forkert password til https serveren.

Endvidere blev der forsøgt at logge på webserveren som http, men dette fejlede også



Figur 32: Forsøg på at logge på http serveren

Konklusion: Testen bekræfter, at WEB-GUI kun kan tilgås via HTTPS og at systemet afviser alle uautoriserede eller ukrypterede adgangsforsøg. Samtlige hændelser registreres på Border routeren. Systemet opfylder dermed krav K-02 vedrørende adgangskontrol, krypteret kommunikation og dokumentation af sikkerhedshændelser.



7.3 WEB-GUI: K-03

Formålet med GUI-testene er at verificere korrekt funktion og pålidelighed i brugergrænsefladen. Testene fokuserer på kommunikation mellem systemets komponenter, korrekt håndtering og visning af alarmer samt simulering af brandsituationer for at sikre, at systemet reagerer som forventet under kritiske hændelser.

Test	Formål	Metode	Forventet Resultat
WEB-GUI enhedstest	Verificering af WEB-GUI	Test af knapper i GUI samt korrekt opdatering af interfacet.	Det forventes at tilsluttet enheder svare tilbage og GUI opdateres korrekt
Alarm test	Test af alarmfunktion.	Brug af web-GUI til at se om testen fungerer som den skal	Det forventes at alarmen kører i 10 sekunder og stopper derefter.
Brandtest	Test af GUI funktion til at alarmere når en brand startes.	Manuel start en brandsimulering ved tryk på programmet knap på dev-board.	Når knappen trykkes, forventes det at BR modtager en besked og alarmerer alle andre enheder i nettet.

7.3.1 WEB-GUI enhedstest

Test type: Enhedsoversigt

Metode: Web GUI funktionskald til aktive enheder i netværket

Resultat: Alle aktive enheder vises med opdateret status og thread informationer.

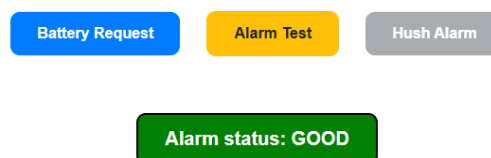
Formål:

Formålet med testen er at verificere, at WEB-GUI korrekt henter og viser information fra de aktive enheder i Thread-netværket. Testen skal bekræfte, at data som batteristatus, alarmstatus og Thread-informationer modtages og vises efter hensigten.

Metode:

Når Thread-netværket startes, vises et tomt WEB-GUI, da der endnu ikke er foretaget nogen forespørgsel på enhederne. Testen påbegyndes ved at trykke på "Battery Request"-knappen i WEB-GUI. Dette sender en forespørgsel til alle aktive enheder, der har rollen *child* (SED).

Thread Network Monitoring Dashboard



Figur 33: Første login på https til WEB-GUI



Når en *child*-enhed modtager forespørgslen med beskeden *BREQ*, udføres to interne funktioner:

1. Batterimåling, som registrerer den aktuelle spænding.
2. Thread-information, som indsamler statusdata fra netværket.

Efter at disse funktioner er udført, pakkes resultaterne i en JSON-besked, som sendes tilbage til border routeren. Routeren modtager herefter data fra de aktive enheder og opdaterer automatisk WEB-GUI med de indkomne informationer.

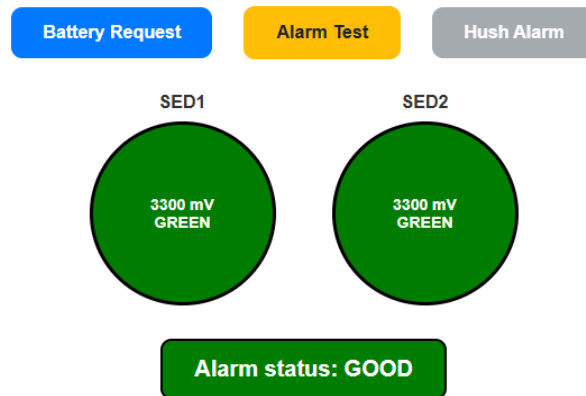
Ved gennemførelse af testen vises to aktive enheder i netværket, begge med tilfredsstillende batterispænding og en alarmstatus markeret som *Good*, hvilket indikerer, at ingen alarmer er aktive.

Hvis brugeren klikker på en af enhederne (repræsenteret som cirkler i GUI), fremvises yderligere detaljer som de specifikke Thread-parametre, modtaget fra forespørgslen.

Konklusion:

Testen bekræfter, at WEB-GUI fungerer som forventet og korrekt modtager data fra de aktive enheder. Funktionskaldet til batterimåling og Thread-information udføres korrekt, og informationerne præsenteres tydeligt i brugerfladen. Systemet lever dermed op til designkravene for visning af enhedsstatus i netværket.

Thread Network Monitoring Dashboard



Figur 34: Opdatering af WEB-GUI efter sendt BREQ



7.3.2 Alarm test

Test type: Alarm test.

Metode: Brug af WEB-GUI knap til at kalde alarm test funktionen på end device.

Resultat: Alarm testen fungere som den skal og LED0 lyser i 10 sekunder som den skal.

Formål:

Formålet med testen er at verificere, at alarmfunktionen på end device aktiveres korrekt, når funktionen udløses via WEB-GUI, samt at kommunikationen mellem border router og end device fungerer som forventet.

Metode:

Testen udføres ved at aktivere alarmtest-funktionen gennem "Alarm Test"-knappen i WEB-GUI. Når knappen trykkes, sendes en besked fra border routeren til den valgte end device med kommandoen *LED*.

```
UDP_BR: LED request sent to child RLOC16-0x7c05
```

Ved modtagelse af beskeden kalder end device funktionen *alarm_test()*, som tænder LED0 på udviklings boardet. LED'en forbliver tændt i 10 sekunder, hvorefter den automatisk slukkes igen.

Denne funktion simulerer en reel alarm, hvor LED'en fungerer som visuel indikator i stedet for et lydsignal.



Figur 35: Alarmstatus WEB-GUI og end-device

Under testen observeres, at LED0 aktiveres øjeblikkeligt efter tryk på knappen, lyser i nøjagtigt 10 sekunder og derefter slukker som forventet. Der registreres ingen kommunikationsfejl mellem WEB-GUI, border router og end device.

Konklusion:

Testen bekræfter, at alarmfunktionen fungerer korrekt, og at kommunikationen mellem WEB-GUI, border router og end device er stabil og pålidelig. Systemet reagerer som forventet, og funktionen udfører de tilsigtede handlinger uden afvigelser.

7.3.3 Brandalarms test

Test type: Brandalarm simulering.

Metode: Manuel knap tryk til start at brandalarm

Resultat: Testen viser at kommunikationen virker og alarmering på andre enheder sker hurtigt.

Formål:

Formålet med testen er at verificere systemets reaktion, når en enhed registrerer brand, samt at sikre korrekt kommunikation mellem WEB-GUI, border router og de øvrige tilkoblede enheder i netværket.

Metode:

Testen er designet til at simulere en reel brandsituation ved manuelt at aktivere alarmen på én enhed. Dette gøres ved at trykke på knap 0, hvilket sender en besked med statussen *FIRE* til border routeren. Routeren fungerer som central kommunikationsenhed og skal herefter informere alle aktive enheder i systemet om, at der er opdaget brand.

Den enhed, som testen aktiveres fra, ændrer sin visuelle status i WEB-GUI til en rød blinkende cirkel, som symboliserer brand. Samtidigt begynder LED0 på den fysiske enhed at blinke for at indikere, at alarmen er aktiv.

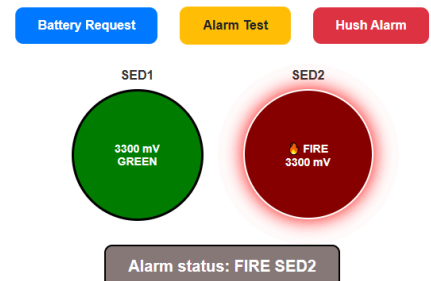
Når border routeren har registreret alarmen, videresender den beskeden til de øvrige enheder på netværket — i dette tilfælde SED1. SED1 modtager alarmbeskeden, svarer tilbage med en kvittering, og ændrer sin egen status til *ALARM*. Dette vises både i GUI med en orange cirkel og ved at dens LED begynder at blinke.

Herved opnås synkronisering mellem systemets visuelle interface og de fysiske enheders status.

Konklusion:

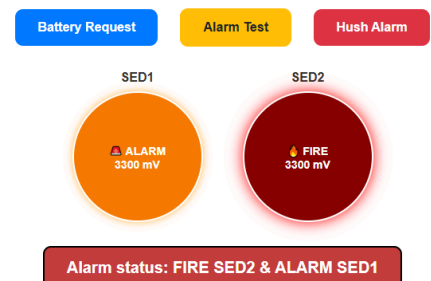
Testen bekræfter, at systemets alarm- og kommunikationsfunktioner fungerer som forventet. Den manuelle simulering af en brand udløser korrekt visuel feedback i både WEB-GUI og på de fysiske enheder. Systemet reagerer pålideligt og konsekvent i henhold til designkravene for en brandhændelse.

Thread Network Monitoring Dashboard



Figur 36: knaptryk detekteret ved SED2

Thread Network Monitoring Dashboard



Figur 37: SED1 opdateret til state Alarm



7.4 Batterilevetid: K-04

Formålet med disse batteri tests er at evaluere systemets strøm relaterede egenskaber og sikre stabil drift over længere tid. Testene omfatter måling af batterispænding, analyse af strømforbrug samt vurdering af polling-periodens betydning for energiforbruget. Resultaterne bruges til at vurdere løsningens energieffektivitet og egnethed til batteridrevet drift.

Test	Formål	Metode	Forventet Resultat
Batteritest - strømmåling	At bruge adc til at måle på batteriet.	Måling på 2 seriekoblet batterier ved brug af adc.	At få en måling der kan bruges til simpel batteri overvågning.
Strømforbrugs test	Teste hvor meget strøm SED bruger når den sover.	Brug af NSC power profiler til at lave en længere forbrugsmåling.	At få et lavt forbrug så der kan regnes ud hvor lang tid enheden kan køre på batterier.
Polling period test	Test af hvor stor en betydning pollingperioden har for forbruget.	Ændring af polling period i koden.	Ved ændring af poll perioden forventes det jo lavere poll perioden, jo større forbrug.

7.4.1 Batteritest - Strømmåling

Testtype: Funktions- og kommunikationstest af batteri måling

Metode: Måling via ADC og forespørgsel gennem WEB-GUI

Resultat: Batterimålingen virker som den skal og returnere korrekte batteri spændinger til GUI.

Formål:

Formålet med testen er at verificere, at spændingsmålingen via ADC-indgangen på development boardet fungerer korrekt, samt at de målte værdier overføres og vises korrekt i WEB-GUI. Testen skal samtidig sikre, at kommunikationen mellem end device og border router håndterer batteridata uden fejl.

Metode:

Testen udføres på et nRF5340 development board, forsynet med to 1.5V AA-batterier i serie. Målingen af batterispændingen udføres direkte via ADC-indgangen på boardet.

Testen initieres ved at trykke på "Battery Request"-knappen i WEB-GUI. Dette sender en forespørgsel via UDP til end-device. Når beskeden modtages, udføres en batterimåling, hvor spændingsværdien beregnes og konverteres til millivolt (mV).

Resultatet pakkes herefter i en JSON-besked, som sendes tilbage til border routeren, der videresender data til WEB-GUI.

I WEB-GUI præsenteres batteristatus både som en numerisk spænding og som en farveindikator efter følgende niveauer:



≥ 2100 mV: Grøn (god)	≥ 1900 mV: Gul (middel)	< 1900 mV: Rød (lav)	≤ 100 mV: Ingen batterier
------------------------------	--------------------------------	--------------------------------	----------------------------------

Under testen blev batterierne fjernet og udskiftet for at verificere, at systemet korrekt registrerer ændringer i spændingsniveauet. GUI opdaterede farveindikatoren i takt med de ændrede måleværdier, og ændringerne kunne ligeledes observeres som terminal udskrifter på border routeren.

Konklusion:

Testen bekræfter, at batterimålingen via ADC fungerer korrekt, og at de målte data transmitteres og vises som forventet i WEB-GUI. Kommunikation mellem end device og border router foregår fejlfrit, og visningen af batteristatus opdateres pålideligt i forhold til de faktiske spændingsniveauer. Systemet lever dermed op til kravene for præcis og pålidelig batteriovervågning.



7.4.2 Strømforbrugs test

Testtype: Fysisk.

Metode: Brug af nRF power profiler 2.

Resultat: Viser et lavt forbrug, men med klare tegn for optimering.

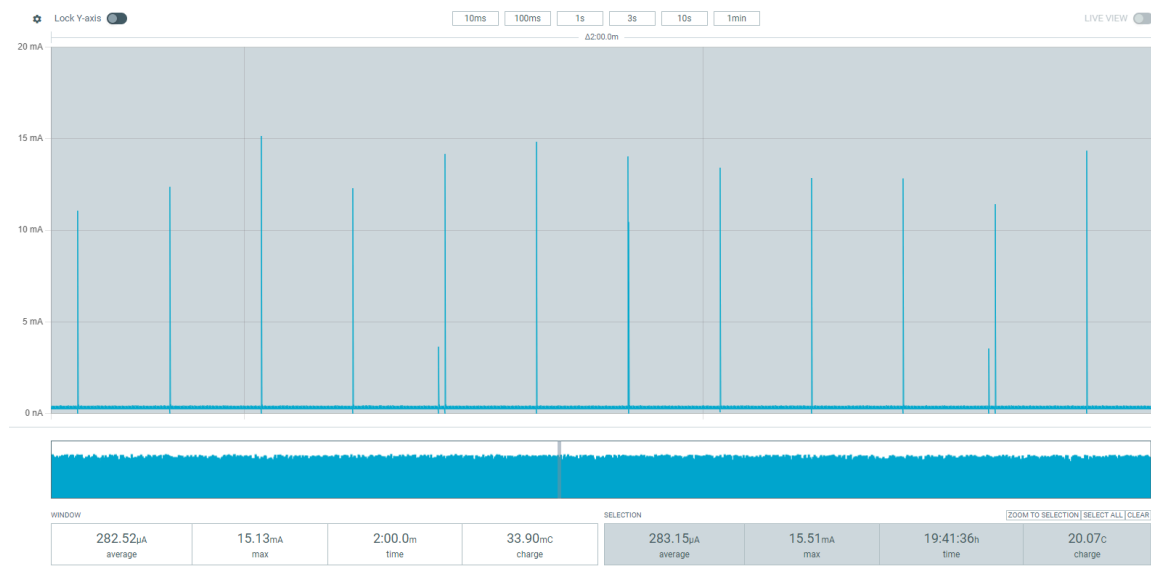
Formål:

Formålet med testen var at måle strømforbruget for end-enheden under normal drift over en længere periode. Målingen blev udført for at opnå et præcist billede af enhedens gennemsnitlige forbrug samt de periodiske strømspidser, der opstår ved kommunikation. Resultaterne bruges som grundlag for estimering af batterilevetid og fremtidige optimeringer af strømstyring.

Metode:

Testen blev udført ved hjælp af et Power Profiler Kit 2 (PPK2), som målte strømforbruget kontinuerligt i ca. 24 timer. Enheden, som indgår i et trådløst brandalarmsystem, blev testet i normal driftstilstand. Enheden fungerer som end device og er konfigureret med en pollperiode på 10 sekunder, hvilket betyder, at radioen aktiveres kortvarigt hvert 10. sekund for at kommunikere med netværket og derefter vender tilbage til sleep-mode. Under testen var kun denne funktion aktiv for at simulere den forventede drift i et typisk miljø.

Resultat:



Figur 38: nRF power profiler måling af strøm

Af målingen (se figur) fremgår det, at enheden har et gennemsnitligt strømforbrug på ca. 282,5 µA, med maksimale strømspidser på omkring 15,1 mA i de korte perioder, hvor radioen aktiveres. Den samlede måletid udgjorde ca. 19 timer og 41 minutter, hvor det samlede forbrug blev registreret til 20,07 coulomb.



Det gennemsnitlige forbrug viser, at enheden bruger minimal strøm i sleep-tilstand, og at det primære forbrug sker under de periodiske radioaktiviteter. Ud fra det målte gennemsnitsforbrug kan batterilevetiden estimeres for to AA-alkaline batterier i serie (typisk kapacitet ca. 2000 mAh hver, samlet 2000 mAh).

Forbruget svarer til:

$$I_{avg} = 0,2825 \text{ mA} \rightarrow T = \frac{2000 \text{ mAh}}{0,2825 \text{ mA}} = 7079 \text{ timer} \approx 295 \text{ dage} \approx 0,8 \text{ år}$$

Dermed kan enheden forventes at kunne fungere i omkring 0,8 år på et sæt AA-batterier under de givne driftsforhold.

Tiden som målingen har kørt 19 timer og 41 minutter (70.860 sekunder) svarer til at der i alt har løbet 20,07 coulomb elektriske ladninger gennem enheden.

Forbruget udregnet i ved brug af coulomb:

$$1 \text{ C} = \frac{1}{3600} \text{ Ah} = 0,2778 \text{ mAh}$$

$$20,07 \text{ C} * 0,2778 = 5,57 \text{ mAh}$$

Konklusion:

Testen viste, at enheden har et lavt gennemsnitligt strømforbrug på ca. 282 µA, med korte strømspidser, når radioen aktiveres. Med to AA-alkaline batterier (samlet kapacitet ca. 4000 mAh) giver dette en teoretisk driftstid på omkring 295 dage (ca. 0,8 år) under normale forhold. Resultatet bekræfter, at systemet er energieffektivt i sin nuværende form, og giver et solidt grundlag for videre beregninger og fremtidig strømoptimering af systemet.



7.4.3 Polling period test

Testtype: Software

Metode: Ændring af sleep cyklus

Resultat: Mindre sovetid = større forbrug

Formål:

Formålet med denne test var at undersøge, hvordan forskellige poll-perioder påvirker strømforbruget for enheden i hviletilstand, og at kvantificere energiforbruget per polling-event. Målet var at identificere, om enheden når forventet lavstrøms-tilstand (sleep-mode) når radioen er slukket, samt at evaluere poll-frekvensens effekt på gennemsnitsstrømmen.

Metode:

Enheden blev tilsluttet Power Profiler Kit II (PPK2) i Power Output ON mode, så PPK2 leverede strøm og samtidig målte forbruget. Følgende procedure blev udført:

1. Baseline måling uden polling-events, kun idle/standby, for at fastslå enhedens hvilestrøm.
2. Målinger med forskellige poll-perioder:
 - 200 ms
 - 1 s
 - 5 s
 - 20 s
3. For hver poll-periode blev gennemsnitligt strømforbrug logget over flere polling-cykler.
4. Målingerne blev gennemført ved høj sample-rate for at fange korte spidsstrømme, der opstår når radioen vækkes midlertidigt under polling.

Strømforbrug blev registreret i mikroampere (μA) og sammenlignet med baseline.

Resultat:

Poll-period	Gennemsnitligt strømforbrug
Baseline	284 μA
20 sekunder	284.94 μA
5 sekunder	288.62 μA
1 sekund	301.89 μA
200 millisekunder	360.8 μA



Konklusion:

Testen viser tydeligt, at poll-perioden har direkte effekt på det gennemsnitlige strømforbrug: hyppigere polling øger forbruget markant, mens sjældne polling-events kun har minimal effekt.

Samtidig indikerer baseline-målingen, at enheden bruger relativt høj strøm ($\approx 284 \mu\text{A}$) selv i hvile, hvilket tyder på, at MCU og/eller radio ikke går i fuld deep-sleep, eller at perifere komponenter bidrager til standby-forbruget.

7.5 Produkt bevis

I vores video bevis viser vi en fuld gennemgang af brandalarmsystemet. Formålet er at gøre traditionel brandalarmering intelligent ved at lade alarmer kommunikere trådløst, reagere hurtigere og dele kritisk information på tværs af enheder så det kan bruges i ældre bygninger. Løsningen danner grundlag for et mere sikkert og skalerbart alarmsystem.

Link til video bevis:

<https://youtu.be/OyxrX3Uqla8>



8. Perspektivering

I løbet af projektet er vi kommet frem til nogle forskellige overvejelser som kan være relevante for videreudvikling af vores prototype. Det hele er samlet med overskrifter for perspektiver om efterfølgende arbejdsgange, der med fordel kunne komme i fokus for at nærme sig et mere færdigt og salgbart produkt.

8.1 PCB

Da vores prototype er bygget med development boards og tager udgangspunkt i en batteridrevet brandalarm, har vi tænkt at det vil give mening at lave et samlet pcb, hvor alle funktioner er inddraget.

8.2 Sikkerhed ved fysisk manipulation.

I bilag 10.13 er der beskrevet angrebsmetoder på AES og det er en fordel at have disse for øje i udviklingen af det færdige produkt, således at det vil være umuligt at manipulere med alarmen - Det kan fx være at fastbrænde koden på enhederne og lave en mekanisme, der sikrer logning eller kode ved åbning af produktet.

8.3 Monitorering af enheders forbindelse.

Vores WEB-GUI viser allerede mange informationer, men den kunne vise endnu flere. En af de overvejelser vi har haft under projektet er en log håndtering i WEB-GUI og en overordnet systemoversigt hvor man kan monitorere forbindelsen afhængig af signalstyrke, rolle osv.

8.4 Genopladelige batterier

De fleste brand-/røgalarmer der er installeret i folks hjem er batteri drevet, hvilket også er fint, men nu hvor at bygninger fra 2020 er omfattet af den nyeste version af BR18 er det et krav at røgalarmen skal være tilsluttet til fast forsyning, men stadig have batteri som backup. Dog er det lavet så dumt at batteriet ikke bruges medmindre at strømmen går og derfor i de fleste tilfælde bare aflades i stedet. Derfor ville det give godt mening at tilføje en form for ladning af et genopladeligt batteri enten via forsyning på de fastmonteret enheder eller ved brug af solpaneler til at høste energi udefra for at øge batterilevetiden



9. Konklusion

Formålet med projektet har været at undersøge, hvordan et brandalarmsystem bygget på Thread-kommunikation kan designes og demonstreres som en mulig løsning til eksisterende bygninger opført før 2020, med fokus på pålidelig alarmering, driftssikker kommunikation, brugervenlighed, energieffektivitet og øget tryghed i overensstemmelse med relevante dele af BR18.

Projektet har resulteret i en fungerende prototype, der demonstrerer et trådløst Thread mesh-netværk bestående af en ESP32 Border Router og 3 nRF54L15 sleep end-devices. Systemet har gennem en kravstyret testplan vist, at Thread-protokollen understøtter robust og selvorganiserende kommunikation, hvor alarmer kan distribueres på tværs af netværket, selv under varierende dækningsforhold. Robusthedstests dokumenterer, at mesh-topologien bidrager til driftssikker kommunikation, hvilket er centralt i forhold til krav om pålidelig alarmering i brandsikringssystemer.

Systemets sikkerhed er understøttet gennem anvendelse af OpenThreads indbyggede AES-128-kryptering samt HTTPS-beskyttet adgang til WEB-GUI. De gennemførte sikkerhedstests viser, at uautoriserede enheder afvises, og at kommunikationen mellem enheder og brugergrænseflade er beskyttet. Dette bidrager til systemets samlede pålidelighed og understøtter krav om datasikkerhed og driftssikkerhed.

For at sikre brugervenlighed er systemet udstyret med en WEB-GUI, som giver et samlet overblik over enhedernes status, batteriniveau og alarmtilstand. Testene viser, at brugerfladen er intuitiv at anvende og gør det muligt at udføre både alarm test og hush-funktioner på en overskuelig måde. Den løbende monitorering i form af visuelle statusindikatorer øger brugerens tryghed i daglig drift. Brugeren kan monitorere om der er lav batterispænding eller om der er aktive alarmer.

Energieffektivitet er opnået gennem valg af sleep end-devices og anvendelse af OpenThreads understøttelse af polling-intervaller. Batteri tests viser, at energiforbruget kan reduceres markant ved at justere polling-perioden, hvilket forlænger systemets forventede levetid og reducerer behovet for vedligeholdelse. Samtidig illustrerer testene den nødvendige balance mellem lavt strømforbrug og systemets reaktionstid, som er en central overvejelse i batteridrevne alarmsystemer.

Samlet set demonstrerer projektet, at et brandalarmsystem bygget på OpenThread kan fungere som en fleksibel og energieffektiv løsning til eksisterende bygninger opført før 2020. Prototypen understøtter centrale funktionelle og tekniske krav relateret til pålidelig alarmering, sikker kommunikation, brugervenlighed og energieffektiv drift, som kan relateres til relevante dele af BR18. Projektet dokumenterer dermed det teknologiske potentiale i en trådløs, softwaremæssig tilgang til brandsikring, samtidig med at det tydeliggør, at yderligere arbejde med fysiske sensorer, certificeret hardware og praksisnære tests vil være nødvendigt for en fuldt færdig løsning.



10. Bilag

10.1 Figurliste

Nr	Titel
1	<i>Figur 1: De første tanker debatteres</i>
2	<i>Figur 2: Første udkast til simulatoren</i>
3	<i>Figur 3: Border Router ESP32C6/nRF52840 vs ESP32S3H2</i>
4	<i>Figur 4: Kredsløbsanalyse af færdiglavet alarm</i>
5	<i>Figur 5: WBS struktureret omkring den itreative proces</i>
6	<i>Figur 6: Blokdiagram af Thread simulator</i>
7	<i>Figur 7: WEB GUI</i>
8	<i>Figur 8: Border Router sender et payload - BREQ</i>
9	<i>Figur 9: OpenThread stack</i>
10	<i>Figur 10: Zephyr stack</i>
11	<i>Figur 11: ESP-IDF - FreeRTOS</i>
12	<i>Figur 12: AES enkryptering</i>
13	<i>Figur 13: Embedded software hierakidiagram</i>
14	<i>Figur 14: dBm signal styrkemeter</i>
15	<i>Figur 15: Neighbor table på Border router</i>
16	<i>Figur 16: Rækkevidde testmiljø</i>
17	<i>Figur 17: Rækkevidde testresultat</i>
18	<i>Figur 18: Rækkevidde end-device afstandsmetode</i>
19	<i>Figur 19: Rækkevidde kort over indendøres test</i>
20	<i>Figur 20: OTNS simulering automatisk logik opsætning</i>
21	<i>Figur 21: OTNS nodernes Rloc16 adresser</i>
22	<i>Figur 22: Wireshark capture - End-device 3001 søger data, men får intet svar</i>
23	<i>Figur 23: Wireshark capture - End-device finder Border router og får ny Rloc16 adresse</i>
24	<i>Figur 24: OTNS med slukket Router og ny rute til end-device med opdateret Rloc16</i>
25	<i>Figur 25: Verificering af data kryptering efter wireshark sniffing</i>
26	<i>Figur 26: Commissioner/joiner netværkslogik</i>
27	<i>Figur 27: Rogue enhed afvises af Border router ved forsøg på forbindelse via fælles kanal</i>



Nr	Titel
28	<i>Figur 28: Rogue enhed prøver commissioner/joiner logik</i>
29	<i>Figur 29: Wireshark capture af join forsøg</i>
30	<i>Figur 30: Rogue forsøg på at multicast med Mle</i>
31	<i>Figur 31: Forsøg på at logge på med forkert password til https serveren</i>
32	<i>Figur 32: Forsøg på at logge på http serveren</i>
33	<i>Figur 33: Første login på https til WEB-GUI</i>
34	<i>Figur 34: Opdatering af WEB-GUI efter sendt BREQ</i>
35	<i>Figur 35: Alarmstatus WEB-GUI og end-device</i>
36	<i>Figur 36: knaptryk detekteret ved SED2</i>
37	<i>Figur 37: SED1 opdateret til state Alarm</i>
38	<i>Figur 38: nRF power profiler måling af strøm</i>



10.2 Referenceliste

Bygningsreglementet 2018, kap. 5, § 92 [2019, 21/10-2025]

https://www.bygningsreglementet.dk/historisk/br18_version3/tekniske-bestemmelser/05/vejledninger/generel_brand/enfamiliehuse/tiltag-til-at-goere-opmaerksom-paa-en-brands-opstaaen/

Bygningsreglementet - Brandtekniske installationer. Available:

https://www.bygningsreglementet.dk/historisk/br18_version3/tekniske-bestemmelser/05/krav/93

[2019, 21/10-2025].

Geeksforgeeks (23.07.2025) - “IEEE 802.11 Architecture”

<https://www.geeksforgeeks.org/computer-organization-architecture/ieee-802-11-architecture/>

Cisco - Ethernet (IEEE 802.3) (20.08.2012)

https://www.cisco.com/c/en/us/td/docs/net_mgmt/prime/network/3-8/reference/guide/ethrnt.html

ESP Thread Border Router SDK (02.04 2025)

<https://docs.espressif.com/projects/esp-thread-br/en/latest/codelab/index.html>

ESP H2 - OpenThread (23.04.2023)

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32h2/api-guides/openthread.html>

Hui, Jonathan (18.01.2019) “Framing Protocol”

<https://chromium.googlesource.com/external/github.com/openthread/openthread/%2Brefs/tags/upstream/thread-reference-20180926/doc/spinel-protocol-src/spinel-framing.md>

Geelen, Pieter (11.07.2025)

“Thread Radio Technology: Inside the IEEE 802.15.4 Protocol and RCP Stack”

<https://medium.com/engineering-iot/thread-radio-technology-inside-the-ieee-802-15-4-protocol-and-rcp-stack-d4f8444a2ce>

OpenThread Documentation:

<https://openthread.io/>

Hisham, Anver (07.02.2025) - “What is modulation?”

<https://www.wirelessexplained.com/what-is-modulation/>

Sun, Sunny (24.09.2018) “DSSS - Direct Sequence Spread Spectrum”

<https://www.youtube.com/watch?v=-1mxYWvfVWQ>

Zephyr stack (25.11.2025) “Network Stack Architecture”

<https://docs.zephyrproject.org/latest/connectivity/networking/net-stack-architecture.html>



Armasu, Lucian (13.07.2015)

"Thread Protocol: Enabling Secure Mesh Networks For Smart Home Devices"

<https://www.tomshardware.com/news/thread-mesh-networking-protocol-homes,29556.html>

Docs.nordicsemi.com (11.10.2024)

"nRF Sniffer for 802.15.4 based on nRF52840 with Wireshark"

https://docs.nordicsemi.com/bundle/nrf5_sdk_thread_zigbee/page/nrf802154_sniffer.html

Cohen, Ran (14.04.2024)

"7 Types of Testing Environments and Best Practices for Yours"

<https://configu.com/blog/7-types-of-testing-environments-and-best-practices-for-yours/>

Szczys, Mike & Gammel, Chris (15.09.2021)

"Using Wireshark to troubleshoot Thread Networks"

<https://www.youtube.com/watch?v=WsRMh5yPoss>

O'Flynn, Colin (14.10.2020) *"ECED4406 0x207 - AES Introduction"*

<https://www.youtube.com/watch?v=yaG1DHSresk>

Sluiter, Shad (23.08.2019)

"How does AES encryption work? Advanced Encryption Standard"

<https://www.youtube.com/watch?v=InKPoWZnNNM>

St. Pierre, James A. (09.05.2023) *"Advanced Encryption Standard (AES)"*

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>

Bhalodia, Chirag (19.05.2022) *"How to solve AES Sub Bytes"*

<https://www.youtube.com/watch?v=xZZ5ywgOWeg>

AES128 - Simulering *"CrypTool-Online"*

<https://legacy.cryptool.org/en/cto/aes-animation>

OpenThread.io (24.01.25) *"Packet Sniffing with Pyspinel"*

<https://openthread.io/guides/pyspinel/sniffer>

NewAE Technology (29.07.2019)

[http://wiki.newae.com/V4:Tutorial_B5_Breaking_AES_\(Straightforward\)](http://wiki.newae.com/V4:Tutorial_B5_Breaking_AES_(Straightforward))

Fässler, Fabian (02.06.2017)

"Breaking AES with ChipWhisperer - Piece of Scafe (Side Channel Analysis 100)"

<https://www.youtube.com/watch?v=Fktl4qSjaE>

O'Flynn, Colin (25.11.2020) *"ECED4406 - 0x501 Power Analysis Attacks"*

<https://www.youtube.com/watch?v=2iDLfuEBcs8>

Nordic semi conductors (14.5 2025) Power profiler opsætning nRF54I15DK

https://docs.nordicsemi.com/bundle/nwp_049/page/WP/nwp_049/setup_board_nrf54I15.html



Virgin Media Edit (18.3 2025)

"What is RSSI?"

<https://www.virginmedia.com/the-edit/glossary/what-is-rssi>

Zephyr Documentation (23.12.2024)

<https://docs.zephyrproject.org/latest/index.html>

Espressif IDF Documentation

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>

everythingRF (6.11 2022) What is OQPSK Modulation?

<https://www.everythingrf.com/community/what-is-oqpsk-modulation>

WSN & IoT (21.12 2021)

"Thread network technology"

<https://www.youtube.com/watch?v=r4ORGVOcvpQ&list=PLoKOKJqggHs9AK8cMkH3Nqyybzfv0q23D>

Virgin Media Edit (18.03.2025) *"What is RSSI? Understanding Your RSSI Level"*

<https://www.virginmedia.com/the-edit/glossary/what-is-rssi>

Rao, Venkateswara M. (Vol12, No 5 - 2021, p.631)

"Improving Packet Delivery Ratio in Wireless Sensor Network with Multi Factor Strategies"

https://thesai.org/Downloads/Volume12No5/Paper_75-Improving_Packet_Delivery_Ratio.pdf

OpenThread(25.7 2025)

"Simulate Thread Networks using OTNS"

<https://openthread.io/codelabs/openthread-network-simulator#1>

Apalrd, Mark (27.03.2025)

"Building an Open-Source THREAD Network for the Internet of Things"

<https://www.youtube.com/watch?v=D9bINRhtN7k>

OpenThread CLI (12.12 2025)

"OTNS CLI Reference"

<https://github.com/openthread/ot-ns/blob/main/cli/README.md>

GeeksforGeeks (23.07.2025) *"Bellman–Ford Algorithm"*

<https://www.geeksforgeeks.org/dsa/bellman-ford-algorithm-dp-23/>



10.3 Bygningsreglement

Da projektets formål er at udvikle et trådløst brandsikringssystem, der kan øge sikkerheden i eksisterende bygninger, er det væsentligt at tage udgangspunkt i de krav, der fremgår af Bygningsreglementet 2018 (BR18). BR18 fastlægger de overordnede rammer for brandsikkerhed i dansk byggeri og beskriver både, hvordan brand skal forebygges, og hvordan personer skal kunne alarmeres og evakueres sikkert i tilfælde af brand (jf. BR18, kapitel 5).

Formålet med at inddrage BR18 er at sikre, at det udviklede system kan vurderes i forhold til de gældende regler for branddetektion, alarmering og tekniske installationer — også når systemet implementeres i bygninger opført før 2020, hvor ældre installationer ofte ikke lever op til nutidens krav.

I dette projekt tages der særligt udgangspunkt i følgende dele af BR18:

“Kapitel 5 – Brand

Indeholder de generelle bestemmelser for brandsikkerhed i bygninger, herunder krav til brandtekniske installationer, detektion og varsling (§ 82–84). Disse paragraffer danner grundlag for, hvordan et alarmsystem skal kunne detektere røg og varme samt effektivt varsle personer ved brand.” (Kilde: BR18, Kapitel 5, §§ 82–84)

Bilag 1, punkt 5.2 – Krav til boliginstallationer

Angiver, at røgalarmer skal placeres i flugtveje og opholdsrum, og at de skal kunne advare alle personer i boligen ved brand. Dette danner grundlag for systemets trådløse kommunikation mellem noder, hvor hver alarm skal kunne videresende signalet til de øvrige enheder for at sikre fuld dækning.

(Kilde: BR18, Bilag 1, punkt 5.2)

Kapitel 5, § 82, stk. 2.1 – Tiltag til at gøre opmærksom på en brands opståen

Bygningsreglementet indeholder præaccepterede løsninger for røgalarmanlæg, som projektets produkt tager udgangspunkt i. Da fokus er rettet mod øget sikkerhed i bygninger opført før 2020, er det dog ikke et krav, at systemet fuldt ud opfylder alle bestemmelser i BR18. Eksempelvis indgår fast forsyning ikke som et krav i projektet, da dette ville begrænse anvendeligheden i ældre bygninger. Løsningen skal desuden kunne installeres uden autoriseret el-installatør for at fremme brugervenlighed og nem implementering.

(Kilde: BR18, Kapitel 5, § 82, stk. 2.1)

Henvisning til standard DS/EN 14604 – Røgalarmer

Standarden fastlægger krav til konstruktion, ydeevne og test af røgalarmer. Projektets prototype tilstræber at opfylde de funktionelle krav i denne standard, herunder pålidelig alarmgivning, lav fejlrate og tydelig indikering af batteristatus.

Ved at anvende ovenstående dele af BR18 som reference sikres, at det udviklede system kan vurderes ud fra de gældende danske sikkerhedskrav. Samtidig tages der højde for brugervenlighed, energieffektivitet og trådløs kommunikation, som beskrevet i projektets problemformulering.



10.4 Detaljerede kravspecifikationer:

Kravspecifikation: Robusthedstest (K-01)

Kravtype: Funktionelt

Beskrivelse:

Systemets Thread-baserede mesh-netværk skal udvise korrekt rækkevidde, robusthed og selvhelende kommunikation ved varierende linkkvalitet, midlertidige forbindelsesafbrydelser og begrænset direkte rækkevidde mellem noder. Netværket skal automatisk kunne opretholde eller genetablere forbindelser ved hjælp af Thread's mesh-routingmekanismer uden manuel indgriben.

Begrundelse:

Robusthed i et Thread-netværk afhænger ikke alene af den fysiske rækkevidde mellem to enheder, men af netværkets evne til at tilpasse sig ændrede forbindelsesforhold. Thread-protokollen er designet som et selvorganiserende mesh-netværk, hvor routing dynamisk beregnes ud fra linkkvalitet og path cost ved hjælp af en tilnærmet Bellman-Ford afstandsvektor algoritme.

I praksis kan signalforringelse, forhindringer eller midlertidige udfald føre til, at direkte kommunikation mellem noder ikke er mulig. Robusthed testes derfor ved at undersøge, om netværket fortsat kan levere pålidelig datakommunikation gennem alternative ruter og genetablere forbindelser uden datatab.

Verifikation:

Krav K-01 verificeres gennem en kombination af fysiske målinger og simulerede mesh-scenarier:

RSSI (dBm) vurdering

PDR (Packet Delivery Ratio) til vurdering af kommunikationsstabilitet

Observation af forbindelsesbrud og genetableringstid

Verifikation af automatisk routing ved manglende direkte forbindelse

Bekræftelse af korrekt multi-hop forwarding

Analyse af routingbeslutninger via MLE Route TLVs

Acceptkriterier:

- PDR ≥ 95 % ved stabil direkte forbindelse
- Netværket etablerer automatisk alternativ rute ved linkafbrydelse
- Forbindelse genetableres uden manuel intervention
- Routing sker i overensstemmelse med Thread's afstandsvektor-logik

Konklusion:

Testen betragtes som bestået, når der opnås stabil kommunikation inden for den specificerede afstand, og målinger af RSSI og PDR dokumenterer en robust og vedvarende forbindelse mellem Thread-noderne.



Kravspekifikation: Sikkerhed (K-02)

Kravtype: Ikke-funktionelt krav

Beskrivelse:

Systemets kommunikation i Thread-netværket skal være beskyttet af korrekt implementeret AES-128-kryptering og autentificeringsmekanismer. Løsningen skal sikre fortrolighed og integritet i datatransmissionen mellem enheder og Border Router, og forhindre uautoriseret adgang eller manipulation af netværkskontrollfunktioner. Implementeringen skal understøtte sikker nøgleudveksling og robust håndtering af join-flows.

Begrundelse:

Kompromitteret kommunikation i et mesh-miljø kan føre til tab af kontrol, datalæk og driftstab. Selvom Thread anvender etablerede sikkerhedsmekanismer, skal den konkrete implementering verificeres i praksis. Formålet er at dokumentere, at kryptering og adgangskontrol fungerer korrekt, at uautoriserede tilslutningsforsøg afvises, og at relevante hændelser logges og kan efterspores.

Verifikation:

Sikkerheden verificeres ved tre hovedtests, der kombinerer analyse og praktiske demonstrationer på proof-of-concept niveau:

1. Krypteringsverifikation — dokumentere at nytte-data på netværksniveau fremstår krypterede og ikke kan aflæses i klartekst.
2. Uautoriseret-tilslutnings-simulering + POC side-channel vurdering — simulere rogue/joiner-enheder for at verificere adgangskontrol, og beskrive/udføre begrænsede proof-of-concept observationer af side-channel-fænomener (EM, power, timing) som indikation for implementeringssvagheder.
3. Printverifikation — kontrollere at alle uautoriserede forsøg og relevante sikkerhedshændelser registreres på Border Router med tilstrækkelig metadata (tid, enheds-ID, hændelsestype).

Acceptkriterier:

- Sniffer/Wireshark-analyse viser kun krypterede nytte-data; ingen klartekst kan observeres.
- Forsøg på at tilslutte en uregistreret enhed uden korrekt nøgle resulterer i afvisning; der sker ingen nytte-dataudveksling.
- Alle uautoriserede tilslutningsforsøg og relevante hændelser printes i Border Router med tydelig tidsstempling og identifikation.

Konklusion:

Kravet vurderes opfyldt, når kryptering, adgangskontrol og prints kan dokumenteres, og når uautoriserede tilslutningsforsøg gentagne gange afvises uden kompromittering af netværkets data eller funktionalitet.



Kravspekifikation: WEB-GUI (K-03)

Kravtype: Funktionelt krav

Beskrivelse:

Systemet skal indeholde en webbaseret brugergrænseflade, hvor brugeren har fuld kontrol og overblik over alle enheder i Thread-netværket. GUI skal muliggøre både styring og monitorering i realtid. Al kommunikation mellem bruger og Border Router skal ske via en krypteret HTTPS-forbindelse for at beskytte data mod uautoriseret adgang. Systemet skal desuden logge alle adgangsforsøg, herunder afviste eller uautoriserede login-forsøg, med tidsstempel, IP-adresse og beskrivelse af hændelsen.

Begrundelse:

En centraliseret og brugervenlig grænseflade er nødvendig for at kunne overvåge, administrere og teste systemets funktionalitet. GUI fungerer som forbindelsesled mellem brugeren og Border Routeren og skal præsentere netværkets tilstand på en måde, der gør det muligt at identificere fejl, se signalstyrke, aktivere/deaktivere enheder og kontrollere systemets samlede drift. Implementeringen af HTTPS-kommunikation sikrer, at data mellem bruger og system overføres fortroligt og ikke kan opsnappes af uvedkommende. Logningen af hændelser er afgørende for at kunne dokumentere sikkerhedsbrud, fejl eller mislykkede adgangsforsøg og bidrager dermed til systemets samlede sikkerhed og sporbarhed.

Verifikation:

GUI-funktionaliteten verificeres gennem funktionelle og sikkerhedsmæssige testscenarier:

- Brugeren skal kunne se og identificere alle tilsluttede enheder i netværket.
- Brugeren skal kunne sende kontrolkommandoer til enheder og modtage visuel feedback på handlinger.
- Systemet skal reagere inden for en acceptabel responstid.
- Al kommunikation med GUI skal ske via HTTPS-protokollen, og forsøg på HTTP-adgang skal afvises eller omdirigeres.

Acceptkriterier:

- Fuld kontrol over alle enheder.
- Responstid inden for 10 sekunder.
- HTTPS login adgang

Konklusion:

Testen betragtes som bestået, når GUI giver fuld funktionel styring og realtidsmonitorering af alle aktive enheder i Thread-netværket via en sikret HTTPS-forbindelse, og når alle uautoriserede forsøg på adgang logges korrekt uden datatab eller ukorrekt statusvisning.



Kravspekifikation: Batterilevetid (K-04)

Kravtype: Ikke-funktionelt krav

Beskrivelse:

Systemets trådløse enheder skal være designet til at kunne fungere på batteridrift i en forventet levetid på 6 måneder under normale driftsforhold. Det skal verificeres, om denne levetid kan opnås uden udskiftning, eller om der kræves periodisk vedligeholdelse for at nå den specificerede driftstid. Enhederne skal implementere energibesparende funktioner såsom sleep-tilstand, reduceret transmissionsfrekvens og effektiv håndtering af sensordata. Systemet skal desuden kunne logge relevante data om batteristatus og forbrug for at muliggøre overvågning af levetid og tidlig varsling ved lav spænding.

Begrundelse:

Lang batterilevetid er afgørende for at sikre lavt vedligeholdelsesbehov og høj driftssikkerhed i et trådløst sensor- og aktuatornetværk. Da Thread-enheder ofte anvendes i lavenergi-applikationer, skal systemet optimeres med hensyn til strømforbrug, sendefrekvens, vågetid og transmissionsstyrke. Logning og overvågning af batteritilstand giver mulighed for at opdage unormalt forbrug og foretage rettidige justeringer. Evaluering af batteriets ydeevne skal afdække, om designet lever op til levetidskravet eller kræver ændringer i hardware, firmware eller driftsstrategi.

Verifikation:

Batterilevetiden verificeres gennem målinger, simuleringer og analyser af strømforbrug i typiske driftscyklusser:

- Måling af gennemsnitligt strømforbrug ved aktiv, sovende og transmissions-tilstand.
- Beregning af estimeret levetid baseret på batterikapacitet, poll period og forbrugsmønster.
- Analyse af, hvordan wake-up-intervaller og sendefrekvens påvirker samlet energiforbrug.
- Vurdering af nødvendige vedligeholdelsestiltag for at opretholde specificeret levetid.

Acceptkriterier:

- Batterimåling kan udføres præcist via burstmåling
- En batterilevetid på 6 måneder med 2xAA 1,5v Alkaline

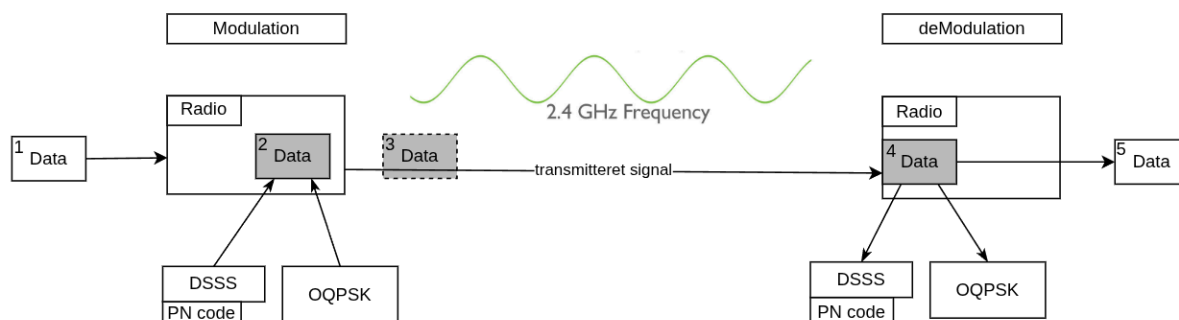
Konklusion:

Testen betragtes som bestået, når beregninger og praktiske tests dokumenterer, at enheden opnår den krævede levetid på 6 måneder, og at systemet kan overvåge og logge strømforbrug samt understøtte en realistisk vedligeholdelsesstrategi for at sikre denne levetid.



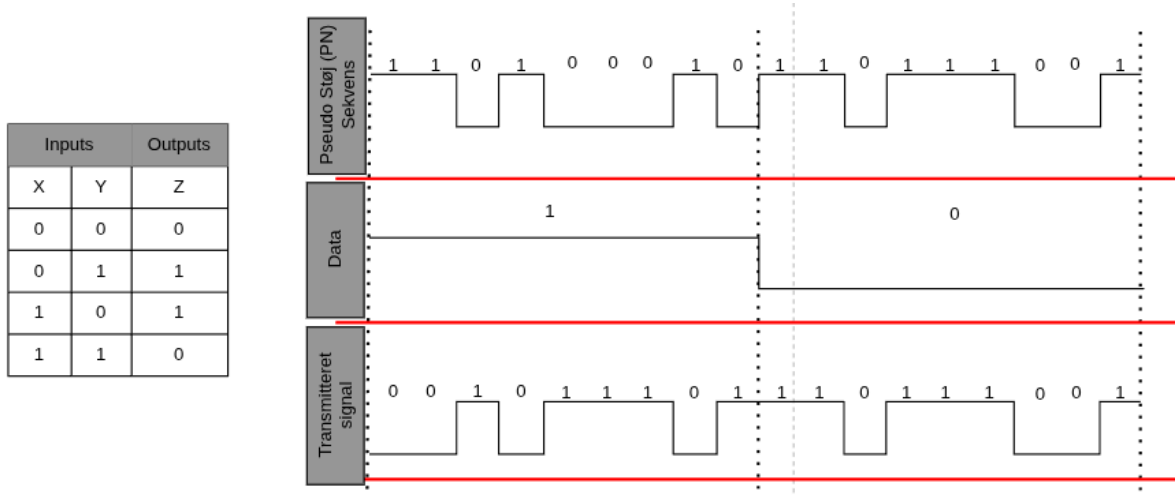
10.5 O-QPSK Modulation og DSSS i IEEE 802.15.4

Figuren herunder illustrerer den overordnede signalvej i et IEEE 802.15.4 - 2,4 GHz radiosystem, fra digital data ved afsender til genskabt data hos modtageren. På transmittersiden behandles den indgående databitstrøm først gennem Direct Sequence Spread Spectrum - DSSS, hvor data spredes ved hjælp af en pseudo-tilfældig PN-kode for at øge robustheden mod støj og interferens. Herefter moduleres signalet ved offset quadrature phase shift keying - O-QPSK, som sikrer mere stabile faseovergange og et Radiofrekvent-venligt signal. Det modulerede signal opkonverteres og udsendes som et 2,4 GHz radiosignal.



På modtagersiden gennemløber signalet den omvendte proces: Radiofrekvens-signalet nedkonverteres, demoduleres ved O-QPSK, og den anvendte PN-kode benyttes til despreading af signalet, så den oprindelige databitstrøm kan genskabes. Figuren giver dermed et samlet overblik over samspillet mellem DSSS, O-QPSK-modulation og radiofrekvent transmission i et Thread-netværk.

Ved DSSS spredning multipliceres eller XOR'es det oprindelige databit med en hurtig pseudo-tilfældig PN-kode - pseudo-støj. Hvert databit bliver dermed spredt ud over flere chips, hvilket øger signalets båndbredde. PN-koden gør signalet mere robust over for støj og interferens. Modtageren bruger den samme PN-kode til at despread'e signalet og genskabe de oprindelige data.



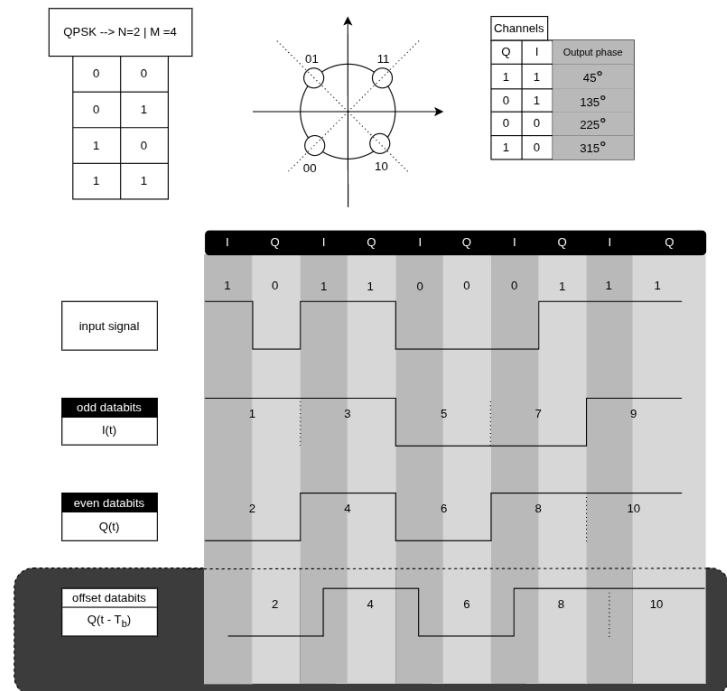
OQPSK-modulationen¹² på tegningen viser, hvordan en normal QPSK-bitstrøm omformes, så fase ændringerne i signalet bliver mindre og mere stabile. Den indgående bitstrøm opdeles først i to parallelle kanaler:

QPSK svarer til $N=2$ og $M=4$, altså $2^2 = 4$ mulige kombinationer, men for at undgå krydsning på samme tid skal bits forskydes med en halv bit og det gør OQPSK

Den klassiske QPSK-cirkel: fire punkter med faserne $+45^\circ$, $+135^\circ$, -135° og -45° . Hvert punkt repræsenterer to bits (I , Q).

OQPSK tager disse bits og deler dem op:

$I(t)$ får de ulige bitpositioner
 $Q(t)$ får de lige bitpositioner



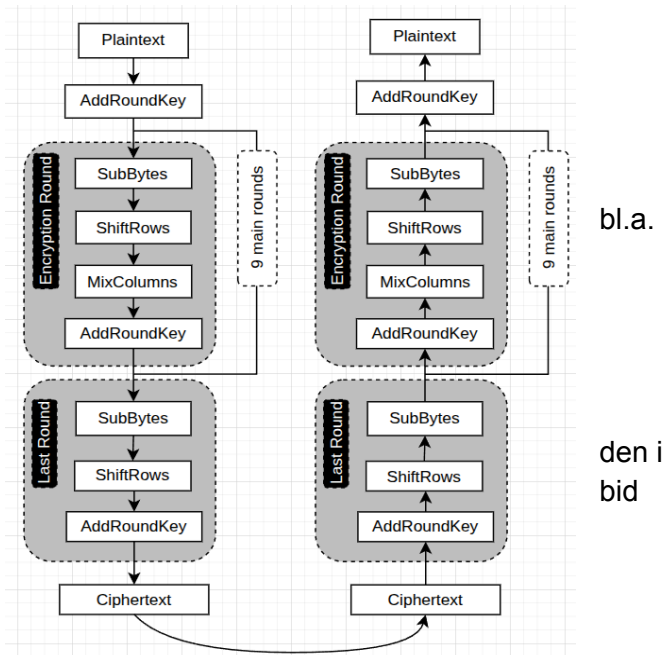
Det der gør QPSK til Offset-QPSK: Q-kanalen forsinkes et halvt symbol ($t - T_b$). Det betyder, at I og Q aldrig skifter samtidig.

Konstellationsdiagrammet forbliver det samme som i QPSK med fire faser: $\pm 45^\circ$ og $\pm 135^\circ$, men fordi kun én kanal ændrer sig ad gangen, bliver faseovergangen mellem punkterne altid begrænset til maks. 90° . Resultatet er et mere stabilt signal uden amplitude-nulpassager og uden de kraftige fasehop, der kan skabe problemer i RF-forstærkning. Tegningen illustrerer denne proces fra bitinddeling over forskydningen i tid til den resulterende glattere faseoutputkurve.

¹² <https://www.wirelessexplained.com/what-is-modulation/>

10.6 AES

AES står for Advanced Encryption Standard - en symmetrisk krypteringsmetode, som betyder, at samme nøgle bruges både til at kryptere og dekryptere data. Det er en international standard, godkendt af NIST i USA, og bruges i alt fra Wi-Fi - WPA2/WPA3 til Thread-netværk, banker, og regeringssystemer. AES krypterer data i blokke af 128 bits (16 bytes). Hvis man sender en UDP besked eller datapakke - AES tager bidder på 128 bits og behandler hver gennem en række matematiske transformationer baseret på en hemmelig nøgle.



AES 128 - 128 bit - 16 bytes - 10 runder

AES 192 - 192 bit - 24 bytes - 12 runder

AES 256 - 256 bit - 32 bytes - 14 runder

Figur x: Enkryptering og dekryptering i AES128

Hver runde i AES udfører fire typer operationer på dataene:

AddRoundKey – Dataen kombineres med en del af den hemmelige nøgle.

SubBytes – Hver byte erstattes via en fast opslagstabel - S-Box, som gør det svært at forudsige output.

ShiftRows – Flytter rækker i datablokken for at blande information.

MixColumns – Kombinerer kolonner ved hjælp af matematik (Galois Field-operationer) for at skabe yderligere diffusion.

Ved dekryptering sker det samme – men i omvendt rækkefølge. Da AES bruger samme nøgle til begge veje, skal denne nøgle holdes hemmelig. Hvis nogen får fat i nøglen, kan de både dekryptere og kryptere alt. Derfor er nøgleudveksling og nøglebeskyttelse ekstremt vigtig i systemer som f.eks. Thread, TLS, eller VPN.

Eksempel konvertering med beskeden "HELLO" - konverteret til bytes 48 45 4C 4C 4F - AES arbejder i blokke på 16 bytes, så resten fyldes op med padding:
48 45 4C 4C 4F 00 00 00 00 00 00 00 00 00 00 00

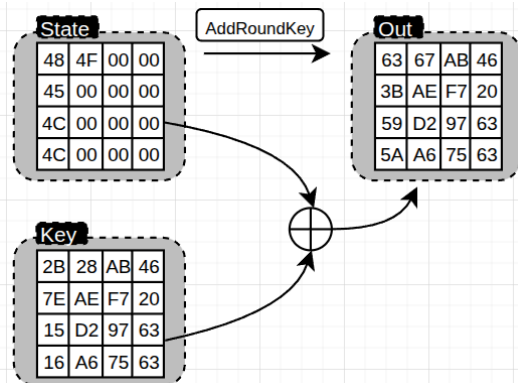
AES-128 bruger en nøgle på 16 bytes, denne kan ændres (128 bit): 2B 7E 15 16 28 AE D2 A6 AB F7 97 75 46 20 63 63

AES organiserer data i en 4x4 matrix, kaldet state. Hver byte i state kombineres med en del af nøglen ved hjælp af XOR (eksklusiv-ELLER):

48	4F	00	00
45	00	00	00
4C	00	00	00
4C	00	00	00

I operationen *AddRoundKey* sker der dette pr. byte, UDP state matrix bliver lagt sammen med thread network key ved en XOR sammenlægning som beskrevet i tabellen og operationen herunder.

State	Key	$\oplus$ sammenlægning	$\oplus$
0x48	0x2B	01001000 $\oplus$ 00101011 = 01100011	0x63
0x45	0x7E	01000101 $\oplus$ 01111110 = 00111011	0x3B
0x4C	0x15	01001100 $\oplus$ 00010101 = 01011001	0x59
0x4C	0x16	01001100 $\oplus$ 00010110 = 01011010	0x5A
0x4F	0x28	01001111 $\oplus$ 00101000 = 01100111	0x67
0x00	0xAE		0xAE
...	...	...	...



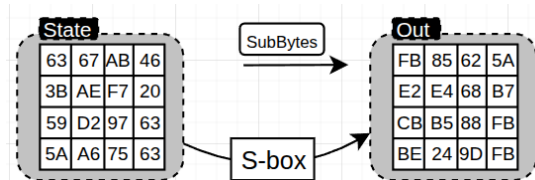
Figur x: UDP Hello med "AddRoundKey" - binær sammenlægning og ny state

Operationen *SubBytes*, bliver ny state konverteret gennem denne tabel, så det første matrix 63 går igennem tabellen hvor tallet deles i 2, det vil sige 6 = x og 3 = y her konverteres til FB og 3B bliver konverteret til E2.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

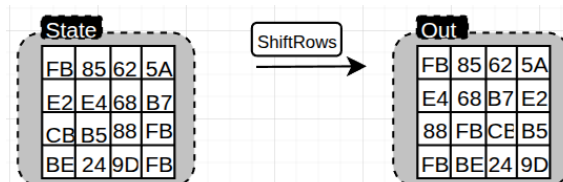
Figur x: S-box med de to første bytes konverteret

Efter konvertering får matrix en ny state.



Figur x: Fuld konvertering til ny state

Operationen *ShiftRows*, skifter rækkerne, så første række forbliver som den er, anden række flytter sig en position, tredje række flytter sig to positioner og fjerde række flytter sig tre positioner. Efter konvertering får matrix igen en ny state.



Figur x: ShiftRows fuld konvertering

Operationen *MixColumns* behandler hver kolonne separat og "mixes" med et predefined matrix, som NIST-standard definerer sådan:

Predefined			
2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

Figur x: Predefined matrix

Måden dette regnes ud på er ved at anvende Galois Field og irreducible polynomial theorem - (2^8) første række i predefined matrix skal ganges med første kolonne i state fra *shiftrows*, og anden række med anden kolonne osv.

Irreducible Polynomial Theorem, GF (2^8) $X^4 + X^3 + X + 1$

Predefine

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

State

FB	85	62	5A
E4	68	B7	E2
88	FB	CB	B5
FB	BE	24	9D

Out

A9	50	02	97
EC	FD	D9	60
E9	75	34	92
A1	DC	75	98

Result

a9	50	02	97
ec	fd	d9	60
e9	75	34	92
a1	dc	75	98

$\{02\} * \{FB\} \oplus \{03\} * \{E4\} \oplus \{01\} * \{88\} \oplus \{01\} * \{FB\} = A9$

02 =

X^7	X^6	X^5	X^4	X^3	X^2	X^1	1
0	0	0	0	0	0	1	0

FB =

X^7	X^6	X^5	X^4	X^3	X^2	X^1	1
1	1	1	1	1	0	1	1

$= X^7 + X^6 + X^5 + X^4 + X^3 + X^1 + 1$

$\{02\} * \{FB\} = X * (X^7 + X^6 + X^5 + X^4 + X^3 + X^1 + 1)$
 $= [X^8] + X^7 + X^6 + X^5 + X^4 + X^2 + X^1$
 $X^8 \text{ conv} = [X^4 + X^3 + X + 1] + X^7 + X^6 + X^5 + X^4 + X^2 + X$
 $\oplus = [X^4 + X^3 + X^1 + 1] + X^7 + X^6 + X^5 + X^4 + X^2 + X$
 $= X^7 + X^6 + X^5 + X^3 + X^2 + 1$

03 =

X^7	X^6	X^5	X^4	X^3	X^2	X^1	1
1	1	1	0	1	1	0	1

E4 =

X^7	X^6	X^5	X^4	X^3	X^2	X^1	1
0	0	0	0	0	0	1	1
1	1	1	0	0	1	0	0

$= X + 1$
 $= X^7 + X^6 + X^5 + X^2$

$\{03\} * \{E4\} = X * (X^7 + X^6 + X^5 + X^2) + 1 * (X^7 + X^6 + X^5 + X^2)$
 $= [X^8] + X^7 + X^6 + X^3 + X^7 + X^6 + X^5 + X^2$
 $X^8 \text{ conv} = [X^4 + X^3 + X + 1] + X^7 + X^6 + X^3 + X^7 + X^6 + X^5 + X^2$
 $\oplus = [X^4 + X^3 + X + 1] + X^7 + X^6 + X^3 + X^7 + X^6 + X^5 + X^2$
 $= X^5 + X^4 + X^2 + X + 1$

01 =

X^7	X^6	X^5	X^4	X^3	X^2	X^1	1
0	0	1	1	0	1	1	1

ED

ED	1	1	1	0	1	1	0	1
37	0	0	1	1	0	1	1	1
88	1	0	0	0	1	0	0	0
FB	1	1	1	1	1	0	1	1
A9	1	0	1	0	1	0	0	1

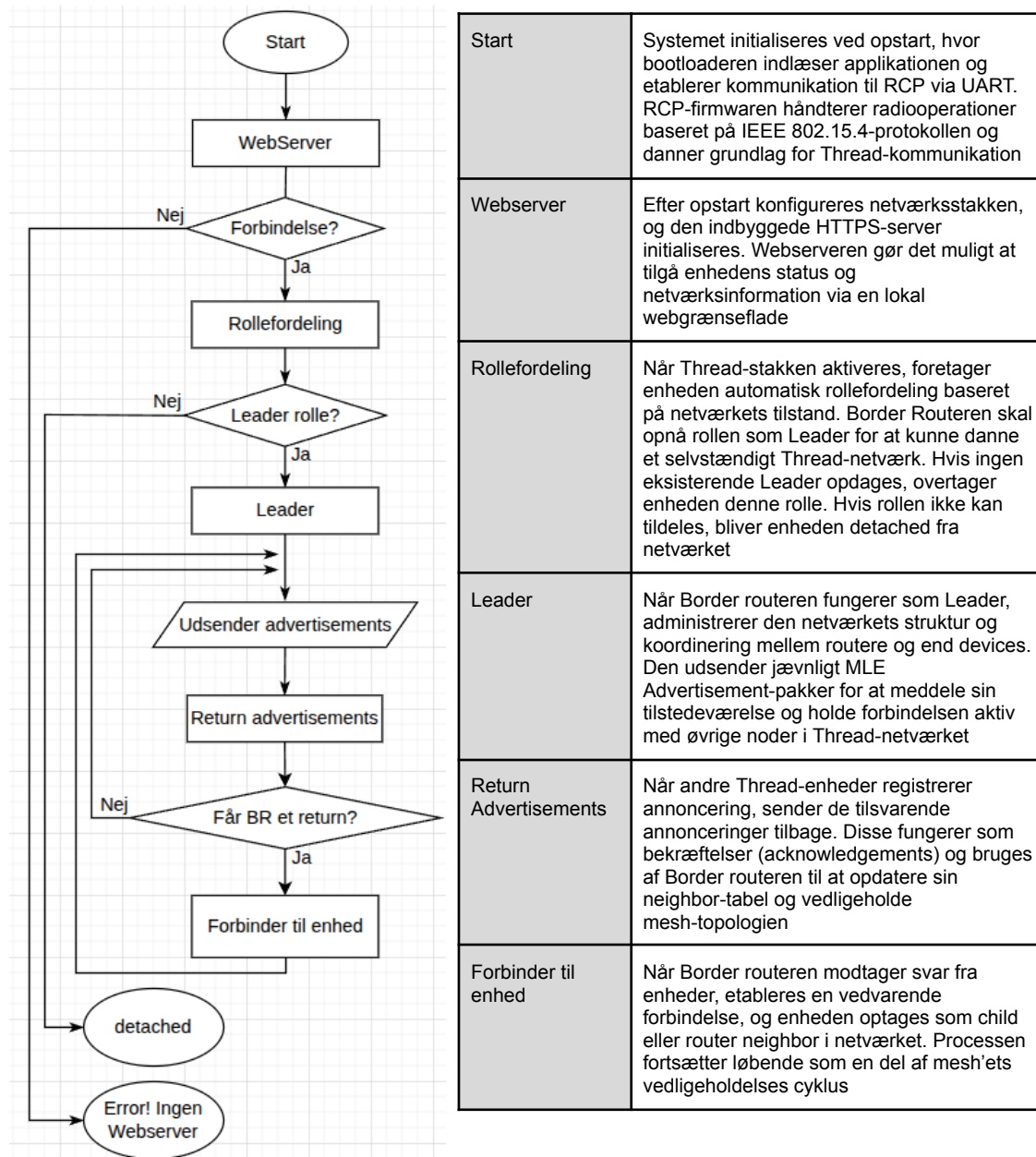
XOR:
ulige = 1
lige = 0

En fuld matrix udregning af Mixcolumns blev lavet med denne kode på github:

https://github.com/rollehenrik-a11y/aes_mixcolumn_calc/tree/39a261dbd7e39c911278d3e1294513f1e35cd701

Til slut tilføjes addRoundKey igen og dette udføres 9 gange inden der udføres operationerne fra last round - se figur x. og UDP beskeden er enkrypteret og for at dekryptere beskeden, udføres dette bare omvendt.

10.7 Opstart af Border Router



```

idf.py -p /dev/ttyACM1 monitor
Executing action: monitor
Running idf_monitor in directory /home/skradfy/ECT_OT_UDVIKLING/ot_br_wifi/basic_thread_border_router
Executing "/home/skradfy/.espressif/python_env/idf5.5_py3.12_env/bin/python
/home/skradfy/esp-idf/tools/idf_monitor.py -p /dev/ttyACM1 -b 115200 --toolchain-prefix xtensa-esp32s3-elf-
--target esp32s3 --revision 0
/home/skradfy/ECT_OT_UDVIKLING/ot_br_wifi/basic_thread_border_router/build/esp_ot_br.elf
/home/skradfy/ECT_OT_UDVIKLING/ot_br_wifi/basic_thread_border_router/build/bootloader/bootloader.elf -m
'/home/skradfy/.espressif/python_env/idf5.5_py3.12_env/bin/python' '/home/skradfy/esp-idf/tools/idf.py' '-p'
'/dev/ttyACM1'..."
--- esp-idf-monitor 1.8.0 on /dev/ttyACM1 115200
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
I (279) esp_image: segment 3: paddr=00176c8c vaddr=3fc9eda8 sizeI (458) uart: ESP_INTR_FLAG_IRAM flag not set

```



```
while CONFIG_UART_ISR_IN_IRAM is enabled, flag updated
I (458) OPENTHREAD: spinel UART interface initialization completed
I (458) UDP_BR: UDP listener started on port 54321
I (468) main_task: Returned from aESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x15 (USB_UART_CHIP_RESET),boot:0x29 (SPI_FAST_FLASH_BOOT)
Saved PC:0x4037a2be
--- 0x4037a2be: esp_cpu_wait_for_intr at /home/skradfy/esp-idf/components/esp_hw_support/cpu.c:64
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x158c
load:0x403c8700,len:0xc80
--- 0x403c8700: _stext at ???
load:0x403cb700,len:0x2f40
entry 0x403c8910
--- 0x403c8910: call_start_cpu0 at
/home/skradfy/esp-idf/components/bootloader/subproject/main/bootloader_start.c:25
I (24) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
I (24) boot: compile time Nov 5 2025 11:55:51
I (25) boot: Multicore bootloader
I (25) boot: chip revision: v0.2
I (25) boot: efuse block revision: v1.3
I (25) boot.esp32s3: Boot SPI Speed : 80MHz
I (25) boot.esp32s3: SPI Mode : DIO
I (26) boot.esp32s3: SPI Flash Size : 8MB
I (26) boot: Enabling RNG early entropy source...
I (26) boot: Partition Table:
I (26) boot: ## Label Usage Type ST Offset Length
I (27) boot: 0 nvs WiFi data 01 02 00009000 00006000
I (27) boot: 1 otadata OTA data 01 00 0000f000 00002000
I (27) boot: 2 phy_init RF data 01 01 00011000 00001000
I (28) boot: 3 ota_0 OTA app 00 10 00020000 00190000
I (28) boot: 4 ota_1 OTA app 00 11 001b0000 00190000
I (29) boot: 5 web_storage Unknown data 01 82 00340000 00082000
I (29) boot: 6 rcp_fw Unknown data 01 82 003c2000 000a0000
I (29) boot: End of partition table
I (30) esp_image: segment 0: paddr=00020020 vaddr=3c110020 size=4b848h (309320) map
I (85) esp_image: segment 1: paddr=0006b870 vaddr=3fc9a600 size=047a8h ( 18344) load
I (89) esp_image: segment 2: paddr=00070020 vaddr=42000020 size=106c64h (1076324) map
I (279) esp_image: segment 3: paddr=00176c8c vaddr=3fc9eda8 size=008a4h ( 2212) load
I (280) esp_image: segment 4: paddr=00177538 vaddr=40374000 size=16564h ( 91492) load
I (300) esp_image: segment 5: paddr=0018daa4 vaddr=50000000 size=00020h ( 32) load
I (309) boot: Loaded app from partition at offset 0x20000
I (309) boot: Disabling RNG early entropy source...
I (310) cpu_start: Multicore app
I (319) cpu_start: Pro cpu start user code
I (319) cpu_start: cpu freq: 160000000 Hz
I (320) app_init: Application information:
I (320) app_init: Project name: esp_ot_br
I (320) app_init: App version: 54e582a-dirty
I (320) app_init: Compile time: Nov 5 2025 11:55:45
I (320) app_init: ELF file SHA256: d1a18c8c5...
I (320) app_init: ESP-IDF: v5.5.1-dirty
I (321) efuse_init: Min chip rev: v0.0
I (321) efuse_init: Max chip rev: v0.99
I (321) efuse_init: Chip rev: v0.2
I (321) heap_init: Initializing. RAM available for dynamic allocation:
I (321) heap_init: At 3FCB0120 len 000395F0 (229 KiB): RAM
I (322) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (322) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (322) heap_init: At 600FE000 len 00001FE8 (7 KiB): RTCRAM
I (323) spi_flash: detected chip: gd
I (323) spi_flash: flash io: dio
I (324) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (324) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (325) main_task: Started on CPU0
I (335) main_task: Calling app_main()
I (415) mdns_mem: mDNS task will be created from internal RAM
I (415) RCP_UPDATE: RCP: using update sequence 0
I (415) uart: ESP_INTR_FLAG_IRAM flag not set while CONFIG_UART_ISR_IN_IRAM is enabled, flag updated
I (415) OPENTHREAD: spinel UART interface initialization completed
I (425) UDP_BR: UDP listener started on port 54321
```



```
I(425) OPENTHREAD:[I] P-SpinelDrive-: co-processor reset: RESET_POWER_ON
I (425) main_task: Returned from app_main()
E(425) OPENTHREAD:[C] P-SpinelDrive-: Software reset co-processor successfully
I(465) OPENTHREAD:[I] CslTxScheduler: Set frame request ahead: 26605 usec
I(475) OPENTHREAD:[I] ChildSupervsn-: Timeout: 0 -> 190
I(485) OPENTHREAD:[I] Settings-----: Read NetworkInfo {rloc:0x3400, extaddr:1229185b3875d707, role:leader,
mode:0x0f, version:5, keyseq:0x0, ...
I(495) OPENTHREAD:[I] Settings-----: ... pid:0x292769f4, mlecntr:0x1ac8d, maccntr:0x15309,
mliid:ddd796acec481884}
I(505) OPENTHREAD:[I] Settings-----: Read ChildInfo {rloc:0x3401, extaddr:262d2ae1b60b2670, timeout:240,
mode:0x04, version:5}
I (505) esp_ot_br: Internal RCP Version: openthread-esp32/fcae32885b-b945928d7; esp32h2; 2025-11-04 11:39:12 UTC
I (515) esp_ot_br: Running RCP Version: openthread-esp32/fcae32885b-b945928d7; esp32h2; 2025-11-04 11:39:12 UTC
I (515) OPENTHREAD: OpenThread attached to netif
> I (515) esp_ot_br: use the Wi-Fi config from NVS
I (515) pp: pp rom version: e7ae62f
I (525) net80211: net80211 rom version: e7ae62f
I (535) wifi:wifi driver task: 3fcbf0d8, prio:23, stack:6144, core=0
I (535) wifi:wifi firmware version: 14da9b7
I (535) wifi:wifi certification version: v7.0
I (535) wifi:config NVS flash: enabled
I (535) wifi:config nano formatting: enabled
I (535) wifi:Init data frame dynamic rx buffer num: 32
I (545) wifi:Init static rx mgmt buffer num: 5
I (545) wifi:Init management short buffer num: 32
I (545) wifi:Init dynamic tx buffer num: 32
I (545) wifi:Init static tx FG buffer num: 2
I (545) wifi:Init static rx buffer size: 1600
I (545) wifi:Init static rx buffer num: 10
I (545) wifi:Init dynamic rx buffer num: 32
I (545) wifi_init: rx ba win: 6
I (545) wifi_init: accept mbox: 6
I (545) wifi_init: tcpip mbox: 32
I (545) wifi_init: udp mbox: 6
I (545) wifi_init: tcp mbox: 6
I (545) wifi_init: tcp tx win: 5760
I (545) wifi_init: tcp rx win: 5760
I (545) wifi_init: tcp mss: 1440
I (545) wifi_init: WiFi IRAM OP enabled
I (555) wifi_init: WiFi RX IRAM OP enabled
I (555) phy_init: phy_version 701,f4f1da3a,Mar 3 2025,15:50:10
I (585) wifi:mode : sta (cc:8d:a2:29:a1:18)
I (585) wifi:enable tsf
I (595) wifi:Set ps type: 2, coexist: 0

I (595) ot_ext_cli: Start example_connect
I (595) example_connect: Connecting to >>NETWORK<<...
W (595) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (595) example_connect: Waiting for IP(s)
> I (3095) wifi:new:<8,2>, old:<1,0>, ap:<255,255>, sta:<8,2>, prof:1, snd_ch_cfg:0x0
I (3095) wifi:state: init -> auth (0xb0)
I (3105) wifi:state: auth -> assoc (0x0)
I (3135) wifi:state: assoc -> run (0x10)
I (3145) wifi:connected with >>NETWORK<<, aid = 5, channel 8, 40D, bssid = c4:bd:12:3f:f1:55
I (3155) wifi:security: WPA2-PSK, phy: bgn, rssi: -52
I (3155) wifi:pm start, type: 2
I (3155) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (3155) wifi:set rx beacon pti, rx_bcn_pti: 0, bcn_timeout: 25000, mt_pti: 0, mt_time: 10000
I (3165) wifi:<ba-add>idx:0 (ifx:0, c4:ad:34:6f:f0:77), tid:0, ssn:3, winSize:64
I (3235) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (4175) obtr_web: Listing files in /spiffs:
I (4175) obtr_web: index.html
I (4175) obtr_web: device.html
I (4175) obtr_web: static/style.css
I (4175) obtr_web: static/restful.js
I (4185) obtr_web: Done listing files.
I (4185) esp_https_server: Starting server
I (4195) esp_https_server: Server listening on port 443
I (4195) obtr_web: <=====HTTPS server start=====>
I (4195) obtr_web: https://10.0.0.84:443/index.html
I (4195) obtr_web: <=====>
I (4195) esp_netif_handlers: example_netif_sta ip: 10.0.0.84, mask: 255.255.255.0, gw: 10.0.0.1
```



```
I (4195) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 10.0.0.84
I (4415) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
fe80:0000:0000:0000:ce8d:a2ff:fe29:a118, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I(4425) OPENTHREAD:[N] RoutingManager: BR ULA prefix: fd60:e7fd:5a3f::/48 (loaded)
I(4425) OPENTHREAD:[N] RoutingManager: Local on-link prefix: fd68:dd05:9863:543b::/64
I (4435) OPENTHREAD: Platform UDP bound to port 53
I (4445) OPENTHREAD: Platform UDP bound to port 49153
I(4445) OPENTHREAD:[N] Mle-----: Role disabled -> detached
I (4455) OT_STATE: netif up
I (4465) OPENTHREAD: NAT64 ready
I (4465) OPENTHREAD: Platform UDP bound to port 61631
I (12155) esp_https_server: performing session handshake
I(31435) OPENTHREAD:[N] Mle-----: RLOC16 3400 -> fffe
I(32155) OPENTHREAD:[N] Mle-----: Attach attempt 1, AnyPartition reattaching with Active Dataset
I(38765) OPENTHREAD:[N] RouterTable---: Allocate router id 13
I(38765) OPENTHREAD:[N] Mle-----: RLOC16 fffe -> 3400
I(38775) OPENTHREAD:[N] Mle-----: Role detached -> leader
I(38775) OPENTHREAD:[N] Mle-----: Partition ID 0xb0f66c
I (38835) OPENTHREAD: Platform UDP bound to port 49154
W(39345) OPENTHREAD:[W] Mle-----: Failed to process UDP: Duplicated
I (50415) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
fd68:dd05:9863:543b:ce8d:a2ff:fe29:a118, type: ESP_IP6_ADDR_IS_UNIQUE_LOCAL
I (132285) esp_https_server: performing session handshake
```



10.8 Kode SED

10.8.1 main.c

```
#include <zephyr/kernel.h>
#include <zephyr/net/openthread.h>
#include <openthread/thread.h>
#include "network.h"
#include "adc.h"
#include "usb_console.h"
#include "udp.h"
#include "alarm_test.h"
#include "fire.h"

int main(void)
{
    otInstance *instance = openthread_get_default_instance();
    configure_thread_network_leader(instance);

    if (battery_init() == 0) {
        printk("Dummy battery ADC ready\n");
    } else {
        printk("Dummy battery ADC init failed!\n");
    }

    alarm_test_init();
    fire_init(instance);
    udp_listener_start(instance);

    return 0;
}
```

10.8.2 network.c

```
#include "network.h"
#include <string.h>
#include <zephyr/kernel.h>
#include <openthread/thread.h>
#include <openthread/dataset.h>
#include <openthread/ip6.h>
#include <openthread/error.h>

void configure_thread_network_leader(otInstance *instance)
{
    otOperationalDataset dataset;
    memset(&dataset, 0, sizeof(dataset));
```



```
// Samme netværksnavn som leader
const char *networkName = "OpenThread-ESP";
memcpy(dataset.mNetworkName.m8, networkName, strlen(networkName));
dataset.mComponents.mIsNetworkNamePresent = true;

// Samme Network Key
const uint8_t networkKey[OT_NETWORK_KEY_SIZE] =
    { 0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
      0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff };
memcpy(dataset.mNetworkKey.m8, networkKey, sizeof(networkKey));
dataset.mComponents.mIsNetworkKeyPresent = true;

// PAN ID
dataset.mPanId = 0x1234;
dataset.mComponents.mIsPanIdPresent = true;

// Channel
dataset.mChannel = 15;
dataset.mComponents.mIsChannelPresent = true;

// Extended PAN ID
const uint8_t extPanId[OT_EXT_PAN_ID_SIZE] =
    { 0x68, 0xdd, 0x05, 0x98, 0x63, 0x26, 0x54, 0x3b };
memcpy(dataset.mExtendedPanId.m8, extPanId, sizeof(extPanId));
dataset.mComponents.mIsExtendedPanIdPresent = true;

// Mesh Local Prefix
otIp6Prefix prefix;
memset(&prefix, 0, sizeof(prefix));
prefix.mPrefix.mFields.m8[0] = 0xfd;
prefix.mPrefix.mFields.m8[1] = 0xc2;
prefix.mPrefix.mFields.m8[2] = 0x26;
prefix.mPrefix.mFields.m8[3] = 0x5f;
prefix.mPrefix.mFields.m8[4] = 0x92;
prefix.mPrefix.mFields.m8[5] = 0x88;
prefix.mPrefix.mFields.m8[6] = 0x9e;
prefix.mPrefix.mFields.m8[7] = 0x0a;
prefix.mLength = 64;
memcpy(&dataset.mMeshLocalPrefix, &prefix, sizeof(prefix));
dataset.mComponents.mIsMeshLocalPrefixPresent = true;

// Active Timestamp (samme struktur som Leader)
dataset.mActiveTimestamp.mSeconds = 1;
dataset.mActiveTimestamp.mTicks = 0;
dataset.mActiveTimestamp.mAuthoritative = 0;
dataset.mComponents.mIsActiveTimestampPresent = true;
```



```
// Commit dataset
otDatasetSetActive(instance, &dataset);

// Konfigurer som Sleep end-device (SED)
otLinkModeConfig mode;
memset(&mode, 0, sizeof(mode));
mode.mRxOnWhenIdle = false; // Sparer strøm
mode.mDeviceType   = false; // false = end device (ikke router)
mode.mNetworkData  = false; // ikke fuld netværksdata
otThreadSetLinkMode(instance, mode);

// Poll interval (hvor tit den spørger parent om trafik) i ms
otLinkSetPollPeriod(instance, 200);

// Enable IPv6 + Thread
otIp6SetEnabled(instance, true);
otThreadSetEnabled(instance, true);
}
```

10.8.3 udp.c

```
#include "udp.h"
#include "adc.h"
#include <zephyr/kernel.h>
#include <zephyr/net/socket.h>
#include <string.h>
#include <errno.h>
#include <stdio.h>
#include "fire.h"
#include "alarm_test.h"
#include <zephyr/sys/printk.h>
#include <openthread/message.h>
#include <openthread/udp.h>
#include <openthread/thread.h>
#include <openthread/link.h>
#include <openthread/ip6.h>

#define LISTEN_PORT 12345
#define RESPONSE_PORT 54321

/* Gem instancen i en global statisk pointer */
static otInstance *g_instance = NULL;

static void udp_server_thread(void)
{
    int sock;
```



```
struct sockaddr_in6 addr6;
char buf[128];

sock = zsock_socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);
if (sock < 0) {
    printk("UDP socket create error: %d\n", errno);
    return;
}

memset(&addr6, 0, sizeof(addr6));
addr6.sin6_family = AF_INET6;
addr6.sin6_port = htons(LISTEN_PORT);
addr6.sin6_addr = in6addr_any;

if (zsock_bind(sock, (struct sockaddr *)&addr6, sizeof(addr6)) < 0) {
    printk("UDP bind error: %d\n", errno);
    zsock_close(sock);
    return;
}

printk("UDP listener started on port %d\n", LISTEN_PORT);

while (1) {
    struct sockaddr_in6 src_addr;
    socklen_t addr_len = sizeof(src_addr);

    int len = zsock_recvfrom(sock, buf, sizeof(buf) - 1, 0,
                            (struct sockaddr *)&src_addr, &addr_len);

    if (len < 0) {
        printk("UDP recv error: %d\n", errno);
        continue;
    }

    buf[len] = '\0';
    printk("UDP received: %s\n", buf);

    // --- ALARM flow ---
    if (strcmp(buf, "ALARMTEST") == 0) {
        alarm_test_trigger();
        continue;
    }

    // --- ALERT_FIRE flow ---
    if (strcmp(buf, "ALARM_START") == 0) {
        if (fire_is_active()) {
            printk("Already active fire alarm - ignoring broadcast\n");
        }
    }
}
```



```
        continue;
    }

    printk("Received ALERT_FIRE broadcast - activating local
siren\n");
    fire_start(); // starter blink, men uden at sende 'fire' besked

    // Send 'ALARM_ONLY' svar tilbage til Border Router
    const char *resp_msg = "ALARM_ONLY";
    src_addr.sin6_port = htons(RESPONSE_PORT); // port 54321
    int ret = zsock_sendto(sock, resp_msg, strlen(resp_msg), 0,
                          (struct sockaddr *)&src_addr, addr_len);

    if (ret < 0)
        printk("UDP send ALARM_ONLY error: %d\n", errno);
    else
        printk("Sent ALARM_ONLY to Border Router\n");

    continue;
}

// --- HUSH flow ---
if (strcmp(buf, "HUSH") == 0) {
    fire_stop(); // Kalder din funktion fra fire.c
    printk("UDP received HUSH - stopping fire alarm\n");

    // send bekræftelse tilbage til Border Router
    const char *resp_msg = "HUSH_ACK";
    src_addr.sin6_port = htons(RESPONSE_PORT); // port 54321
    int ret = zsock_sendto(sock, resp_msg, strlen(resp_msg), 0,
                          (struct sockaddr *)&src_addr, addr_len);
    if (ret < 0) {
        printk("UDP send HUSH_ACK error: %d\n", errno);
    } else {
        printk("Sent HUSH_ACK to Border Router\n");
    }

    continue;
}

// --- BREQ flow ---
if (strstr(buf, "BREQ")) {
    int voltage = get_battery_voltage_mv();
    battery_status_t bat_status = get_battery_status();

    const char *status_str =
```



```

        bat_status == BAT_GREEN ? "GREEN" :
        bat_status == BAT_YELLOW ? "YELLOW" :
        bat_status == BAT_RED ? "RED" :
                                "NO BATTERY";

    char mleIdStr[OT_IP6_ADDRESS_STRING_SIZE] = "<none>";
    char extAddrStr[32] = "<none>";
    uint16_t rloc16 = 0xffff;

    if (g_instance) {
        const otIp6Address *mleid =
otThreadGetMeshLocalEid(g_instance);
        if (mleid) {
            otIp6AddressToString(mleid, mleIdStr, sizeof(mleIdStr));
        }

        rloc16 = otThreadGetRloc16(g_instance);

        otExtAddress extAddr;
        otLinkGetFactoryAssignedIeeeEui64(g_instance, &extAddr);
        snprintf(extAddrStr, sizeof(extAddrStr),
                "%02x:%02x:%02x:%02x:%02x:%02x:%02x:%02x",
                extAddr.m8[0], extAddr.m8[1], extAddr.m8[2],
extAddr.m8[3],
                extAddr.m8[4], extAddr.m8[5], extAddr.m8[6],
extAddr.m8[7]);
    }

    char resp[256];
    snprintf(resp, sizeof(resp),
            "{ \"voltage\": %d, \"status\": \"%s\", \"
            \"mleid\": \"%s\", \"rloc16\": \"%0x%04x\", \"
            \"extaddr\": \"%s\" }",
            voltage, status_str,
            mleIdStr, rloc16,
            extAddrStr);

    src_addr.sin6_port = htons(RESPONSE_PORT);
    int ret = zsock_sendto(sock, resp, strlen(resp), 0,
                            (struct sockaddr *)&src_addr, addr_len);
    if (ret < 0) {
        printk("UDP send response error: %d\n", errno);
    } else {
        printk("Sent response: %s\n", resp);
    }
}
}

```



```
    zsock_close(sock);
}

void udp_listener_start(otInstance *instance)
{
    g_instance = instance; /* gem instancen til brug i tråden */

    static K_THREAD_STACK_DEFINE(stack_area, 2048);
    static struct k_thread thread_data;

    k_thread_create(&thread_data,
                    stack_area, K_THREAD_STACK_SIZEOF(stack_area),
                    (k_thread_entry_t)udp_server_thread,
                    NULL, NULL, NULL,
                    7, 0, K_NO_WAIT);
}
```

10.8.4 fire.c

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/net/socket.h>
#include <zephyr/sys/printk.h>
#include <string.h>
#include <stdbool.h>
#include <errno.h>
#include "fire.h"

// Border Router IPv6-adresse og port
// HUSK at ændre Border Router's adresse
// #define BR_IP6 "fd94:59b5:b7ee:779a:6274:9eaa:9cdd:8073"
#define BR_IP6 "fd28:42ac:246c:1:540d:109a:6034:d9d9"
// #define BR_IP6 "fd78:ee93:e283:a41:ddd7:96ac:ec48:1884"
#define BR_PORT 54321
#define TAG "FIRE"

// --- Hardware konfiguration ---
#define BUTTON_NODE DT_ALIAS(sw0)
#define LED_NODE DT_ALIAS(led0)

static const struct gpio_dt_spec fire_button = GPIO_DT_SPEC_GET(BUTTON_NODE,
gpios);
static const struct gpio_dt_spec fire_led = GPIO_DT_SPEC_GET(LED_NODE,
gpios);
static struct gpio_callback fire_button_cb;

// --- Global state ---
```



```
static otInstance *g_instance = NULL;
static volatile bool fire_active = false;
static struct k_thread fire_thread_data;
K_THREAD_STACK_DEFINE(fire_stack_area, 512);

// --- LED blink ---
void fire_thread(void *a, void *b, void *c)
{
    while (1) {
        if (fire_active) {
            gpio_pin_set_dt(&fire_led, 1);
            k_msleep(300);
            gpio_pin_set_dt(&fire_led, 0);
            k_msleep(300);
        } else {
            k_msleep(100);
        }
    }
}

// --- Start og stop funktioner ---
void fire_start(void)
{
    if (!fire_active) {
        fire_active = true;
        printk("[s] Fire alarm activated\n", TAG);
    }
}

void fire_stop(void)
{
    fire_active = false;
    gpio_pin_set_dt(&fire_led, 0);
    printk("[s] Fire alarm stopped (HUSH received)\n", TAG);
}

// --- Knaphåndtering ---
static void fire_button_pressed(const struct device *dev, struct
gpio_callback *cb, uint32_t pins)
{
    printk("[s] Button pressed -- sending FIRE!\n", TAG);

    // Start blink lokalt
    fire_start();

    // Send FIRE-besked til Border Router
    int sock = zsock_socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
```



```
        printk("[%s] Socket create failed: %d\n", TAG, errno);
        return;
    }

    struct sockaddr_in6 dest = {0};
    dest.sin6_family = AF_INET6;
    dest.sin6_port = htons(BR_PORT);
    inet_pton(AF_INET6, BR_IP6, &dest.sin6_addr);

    const char *msg = "fire";
    int ret = zsock_sendto(sock, msg, strlen(msg), 0,
                          (struct sockaddr *)&dest, sizeof(dest));
    if (ret < 0) {
        printk("[%s] UDP send error: %d\n", TAG, errno);
    } else {
        printk("[%s] UDP sent: %s\n", TAG, msg);
    }

    zsock_close(sock);
}

// --- Init ---
void fire_init(otInstance *instance)
{
    g_instance = instance; // <- MANGLEDE SEMIKOLON HER

    if (!device_is_ready(fire_button.port)) {
        printk("[%s] Button device not ready\n", TAG);
        return;
    }

    if (!device_is_ready(fire_led.port)) {
        printk("[%s] LED device not ready\n", TAG);
        return;
    }

    // Init LED
    gpio_pin_configure_dt(&fire_led, GPIO_OUTPUT_INACTIVE);

    // Init knap
    gpio_pin_configure_dt(&fire_button, GPIO_INPUT);
    gpio_pin_interrupt_configure_dt(&fire_button, GPIO_INT_EDGE_TO_ACTIVE);
    gpio_init_callback(&fire_button_cb, fire_button_pressed,
BIT(fire_button.pin));
    gpio_add_callback(fire_button.port, &fire_button_cb);

    // Start blink-tråd
    k_thread_create(&fire_thread_data, fire_stack_area,
```



```
        K_THREAD_STACK_SIZEOF(fire_stack_area),
        fire_thread, NULL, NULL, NULL,
        5, 0, K_NO_WAIT);

    printk("[%s] Button interrupt configured on pin %d\n", TAG,
fire_button.pin);
}

// --- Aktiv-statusfunktion ---
bool fire_is_active(void)
{
    return fire_active;
}
```

10.8.5 alarm_test.c

```
#include "alarm_test.h"
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/sys/printk.h>

#define LED_NODE DT_ALIAS(led0)
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED_NODE, gpios);

void alarm_test_init(void)
{
    if (!device_is_ready(led.port)) {
        printk("Alarm for test - LED device not ready\n");
        return;
    }
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);
    printk("Alarm test initialized (LED ready)\n");
}

void alarm_test_trigger(void)
{
    printk("Alarm test triggered - LED on!\n");
    gpio_pin_set_dt(&led, 1);

    // Sluk LED efter 10 sekunder
    k_sleep(K_SECONDS(10));

    gpio_pin_set_dt(&led, 0);
    printk("Alarm test ended - LED off\n");
}
```



10.8.6 adc.c

```
#include "adc.h"
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

static int dummy_voltage = 3300;
static battery_status_t dummy_status = BAT_GREEN;

int battery_init(void)
{
    printk("Dummy ADC init på nRF54L15\n");
    return 0;
}

battery_status_t get_battery_status(void)
{
    /* Du kan evt. skifte status dynamisk baseret på dummy_voltage */
    if (dummy_voltage > 3000) {
        dummy_status = BAT_GREEN;
    } else if (dummy_voltage > 2600) {
        dummy_status = BAT_YELLOW;
    } else {
        dummy_status = BAT_RED;
    }

    return dummy_status;
}

int get_battery_voltage_mv(void)
{
    /* Returnér bare dummy værdien */
    return dummy_voltage;
}
```

10.9 Kode Border router

10.9.1 esp_br_web.c

```
#include "esp_br_web.h"
#include "udp_br_func.h"
#include "esp_https_server.h"
#include "esp_check.h"
#include "esp_http_server.h"
#include "esp_log.h"
#include "esp_openthread.h"
#include "esp_openthread_border_router.h"
#include "esp_vfs.h"
#include "sdkconfig.h"
#include "cJSON.h"
#include <limits.h>
```



```
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <inttypes.h>

#define SERVER_IPV4_LEN 16
#define FILE_CHUNK_SIZE 1024
#define WEB_TAG "obtr_web"

/*-----
Note: Authentication
-----*/
#define AUTH_STRING "Basic YWRtaW46dGVzdDEyMw==" // "admin:test123" base64
static const char *AUTH_TAG = "auth";

static esp_err_t check_auth(httpd_req_t *req)
{
    char auth_value[128];
    size_t auth_len = httpd_req_get_hdr_value_len(req, "Authorization") + 1;

    if (auth_len <= 1) {
        httpd_resp_set_hdr(req, "WWW-Authenticate", "Basic realm=\"Secure\"");
        httpd_resp_send_err(req, HTTPD_401_UNAUTHORIZED, "Unauthorized");
        ESP_LOGW(AUTH_TAG, "Access denied: no credentials");
        return ESP_FAIL;
    }

    httpd_req_get_hdr_value_str(req, "Authorization", auth_value, sizeof(auth_value));

    if (strcmp(auth_value, AUTH_STRING) != 0) {
        httpd_resp_set_hdr(req, "WWW-Authenticate", "Basic realm=\"Secure\"");
        httpd_resp_send_err(req, HTTPD_401_UNAUTHORIZED, "Unauthorized");
        ESP_LOGW(AUTH_TAG, "Access denied: wrong credentials");
        return ESP_FAIL;
    }

    ESP_LOGI(AUTH_TAG, "Access granted");
    return ESP_OK;
}

/*-----
Note: Https Server
-----*/
typedef struct https_server_data {
    char base_path[ESP_VFS_PATH_MAX + 1]; /* the storaged file path */
} https_server_data_t;

typedef struct http_server {
    httpd_handle_t handle; /* server handle, unique */
    https_server_data_t data; /* data */
    char ip[SERVER_IPV4_LEN]; /* ip */
    uint16_t port; /* port */
} http_server_t;

static http_server_t s_server = {0};

#define PROTOCOL_MAX_SIZE 12
#define FILENAME_MAX_SIZE 64
#define FILEPATH_MAX_SIZE (FILENAME_MAX_SIZE + ESP_VFS_PATH_MAX)
typedef struct request_url {
    char protocol[PROTOCOL_MAX_SIZE];
    uint16_t port;
    char file_name[FILENAME_MAX_SIZE];
    char file_path[FILEPATH_MAX_SIZE];
} request_url_t;

/*-----
Note: Tidlig deklaration
-----*/
```



```
-----*/
static esp_err_t battery_request_handler(httpd_req_t *req);
static esp_err_t hush_alarm_handler(httpd_req_t *req);
static esp_err_t led_test_handler(httpd_req_t *req);
static esp_err_t battery_status_handler(httpd_req_t *req);
static esp_err_t device_handler(httpd_req_t *req);

static httpd_uri_t s_resource_handlers[] = {
    {
        .uri = "/battery_request",
        .method = HTTP_GET,
        .handler = battery_request_handler,
        .user_ctx = NULL,
    },
    {
        .uri = "/battery_status",
        .method = HTTP_GET,
        .handler = battery_status_handler,
        .user_ctx = NULL,
    },
    {
        .uri = "/api/device",
        .method = HTTP_GET,
        .handler = device_handler,
        .user_ctx = NULL,
    },
    {
        .uri = "/hush_alarm",
        .method = HTTP_GET,
        .handler = hush_alarm_handler,
        .user_ctx = NULL,
    },
    {
        .uri = "/led_test",
        .method = HTTP_GET,
        .handler = led_test_handler,
        .user_ctx = NULL,
    },
};

/*-----
Note: Egne funktioner
-----*/
static esp_err_t battery_request_handler(httpd_req_t *req)
{
    otInstance *instance = esp_openthread_get_instance();
    if (!instance) {
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "No OpenThread instance");
        return ESP_FAIL;
    }

    send_udp_to_all_seds_with_payload(instance, "BREQ", "Battery request");

    httpd_resp_set_type(req, "application/json");
    httpd_resp_sendstr(req, "{\"message\": \"Battery request sent\"}");
    return ESP_OK;
}

static esp_err_t hush_alarm_handler(httpd_req_t *req)
{
    otInstance *instance = esp_openthread_get_instance();
    if (!instance) {
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "No OpenThread instance");
        return ESP_FAIL;
    }

    send_udp_to_all_seds_with_payload(instance, "HUSH", "HUSH request");

    httpd_resp_set_type(req, "application/json");
}
```



```
    httpd_resp_sendstr(req, "{\"message\": \"Hush request sent\"}");
    return ESP_OK;
}

static esp_err_t led_test_handler(httpd_req_t *req)
{
    otInstance *instance = esp_openthread_get_instance();
    if (!instance) {
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "No OpenThread instance");
        return ESP_FAIL;
    }

    send_udp_to_all_seds_with_payload(instance, "ALARMTEST", "LED request");

    httpd_resp_set_type(req, "application/json");
    httpd_resp_sendstr(req, "{\"message\": \"LED ON request sent\"}");
    return ESP_OK;
}

static esp_err_t battery_status_handler(httpd_req_t *req)
{
    char json[1024];
    udp_get_all_status(json, sizeof(json)); // bygger JSON direkte

    httpd_resp_set_type(req, "application/json");
    httpd_resp_sendstr(req, json);
    return ESP_OK;
}

static esp_err_t device_handler(httpd_req_t *req)
{
    char param[32];
    if (httpd_req_get_url_query_str(req, param, sizeof(param)) == ESP_OK) {
        char sedParam[16];
        if (httpd_query_key_value(param, "sed", sedParam, sizeof(sedParam)) == ESP_OK) {
            if (strncmp(sedParam, "SED", 3) == 0) {
                int idx = atoi(sedParam + 3) - 1;

                const sed_status_t *st = udp_get_status(idx);
                if (!st) {
                    httpd_resp_send_err(req, HTTPD_404_NOT_FOUND, "SED not found");
                    return ESP_FAIL;
                }

                sed_status_t sed_copy = *st;
                otInstance *instance = esp_openthread_get_instance();
                if (instance) {
                    update_sed_thread_info(&sed_copy, instance);
                }

                // Konverter last_seen til streng
                char timebuf[32] = "---";
                if (sed_copy.last_seen > 0) {
                    struct tm tm_info;
                    localtime_r(&sed_copy.last_seen, &tm_info);
                    strftime(timebuf, sizeof(timebuf), "%Y-%m-%d %H:%M:%S", &tm_info);
                }

                // Byg JSON med cJSON
                cJSON *root = cJSON_CreateObject();
                cJSON_AddStringToObject(root, "id", sedParam);
                cJSON_AddNumberToObject(root, "voltage", sed_copy.voltage);
                cJSON_AddStringToObject(root, "status", sed_copy.status);

                cJSON_AddStringToObject(root, "alarm", "");
                cJSON_AddStringToObject(root, "last_seen", timebuf);
                cJSON_AddStringToObject(root, "location", "");

                // Thread sub-objekt
                cJSON *thread = cJSON_CreateObject();
```



```

        cJSON_AddStringToObject(thread, "device_type", "SED");
        cJSON_AddStringToObject(thread, "mleid", sed_copy.mleid);

        char rloc16_str[16];
        snprintf(rloc16_str, sizeof(rloc16_str), "0x%04x", sed_copy.rloc16);
        cJSON_AddStringToObject(thread, "rloc16", rloc16_str);

        cJSON_AddStringToObject(thread, "parent_id", "");
        cJSON_AddStringToObject(thread, "partition_id", "");

        cJSON_AddItemToObject(root, "thread", thread);

        // Send svaret
        char *json_str = cJSON_PrintUnformatted(root);
        httpd_resp_set_type(req, "application/json");
        httpd_resp_sendstr(req, json_str);

        // Free hukommelse
        cJSON_Delete(root);
        free(json_str);

        return ESP_OK;
    }
}

httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "Invalid request");
return ESP_FAIL;
}

static esp_err_t favicon_get_handler(httpd_req_t *req)
{
    extern const unsigned char favicon_ico_start[] asm("_binary_favicon_ico_start");
    extern const unsigned char favicon_ico_end[] asm("_binary_favicon_ico_end");
    const size_t favicon_ico_size = (favicon_ico_end - favicon_ico_start);

    ESP_RETURN_ON_ERROR(httpd_resp_set_type(req, "image/x-icon"), WEB_TAG, "Failed to set http respond type");
    ESP_RETURN_ON_ERROR(httpd_resp_send(req, (const char *)favicon_ico_start, favicon_ico_size), WEB_TAG,
        "Failed to send http respond");
    return ESP_OK;
}

static esp_err_t httpd_resp_send_spiffs_file(httpd_req_t *req, char *path)
{
    ESP_LOGI(WEB_TAG, "-----");
    ESP_LOGI(WEB_TAG, "Reading %s", path);

    FILE *fp = fopen(path, "r"); // Open and read file

    ESP_RETURN_ON_FALSE(fp, ESP_FAIL, WEB_TAG, "Failed to open %s file", path);

    char buf[FILE_CHUNK_SIZE]; // the size of chunk
    while (!feof(fp) && !ferror(fp)) {
        fread(buf, FILE_CHUNK_SIZE - 1, 1, fp);
        buf[FILE_CHUNK_SIZE - 1] = '\0';
        httpd_resp_sendstr_chunk(req, buf);
        memset(buf, 0, sizeof(buf));
    };
    return fclose(fp) == 0 ? ESP_OK : ESP_FAIL;
}

static esp_err_t index_html_get_handler(httpd_req_t *req, char *path)
{
    // Tjek først, om brugeren har korrekt login (Basic Auth)
    if (check_auth(req) != ESP_OK) {
        // Hvis auth fejler, returneres ESP_FAIL
    }
}

```



```
// og klienten får automatisk en 401 Unauthorized
return ESP_FAIL;
}

// Hvis auth er godkendt, send HTML-siden
ESP_RETURN_ON_ERROR(httpd_resp_set_type(req, "text/html"), WEB_TAG, "Failed to set http text/html type");
ESP_RETURN_ON_ERROR(httpd_resp_send_spiffs_file(req, path), WEB_TAG, "Failed to send index html file");
ESP_RETURN_ON_ERROR(httpd_resp_sendstr_chunk(req, NULL), WEB_TAG, "Failed to send http string chunk");
return ESP_OK;
}

static esp_err_t style_css_get_handler(httpd_req_t *req, char *path)
{
    // send content-type: "text/css" in http-header
    ESP_RETURN_ON_ERROR(httpd_resp_set_type(req, "text/css"), WEB_TAG, "Failed to set http text/css type");
    ESP_RETURN_ON_ERROR(httpd_resp_send_spiffs_file(req, path), WEB_TAG, "Failed to send css file");
    ESP_RETURN_ON_ERROR(httpd_resp_sendstr_chunk(req, NULL), WEB_TAG, "Failed to send http string chunk");
    return ESP_OK;
}

static esp_err_t script_js_get_handler(httpd_req_t *req, char *path)
{
    // send content-type: "application/javascript" in http-header
    ESP_RETURN_ON_ERROR(httpd_resp_set_type(req, "application/javascript"), WEB_TAG,
        "Failed to set http application/javascript type");
    ESP_RETURN_ON_ERROR(httpd_resp_send_spiffs_file(req, path), WEB_TAG, "Failed to send js file");
    ESP_RETURN_ON_ERROR(httpd_resp_sendstr_chunk(req, NULL), WEB_TAG, "Failed to send http string chunk");
    return ESP_OK;
}

static request_url_t parse_request_url_information(const char *uri, const struct http_parser_url *parse_url,
    const char *base_path)
{
    request_url_t ret = {
        .protocol = "http",
        .port = 80,
        .file_name = "",
        .file_path = "",
    };
    ret.port = parse_url->port;
    if ((parse_url->field_set & (1 << UF_SCHEMA)) != 0 && PROTOCOL_MAX_SIZE >
    parse_url->field_data[UF_SCHEMA].len) {
        memcpy(ret.protocol, uri + parse_url->field_data[UF_SCHEMA].off, parse_url->field_data[UF_SCHEMA].len);
        ret.protocol[parse_url->field_data[UF_SCHEMA].len] = '\0';
    }

    if ((parse_url->field_set & (1 << UF_PATH)) != 0 && FILENAME_MAX_SIZE > parse_url->field_data[UF_PATH].len)
    {
        memcpy(ret.file_name, uri + parse_url->field_data[UF_PATH].off, parse_url->field_data[UF_PATH].len);
        ret.file_name[parse_url->field_data[UF_PATH].len] = '\0';
        memcpy(ret.file_path, base_path, strlen(base_path));
        strcat(ret.file_path, ret.file_name);
    }
    return ret;
}

static esp_err_t default_urls_get_handler(httpd_req_t *req)
{
    struct http_parser_url url;
    ESP_RETURN_ON_ERROR(http_parser_parse_url(req->uri, strlen(req->uri), 0, &url), WEB_TAG, "Failed to parse
url");
    request_url_t info =
        parse_request_url_information(req->uri, &url, ((https_server_data_t *)req->user_ctx)->base_path);

    ESP_LOGI(WEB_TAG, "-----");
    ESP_LOGI(WEB_TAG, "%s", info.file_name);
    if (!strcmp(info.file_name, "")) // check the filename.
    {
        ESP_LOGE(WEB_TAG, "Filename is too long or url error"); /* Respond with 500 Internal Server Error */
    }
}
```



```

        ESP_RETURN_ON_ERROR(
            httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "Filename is too long or url error"),
WEB_TAG,
            "Failed to send error code");
        return ESP_FAIL;
    }
    if (strcmp(info.file_name, "/") == 0) {
        return index_html_get_handler(req, info.file_path);
    } else if (strcmp(info.file_name, "/index.html") == 0) {
        return index_html_get_handler(req, info.file_path);
    } else if (strcmp(info.file_name, "/device.html") == 0) {
        return index_html_get_handler(req, info.file_path);
    } else if (strcmp(info.file_name, "/static/style.css") == 0) {
        return style_css_get_handler(req, info.file_path);
    } else if (strcmp(info.file_name, "/static/restful.js") == 0) {
        return script_js_get_handler(req, info.file_path);
    } else if (strcmp(info.file_name, "/static/bootstrap.min.css") == 0) {
        return script_js_get_handler(req, info.file_path);

    // --- Favicon tilføjet her ---
    } else if (strcmp(info.file_name, "/static/favicon.ico") == 0) {
        ESP_RETURN_ON_ERROR(httpd_resp_set_type(req, "image/x-icon"), WEB_TAG, "Failed to set favicon type");
        ESP_RETURN_ON_ERROR(httpd_resp_send_spiffs_file(req, info.file_path), WEB_TAG, "Failed to send
favicon");
        ESP_RETURN_ON_ERROR(httpd_resp_sendstr_chunk(req, NULL), WEB_TAG, "Failed to send favicon chunk");
        return ESP_OK;

    // --- Hvis ingen af ovenstående matcher ---
    } else {
        ESP_LOGW(WEB_TAG, "File not found: %s", info.file_path);
        httpd_resp_send_err(req, HTTPD_404_NOT_FOUND, "File not found");
        return ESP_FAIL;
    }
}

/*-----
Note: Server Start
-----*/
/* =====
HTTPS Certificate + Key (Embedded from CMakeLists.txt)
===== */

extern const unsigned char servercert_pem_start[] asm("_binary_servercert_pem_start");
extern const unsigned char servercert_pem_end[]   asm("_binary_servercert_pem_end");
extern const unsigned char prvtkkey_pem_start[]   asm("_binary_prvtkey_pem_start");
extern const unsigned char prvtkkey_pem_end[]     asm("_binary_prvtkey_pem_end");

#define SERVER_CERT      servercert_pem_start
#define SERVER_CERT_LEN  (servercert_pem_end - servercert_pem_start)
#define SERVER_KEY       prvtkkey_pem_start
#define SERVER_KEY_LEN   (prvtkey_pem_end - prvtkkey_pem_start)

static void list_spiffs_files(const char *base_path)
{
    ESP_LOGI(WEB_TAG, "Listing files in %s:", base_path);

    DIR *dir = opendir(base_path);
    if (!dir) {
        ESP_LOGE(WEB_TAG, "Failed to open directory %s", base_path);
        return;
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {

```



```

        ESP_LOGI(WEB_TAG, " %s", entry->d_name);
    }

    closedir(dir);
    ESP_LOGI(WEB_TAG, "Done listing files.");
}

static httpd_handle_t *start_esp_br_https_server(const char *base_path, const char *host_ip)
{
    ESP_RETURN_ON_FALSE(base_path, NULL, WEB_TAG, "Invalid https server path");
    strcpy(s_server.ip, host_ip);
    strncpy(s_server.data.base_path, base_path, ESP_VFS_PATH_MAX + 1);
    list_spiffs_files(base_path);

    httpd_ssl_config_t conf = HTTPD_SSL_CONFIG_DEFAULT();
    conf.port_secure = 443;
    conf.servercert      = SERVER_CERT;
    conf.servercert_len  = SERVER_CERT_LEN;
    conf.prvtkey_pem     = SERVER_KEY;
    conf.prvtkey_len     = SERVER_KEY_LEN;

    // Allow wildcard URI matching (so "/" works)
    conf.httpd.uri_match_fn = httpd_uri_match_wildcard;

    // Start HTTPS-server
    esp_err_t ret = httpd_ssl_start(&s_server.handle, &conf);
    if (ret != ESP_OK) {
        ESP_LOGE(WEB_TAG, "Failed to start HTTPS server: %s", esp_err_to_name(ret));
        return NULL;
    }

    // Registrér URI-handlers
    httpd_uri_t default_uris_get = {
        .uri = "/*",
        .method = HTTP_GET,
        .handler = default_urls_get_handler,
        .user_ctx = &s_server.data
    };

    for (int i = 0; i < sizeof(s_resource_handlers) / sizeof(httpd_uri_t); i++) {
        httpd_register_uri_handler(s_server.handle, &s_resource_handlers[i]);
    }
    httpd_register_uri_handler(s_server.handle, &default_uris_get);

    // Info i log
    ESP_LOGI(WEB_TAG, "%s\r\n", "<=====HTTPS server start=====>");
    ESP_LOGI(WEB_TAG, "https://%s:%d/index.html\r\n", s_server.ip, 443);
    ESP_LOGI(WEB_TAG, "%s\r\n", "<=====>");

    return s_server.handle;
}

void connect_handler(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data, const char
*base_path)
{
    httpd_handle_t *server = (httpd_handle_t *)arg;
    ESP_RETURN_ON_FALSE(server, , WEB_TAG, "Http server is invalid, failed to start it");
    ESP_LOGI(WEB_TAG, "Start the web server for Openthread Border Router");
    *server = (httpd_handle_t *)start_esp_br_https_server(base_path, s_server.ip);
}

static bool is_br_web_server_started = false;
static void handler_got_ip_event(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
{
    if (!is_br_web_server_started) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
        char ipv4_address[SERVER_IPV4_LEN];
    }
}

```



```
    sprintf((char *)ipv4_address, IPSTR, IP2STR(&event->ip_info.ip));
    if (start_esp_br_https_server((const char *)arg, (char *)ipv4_address) != NULL) {
        is_br_web_server_started = true;
    } else {
        ESP_LOGE(WEB_TAG, "Fail to start web server");
    }
} else {
    ESP_LOGW(WEB_TAG, "Web server had already been started");
}
}
}

void esp_br_web_start(char *base_path)
{
    ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &handler_got_ip_event,
base_path));
    ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_ETH_GOT_IP, &handler_got_ip_event,
base_path));
}
```

10.9.2 udp_br_func.c

```
#include "udp_br_func.h"
#include "esp_log.h"
#include "esp_openthread.h"
#include "lwip/sockets.h"

#include "lwip/inet.h"
#include <openthread/thread.h>
#include <openthread/thread_ftd.h>
#include <openthread/ip6.h>
#include "cJSON.h"
#include <time.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>

#define UDP_PORT_SEND    12345    // hvor vi sender BREQ
#define UDP_PAYLOAD      "BREQ"

#define UDP_PORT_LISTEN  54321    // samme port som devices svarer på

#define TAG "UDP_BR"
#define MAX_SEDS 10
static sed_status_t sed_status[MAX_SEDS]; // global status for SEDs
void update_sed_thread_info(sed_status_t *sed, otInstance *instance);
static uint16_t find_sed_by_address(struct sockaddr_in6 *source_addr);

// -----
// Hjælp: find ledig eller eksisterende SED-slot ud fra RLOC16
// -----
static int find_sed_slot(uint16_t rloc16)
```



```
{
    for (int i = 0; i < MAX_SEDS; i++) {
        if (sed_status[i].valid && sed_status[i].rloc16 == rloc16) {
            return i; // eksisterende
        }
    }
    for (int i = 0; i < MAX_SEDS; i++) {
        if (!sed_status[i].valid) {
            sed_status[i].rloc16 = rloc16;
            sed_status[i].valid = true;
            return i; // nyt slot
        }
    }
    return -1; // ingen plads
}

// -----
// Send UDP til alle børn (kun SEDs)
// -----
void send_udp_to_all_seds_with_payload(otInstance *instance, const char
*payload, const char *log_tag)
{
    if (!instance) {
        ESP_LOGE(TAG, "No OpenThread instance available");
        return;
    }

    if (!payload) {
        ESP_LOGE(TAG, "No payload provided");
        return;
    }

    int sock_tx = socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);
    if (sock_tx < 0) {
        ESP_LOGE(TAG, "TX socket create failed (errno=%d)", errno);
        return;
    }

    uint16_t max_children = otThreadGetMaxAllowedChildren(instance);

    for (uint16_t i = 0; i < max_children; i++) {
        otChildInfo child;
        if (otThreadGetChildInfoByIndex(instance, i, &child) !=
OT_ERROR_NONE) {
            continue;
        }

        if (!child.mFullThreadDevice) { // Kun SED
```



```
        otChildIp6AddressIterator iter =
OT_CHILD_IP6_ADDRESS_ITERATOR_INIT;
        otIp6Address ip;
        otError err;

        do {
            err = otThreadGetChildNextIp6Address(instance, i, &iter,
&ip);

            if (err == OT_ERROR_NONE) {
                struct sockaddr_in6 dest = {0};
                dest.sin6_family = AF_INET6;
                dest.sin6_port = htons(UDP_PORT_SEND);
                memcpy(&dest.sin6_addr, &ip, sizeof(ip));

                int ret = sendto(sock_tx, payload, strlen(payload), 0,
                                (struct sockaddr *)&dest,
sizeof(dest));

                if (ret < 0) {
                    ESP_LOGE(TAG, "UDP send error (errno=%d)", errno);
                } else {
                    ESP_LOGI(TAG, "%s sent to child RLOC16=0x%04x",
                                log_tag ? log_tag : "Message",
child.mRloc16);
                }
            }
        } while (err == OT_ERROR_NONE);
    }

    close(sock_tx);
}

// -----
// UDP listener task
// -----
static void udp_listener_task(void *arg)
{
    int sock = socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        ESP_LOGE(TAG, "Listener socket create failed: errno=%d", errno);
        vTaskDelete(NULL);
        return;
    }

    struct sockaddr_in6 addr = {0};
    addr.sin6_family = AF_INET6;
    addr.sin6_port = htons(UDP_PORT_LISTEN);
    addr.sin6_addr = in6addr_any;
```



```

if (bind(sock, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    ESP_LOGE(TAG, "Bind failed: errno=%d", errno);
    close(sock);
    vTaskDelete(NULL);
    return;
}

ESP_LOGI(TAG, "UDP listener started on port %d", UDP_PORT_LISTEN);

while (1) {
    char rx_buffer[256];
    struct sockaddr_in6 source_addr;
    socklen_t socklen = sizeof(source_addr);

    int len = recvfrom(sock, rx_buffer, sizeof(rx_buffer) - 1, 0,
                      (struct sockaddr *)&source_addr, &socklen);

    if (len > 0) {
        rx_buffer[len] = 0; // nul-terminer
        ESP_LOGI(TAG, "UDP received: %s", rx_buffer);

        // Check if it's a command message (non-JSON)
        if (strncmp(rx_buffer, "fire", 4) == 0) {
            ESP_LOGI(TAG, "Fire command received from SED!");

            // Find which SED sent this message by comparing source
            address
            uint16_t sender_rloc16 = find_sed_by_address(&source_addr);

            if (sender_rloc16 != 0) {
                // Update only the SED that sent the fire message
                for (int i = 0; i < MAX_SEDS; i++) {
                    if (sed_status[i].valid && sed_status[i].rloc16 ==
sender_rloc16) {
                        sed_status[i].fire_detected = true;
                        strncpy(sed_status[i].status, "FIRE",
sizeof(sed_status[i].status) - 1);
                        ESP_LOGI(TAG, "SED with RLOC16 0x%04x marked as FIRE
DETECTED", sender_rloc16);
                        break;
                    }
                }
                otInstance *instance = esp_openthread_get_instance();
                if (instance) {
                    send_udp_to_all_seds_with_payload(instance,
"ALARM_START", "Fire alarm broadcast");
                    ESP_LOGI(TAG, "Broadcasted FALARM to all SEDs");

```



```
    }
  } else {
    ESP_LOGW(TAG, "Could not identify which SED sent fire
message");
  }

  continue; // Skip JSON parsing for command messages
}

else if (strncmp(rx_buffer, "HUSH_ACK", 8) == 0) {
  ESP_LOGI(TAG, "=== HUSH_ACK RECEIVED ===");
  ESP_LOGI(TAG, "Raw message: %s", rx_buffer);

  // Find which SED sent this message by comparing source address
  uint16_t sender_rloc16 = find_sed_by_address(&source_addr);

  if (sender_rloc16 != 0) {
    ESP_LOGI(TAG, "HUSH_ACK from SED with RLOC16: 0x%04x",
sender_rloc16);

    // Find the SED index by RLOC16 and update its fire state
    for (int i = 0; i < MAX_SEDS; i++) {
      if (sed_status[i].valid && sed_status[i].rloc16 ==
sender_rloc16) {
        ESP_LOGI(TAG, "Found matching SED at index %d", i);

        // Update fire state for this SED
        bool old_state = sed_status[i].fire_detected;
        sed_status[i].fire_detected = false;
        strncpy(sed_status[i].status, "NORMAL",
sizeof(sed_status[i].status) - 1);

        ESP_LOGI(TAG, "SUCCESS: SED%d (RLOC16 0x%04x) fire
state updated: %d -> %d",
                  i+1, sender_rloc16, old_state, false);
        break;
      }
    }
  } else {
    ESP_LOGW(TAG, "Could not identify which SED sent HUSH_ACK");
  }

  ESP_LOGI(TAG, "=== HUSH_ACK PROCESSING COMPLETE ===");
  continue;
}

else if (strncmp(rx_buffer, "ALARM_ONLY", 10) == 0) {
  ESP_LOGI(TAG, "Alarm only (no fire) command received from
```



```
SED!");

    // Find which SED sent this message by comparing source address
    uint16_t sender_rloc16 = find_sed_by_address(&source_addr);

    if (sender_rloc16 != 0) {
        // Update only the SED that sent the alarm_only message
        for (int i = 0; i < MAX_SEDS; i++) {
            if (sed_status[i].valid && sed_status[i].rloc16 ==
sender_rloc16) {
                sed_status[i].fire_detected = false; // Not actual
fire
                strncpy(sed_status[i].status, "ALARM",
sizeof(sed_status[i].status) - 1);
                ESP_LOGI(TAG, "SED with RLOC16 0x%04x marked as
ALARM ONLY", sender_rloc16);
                break;
            }
        }
    } else {
        ESP_LOGW(TAG, "Could not identify which SED sent ALARM_ONLY
message");
    }

    continue; // Skip JSON parsing for command messages
}

// Parse JSON med cJSON (existing battery status functionality)
cJSON *root = cJSON_Parse(rx_buffer);
if (root) {
    int voltage = cJSON_GetObjectItem(root,
"voltage")->valueint;
    const char *status = cJSON_GetObjectItem(root,
"status")->valuestring;
    const char *mleid = cJSON_GetObjectItem(root,
"mleid")->valuestring;
    const char *rloc16s = cJSON_GetObjectItem(root,
"rloc16")->valuestring;
    const char *extaddr = cJSON_GetObjectItem(root,
"extaddr")->valuestring;

    uint16_t rloc16 = (uint16_t)strtol(rloc16s, NULL, 0);

    int idx = find_sed_slot(rloc16);
    if (idx >= 0) {
        sed_status[idx].voltage = voltage;
        strncpy(sed_status[idx].status, status,
sizeof(sed_status[idx].status) - 1);
    }
}
```



```
        strncpy(sed_status[idx].mleid, mleid,
                sizeof(sed_status[idx].mleid) - 1);
        strncpy(sed_status[idx].extaddr, extaddr,
                sizeof(sed_status[idx].extaddr) - 1);
        sed_status[idx].rloc16 = rloc16;

        // Sæt last_seen til nuværende tid
        sed_status[idx].last_seen = time(NULL);

        ESP_LOGI(TAG,
                 "Updated SED%d: %d mV, %s, MLEID=%s,
RLOC16=0x%04x, ExtAddr=%s, LastSeen=%ld",
                 idx + 1,
                 voltage,
                 sed_status[idx].status,
                 sed_status[idx].mleid,
                 sed_status[idx].rloc16,
                 sed_status[idx].extaddr,
                 sed_status[idx].last_seen);
    }

    cJSON_Delete(root);
} else {
    ESP_LOGW(TAG, "Invalid JSON from SED: %s", rx_buffer);
}
}

close(sock);
vTaskDelete(NULL);
}

// -----
// Start listener fra main/init
// -----
void udp_listener_start(otInstance *instance)
{
    (void)instance; // ikke brugt lige nu
    xTaskCreate(udp_listener_task, "udp_listener", 4096, NULL, 5, NULL);
}

// -----
// Getter til web: returnér pointer + antal
// -----
const char *udp_get_all_status(char *buf, size_t bufsize)
{

```



```
cJSON *root = cJSON_CreateObject();

for (int i = 0; i < MAX_SEDS; i++) {
    if (sed_status[i].rloc16 != 0) {
        cJSON *entry = cJSON_CreateObject();
        cJSON_AddNumberToObject(entry, "voltage",
sed_status[i].voltage);
        cJSON_AddStringToObject(entry, "status", sed_status[i].status);
        cJSON_AddStringToObject(entry, "mleid", sed_status[i].mleid);
        cJSON_AddStringToObject(entry, "extaddr",
sed_status[i].extaddr);

        // ADD THIS: Include fire detection status
        cJSON_AddBoolToObject(entry, "fire_detected",
sed_status[i].fire_detected);

        char rloc16_str[16];
        snprintf(rloc16_str, sizeof(rloc16_str), "0x%04x",
sed_status[i].rloc16);
        cJSON_AddStringToObject(entry, "rloc16", rloc16_str);

        cJSON_AddNumberToObject(entry, "last_seen",
(long)sed_status[i].last_seen);

        char sed_id[8];
        snprintf(sed_id, sizeof(sed_id), "SED%d", i + 1);
        cJSON_AddItemToObject(root, sed_id, entry);
    }
}

if (!cJSON_PrintPreallocated(root, buf, bufsize, 0)) {
    ESP_LOGW("UDP_BR", "Buffer too small for JSON status");
    buf[0] = '\0';
}

cJSON_Delete(root);
return buf;
}

const sed_status_t *udp_get_status(int idx)
{
    if (idx < 0 || idx >= MAX_SEDS || !sed_status[idx].valid) {
        return NULL;
    }
    return &sed_status[idx];
}
```



```
void update_sed_thread_info(sed_status_t *sed, otInstance *instance)
{
    otChildInfo childInfo;
    uint16_t maxChildren = otThreadGetMaxAllowedChildren(instance);

    for (uint16_t i = 0; i < maxChildren; i++) {
        if (otThreadGetChildInfoByIndex(instance, i, &childInfo) ==
            OT_ERROR_NONE) {
            if (childInfo.mRloc16 == sed->rloc16) {

                otChildIp6AddressIterator iter =
                    OT_CHILD_IP6_ADDRESS_ITERATOR_INIT;
                otIp6Address ip;
                bool found = false;

                while (otThreadGetChildNextIp6Address(instance, i, &iter,
                    &ip) == OT_ERROR_NONE) {
                    // Mesh-local EID starter altid med "fd"
                    if ((ip.mFields.m8[0] & 0xFE) == 0xFC) {
                        otIp6AddressToString(&ip, sed->mleid,
                            sizeof(sed->mleid));
                        ESP_LOGI("SED", "Found MLEID for child 0x%04x ->
                            %s",
                                sed->rloc16, sed->mleid);
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    strncpy(sed->mleid, "-", sizeof(sed->mleid));
                    sed->mleid[sizeof(sed->mleid) - 1] = '\0';
                    ESP_LOGW("SED", "No MLEID found for child 0x%04x",
                        sed->rloc16);
                }

                return; // færdig for dette barn
            }
        }
    }

    // Hvis vi ikke fandt barnet overhovedet
    strncpy(sed->mleid, "-", sizeof(sed->mleid));
    sed->mleid[sizeof(sed->mleid) - 1] = '\0';
    ESP_LOGW("SED", "Child 0x%04x not found in child table", sed->rloc16);
}
```



```
// Helper function to find SED RLOC16 from source address
static uint16_t find_sed_by_address(struct sockaddr_in6 *source_addr)
{
    otInstance *instance = esp_openthread_get_instance();
    if (!instance) {
        return 0;
    }

    // Convert source address to otIp6Address
    otIp6Address src_ip;
    memcpy(&src_ip, &source_addr->sin6_addr, sizeof(otIp6Address));

    // Search through all children to find which one has this IP
    uint16_t max_children = otThreadGetMaxAllowedChildren(instance);

    for (uint16_t i = 0; i < max_children; i++) {
        otChildInfo child;
        if (otThreadGetChildInfoByIndex(instance, i, &child) !=
OT_ERROR_NONE) {
            continue;
        }

        // Check if this child has the source IP
        otChildIp6AddressIterator iter = OT_CHILD_IP6_ADDRESS_ITERATOR_INIT;
        otIp6Address child_ip;

        while (otThreadGetChildNextIp6Address(instance, i, &iter, &child_ip)
== OT_ERROR_NONE) {
            if (memcmp(&child_ip, &src_ip, sizeof(otIp6Address)) == 0) {
                ESP_LOGI(TAG, "Found matching SED: RLOC16=0x%04x",
child.mRloc16);
                return child.mRloc16;
            }
        }
    }

    ESP_LOGW(TAG, "No SED found with the source IP address");
    return 0;
}

void update_sed_fire_state(const char *sed_name, bool fire_state) {
    ESP_LOGI(TAG, "=== UPDATE_SED_FIRE_STATE CALLED ===");
    ESP_LOGI(TAG, "Input: sed_name='%s', fire_state=%d", sed_name,
fire_state);

    // Extract SED number from name (e.g., "SED1" -> 1, "SED2" -> 2)
    int sed_number;
```



```
if (sscanf(sed_name, "SED%d", &sed_number) == 1) {
    ESP_LOGI(TAG, "Parsed SED number: %d", sed_number);

    // Convert to array index (SED1 -> index 0, SED2 -> index 1, etc.)
    int idx = sed_number - 1;
    ESP_LOGI(TAG, "Target index: %d", idx);

    if (idx >= 0 && idx < MAX_SEDS) {
        if (sed_status[idx].valid) {
            bool old_state = sed_status[idx].fire_detected;
            sed_status[idx].fire_detected = fire_state;

            // Update status text based on fire state
            if (fire_state) {
                strncpy(sed_status[idx].status, "FIRE",
sizeof(sed_status[idx].status) - 1);
            } else {
                strncpy(sed_status[idx].status, "NORMAL",
sizeof(sed_status[idx].status) - 1);
            }

            ESP_LOGI(TAG, "SUCCESS: SED%d fire state updated: %d -> %d",
sed_number, old_state, fire_state);
            ESP_LOGI(TAG, "SED%d status: %s", sed_number,
sed_status[idx].status);
        } else {
            ESP_LOGW(TAG, "SED index %d exists but is not valid", idx);
        }
    } else {
        ESP_LOGW(TAG, "Invalid SED index %d for %s (must be 0-%d)",
idx, sed_name, MAX_SEDS-1);
    }
} else {
    ESP_LOGW(TAG, "Could not parse SED name: '%s'", sed_name);
    ESP_LOGW(TAG, "Expected format: 'SED1', 'SED2', etc.");
}
ESP_LOGI(TAG, "=== UPDATE_SED_FIRE_STATE COMPLETE ===");
}
```

10.9.3 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Thread Network Monitoring Dashboard</title>
    <link rel="stylesheet" href="static/style.css">
```



```
<script>
  let alarmSilenced = false;
  let knownSEDs = new Set(); // Track which SEDs we've seen
  let waitingForHushConfirmation = false; // Track if we're waiting for
SED responses
  let sedFireStates = {}; // Track fire state for each SED individually
  let sedAlarmStates = {}; // Track alarm-only state for each SED

  async function getBatteryAll() {
    // Send BREQ ud til alle SEDs
    await fetch("/battery_request");

    // Vent lidt så SED kan nå at svare
    await new Promise(r => setTimeout(r, 300));

    // Hent seneste status og opdater cirkler
    const response = await fetch("/battery_status");
    const data = await response.json();
    updateCircles(data);
  }

  function updateCircles(data) {
    let fireDetected = false;
    let alarmDetected = false;
    let fireSEDs = [];
    let alarmSEDs = [];

    for (const [sed, info] of Object.entries(data)) {
      // Create circle if it doesn't exist
      if (!knownSEDs.has(sed)) {
        createSEDCircle(sed);
        knownSEDs.add(sed);
      }

      const circle = document.getElementById(sed + "-circle");
      const text = document.getElementById(sed + "-text");

      if (!circle || !text) continue;

      // Update our local fire state tracking
      if (info.fire_detected !== undefined) {
        sedFireStates[sed] = info.fire_detected;
      }

      // Check if this SED is in alarm state (status contains "ALARM")
      sedAlarmStates[sed] = info.status && info.status.includes("ALARM");

      // Update display based on status - priority: FIRE > ALARM > NORMAL
    }
  }
}
```



```
if (sedFireStates[sed]) {
  // This SED detected fire - show dark red with fire icon
  text.innerHTML = `🔥 FIRE<br>${info.voltage} mV`;
  circle.style.backgroundColor = "darkred";
  circle.style.color = "white";
  circle.classList.add('alarm-blinking');
  circle.style.animationName = 'fire-blink';
  fireDetected = true;
  fireSEDs.push(sed);
} else if (sedAlarmStates[sed]) {
  // This SED is in alarm-only state - show orange with alarm icon
  text.innerHTML = `🚨 ALARM<br>${info.voltage} mV`;
  circle.style.backgroundColor = "#ff9800";
  circle.style.color = "white";
  circle.classList.add('alarm-blinking');
  circle.style.animationName = 'alarm-blink-orange';
  alarmDetected = true;
  alarmSEDs.push(sed);
} else {
  // Normal status - your existing code
  text.innerHTML = `${info.voltage} mV<br>${info.status}`;
  circle.classList.remove('alarm-blinking');

  if (info.voltage >= 2100) {
    circle.style.backgroundColor = "green";
    circle.style.color = "white";
  } else if (info.voltage >= 1900) {
    circle.style.backgroundColor = "gold";
    circle.style.color = "black";
  } else {
    circle.style.backgroundColor = "red";
    circle.style.color = "white";
  }
}
}

// Check if all SEDs have confirmed hush
if (waitingForHushConfirmation) {
  const allFiresStopped = !Object.values(sedFireStates).some(state =>
state);
  const allAlarmsStopped = !Object.values(sedAlarmStates).some(state
=> state);

  if (allFiresStopped && allAlarmsStopped) {
    // All SEDs have confirmed hush
    waitingForHushConfirmation = false;
    alarmSilenced = false;
    updateAlarmBox(false, [], []);
  }
}
```



```
        updateHushButton(false);
    } else {
        // Still waiting for some SEDs to confirm
        updateAlarmBox(true, fireSEDs, alarmSEDs);
        updateHushButton(true);
    }
} else {
    // Normal operation
    if (!alarmSilenced) {
        updateAlarmBox(fireDetected || alarmDetected, fireSEDs,
alarmSEDs);
    }
    updateHushButton(fireDetected || alarmDetected);
}
}

function createSEDCircle(sedName) {
    const sedContainer = document.querySelector('.sed-container');

    // Create the SED div structure
    const sedDiv = document.createElement('div');
    sedDiv.className = 'sed';

    // Create SED name label
    const nameLabel = document.createElement('div');
    nameLabel.className = 'sed-name';
    nameLabel.textContent = sedName;

    const link = document.createElement('a');
    link.href = `/device.html?sed=${sedName}`;

    const circle = document.createElement('div');
    circle.id = `${sedName}-circle`;
    circle.className = 'circle';

    const text = document.createElement('span');
    text.id = `${sedName}-text`;
    text.innerHTML = '---';

    // Assemble the structure
    circle.appendChild(text);
    link.appendChild(circle);
    sedDiv.appendChild(nameLabel); // Add name above circle
    sedDiv.appendChild(link);
    sedContainer.appendChild(sedDiv);

    // Initialize states for this SED
    sedFireStates[sedName] = false;
```



```
    sedAlarmStates[sedName] = false;
  }

  async function hushAlarm() {
    // Only proceed if button is not disabled
    const hushButton = document.querySelector('.btn.danger');
    if (hushButton.disabled) return;

    // Set waiting state - don't update alarm status until SEDs confirm
    waitingForHushConfirmation = true;
    alarmSilenced = false; // Reset silenced state until we get
    confirmations

    // Update button immediately to show we're waiting
    updateHushButton(true); // Pass true to show we're in waiting state
    but still have fire

    // Send hush command to all SEDs
    await fetch("/hush_alarm");

    // Note: We'll wait for HUSH_ACK responses to update individual SED
    states
  }

  async function LEDtest() {
    await fetch("/led_test");
  }

  function updateAlarmBox(anyAlarmDetected, fireSEDs, alarmSEDs) {
    const alarmBox = document.getElementById("global-alarm");

    if (waitingForHushConfirmation && anyAlarmDetected) {
      // Show waiting status while SEDs are confirming
      let alarmText = "Hushing...";
      if (fireSEDs.length > 0) {
        alarmText += ` FIRE ${fireSEDs.join(', ')} `;
      }
      if (alarmSEDs.length > 0) {
        if (fireSEDs.length > 0) alarmText += " & ";
        alarmText += ` ALARM ${alarmSEDs.join(', ')} `;
      }
      alarmBox.textContent = alarmText;
      alarmBox.className = "alarm-box alarm-on";
    } else if (anyAlarmDetected) {
      // Normal alarm state
      let alarmText = "Alarm status: ";
      if (fireSEDs.length > 0) {
        alarmText += ` FIRE ${fireSEDs.join(', ')} `;
      }
    }
  }
}
```



```
    }
    if (alarmSEDs.length > 0) {
      if (fireSEDs.length > 0) alarmText += " & ";
      alarmText += `ALARM ${alarmSEDs.join(', ')}`;
    }
    alarmBox.textContent = alarmText;
    alarmBox.className = "alarm-box alarm-on";
  } else {
    // All clear
    alarmBox.textContent = "Alarm status: GOOD";
    alarmBox.className = "alarm-box alarm-good";
  }
}

function updateHushButton(anyAlarmDetected) {
  const hushButton = document.querySelector('.btn.danger');

  if (waitingForHushConfirmation) {
    // We're waiting for confirmation - show "Hushing..." but keep
    enabled
    hushButton.disabled = true; // Disable during hush process
    hushButton.classList.add('disabled');
    hushButton.style.cursor = 'not-allowed';
    hushButton.style.opacity = '0.6';
    hushButton.textContent = 'Hushing...';
  } else if (anyAlarmDetected) {
    // Enable the button - any alarm detected (fire or alarm-only)
    hushButton.disabled = false;
    hushButton.classList.remove('disabled');
    hushButton.style.cursor = 'pointer';
    hushButton.style.opacity = '1';
    hushButton.textContent = 'Hush Alarm';
  } else {
    // Disable the button - no alarms detected
    hushButton.disabled = true;
    hushButton.classList.add('disabled');
    hushButton.style.cursor = 'not-allowed';
    hushButton.style.opacity = '0.6';
    hushButton.textContent = 'Hush Alarm';
  }
}

// Når siden loader første gang, hent seneste kendte status
window.addEventListener("DOMContentLoaded", async () => {
  // Første load - no initial circles will be created
  const response = await fetch("/battery_status");
  const data = await response.json();
  updateCircles(data);
});
```



```
// Initialize Hush button based on initial state
updateHushButton(false);

setInterval(async () => {
  const resp = await fetch("/battery_status");
  const data = await resp.json();
  updateCircles(data);
}, 2000); // Reduced to 2 seconds for faster response to HUSH_ACK
});
</script>
</head>
<body>
  <div class="dashboard">
    <h1>Thread Network Monitoring Dashboard</h1>

    <div class="button-container">
      <button class="btn primary" onclick="getBatteryAll()">Battery
Request</button>
      <button class="btn warning" onclick="LEDtest()">Alarm Test</button>
      <button class="btn danger" onclick="hushAlarm()">Hush Alarm</button>
    </div>

    <!-- Empty container - circles will be added dynamically -->
    <div class="sed-container">
      <!-- SED circles will appear here automatically when data is received
-->
    </div>

    <div id="global-alarm" class="alarm-box alarm-good">Alarm status:
GOOD</div>
  </div>
</body>
</html>
```

10.9.4 device.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Device Details</title>
  <link rel="stylesheet" href="static/style.css">
  <script>
    async function loadDevice() {
      const urlParams = new URLSearchParams(window.location.search);
      const sedId = urlParams.get('sed');
```



```
const response = await fetch("/api/device?sed=" + sedId);
const data = await response.json();

document.getElementById("dev-id").textContent = data.id;
document.getElementById("voltage").textContent = data.voltage + " mV";
document.getElementById("status").textContent = data.status;
document.getElementById("alarm").textContent = data.alarm;
document.getElementById("last_seen").textContent = data.last_seen;
document.getElementById("location").textContent = data.location;

document.getElementById("device_type").textContent =
data.thread.device_type;
document.getElementById("mleid").textContent = data.thread.mleid;
document.getElementById("rloc16").textContent = data.thread.rloc16;
document.getElementById("parent_id").textContent =
data.thread.parent_id;
document.getElementById("partition_id").textContent =
data.thread.partition_id;
}
</script>
</head>
<body onload="loadDevice()">
  <div class="dashboard">
    <h1>Details for <span id="dev-id">---</span></h1>

    <div class="info-block">
      <h2>General Info</h2>
      <table>
        <tr><th>Voltage</th><td id="voltage">---</td></tr>
        <tr><th>Status</th><td id="status">---</td></tr>
        <tr><th>Alarm</th><td id="alarm">---</td></tr>
        <tr><th>Last Seen</th><td id="last_seen">---</td></tr>
        <tr><th>Location</th><td id="location">---</td></tr>
      </table>
    </div>

    <div class="info-block">
      <h2>Thread Network Info</h2>
      <table>
        <tr><th>Device Type</th><td id="device_type">---</td></tr>
        <tr><th>MLEID</th><td id="mleid">---</td></tr>
        <tr><th>RLOC16</th><td id="rloc16">---</td></tr>
        <tr><th>Parent</th><td id="parent_id">---</td></tr>
        <tr><th>Partition ID</th><td id="partition_id">---</td></tr>
      </table>
    </div>

    <a href="/index.html" class="btn primary">← Back to Dashboard</a>
```



```
</div>
</body>
</html>
```

10.9.5 style.css

```
/* ===== General Page Layout ===== */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background: linear-gradient(135deg, #e0f7fa, #e1bee7);
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* ===== Dashboard Container ===== */
.dashboard {
  background: white;
  padding: 30px 50px;
  border-radius: 15px;
  box-shadow: 0 8px 20px rgba(0,0,0,0.2);
  text-align: center;
  width: 85%;
  max-width: 1100px;
}

h1 {
  margin-bottom: 20px;
}

/* ===== Buttons ===== */
.button-container {
  margin-bottom: 30px;
}

.btn {
  padding: 14px 28px;
  margin: 0 12px;
  border: none;
  border-radius: 10px;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
  transition: 0.3s;
```



```
    text-decoration: none;
    display: inline-block;
}

.btn.primary {
    background-color: #007BFF;
    color: white;
}

.btn.primary:hover {
    background-color: #0056b3;
}

.btn.danger {
    background-color: #dc3545;
    color: white;
}

.btn.danger:hover {
    background-color: #a71d2a;
}

.btn.warning {
    background-color: #ffc107;
    color: #212529;
    border: 1px solid #ffc107;
}

.btn.warning:hover {
    background-color: #e0a800;
    border-color: #e0a800;
}

/* ===== Disabled Button Styles ===== */
.btn.disabled {
    background-color: #6c757d !important;
    border-color: #6c757d !important;
    color: white !important;
    cursor: not-allowed !important;
    opacity: 0.6 !important;
}

.btn.danger.disabled {
    background-color: #6c757d !important;
    border-color: #6c757d !important;
    color: white !important;
    cursor: not-allowed !important;
    opacity: 0.6 !important;
}
```



```
}

/* ===== SED Names ===== */
.sed-name {
  font-weight: bold;
  font-size: 16px;
  margin-bottom: 8px;
  color: #333;
  text-align: center;
}

/* ===== SED Circles ===== */
.sed-container {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 50px;
  margin-top: 30px;
  margin-bottom: 30px;
}

.sed {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.circle {
  width: 140px;
  height: 140px;
  border-radius: 50%;
  display: flex;
  justify-content: center;
  align-items: center;
  text-align: center;
  padding: 10px;
  font-weight: bold;
  font-size: 14px;
  background-color: lightgray;
  color: black;
  border: 3px solid black;
  cursor: pointer;
  transition: transform 0.2s;
}

.circle:hover {
  transform: scale(1.1);
}
```



```
a {
  text-decoration: none;
  color: inherit;
}

/* ===== Alarm Box ===== */
.alarm-box {
  font-size: 20px;
  font-weight: bold;
  padding: 14px 28px;
  border: 2px solid black;
  display: inline-block;
  border-radius: 10px;
}

.alarm-good {
  background-color: green;
  color: white;
}

.alarm-on {
  background-color: red;
  color: white;
  animation: blink 1s infinite;
}

@keyframes blink {
  0% { background-color: red; color: white; }
  50% { background-color: gray; color: rgb(255, 255, 255); }
  100% { background-color: red; color: white; }
}

.alarm-blinking {
  animation: alarm-blink 0.8s infinite;
  border: 2px solid #fff;
}

/* Different alarm states */
.alarm-fire {
  background-color: darkred !important;
  border: 2px solid #fff;
}

.alarm-only {
  background-color: #ff9800 !important; /* Orange color */
  border: 2px solid #fff;
}
```



```
/* Alarm blinking animations */
@keyframes alarm-blink {
  0%, 100% {
    box-shadow: 0 0 25px currentColor;
  }
  50% {
    box-shadow: 0 0 10px currentColor;
  }
}

/* Specific animations for different alarm types */
@keyframes fire-blink {
  0%, 100% {
    background-color: darkred;
    box-shadow: 0 0 25px red;
  }
  50% {
    background-color: #800;
    box-shadow: 0 0 10px red;
  }
}

@keyframes alarm-blink-orange {
  0%, 100% {
    background-color: #ff9800; /* Bright orange */
    box-shadow: 0 0 25px #ffb74d;
  }
  50% {
    background-color: #f57c00; /* Darker orange */
    box-shadow: 0 0 10px #ffb74d;
  }
}

/* ===== Device Info Page ===== */
.info-block {
  margin: 25px auto;
  padding: 20px;
  border-radius: 12px;
  background: #f9f9f9;
  width: 70%;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  text-align: left;
}

.info-block h2 {
  margin-bottom: 15px;
  text-align: center;
}
```



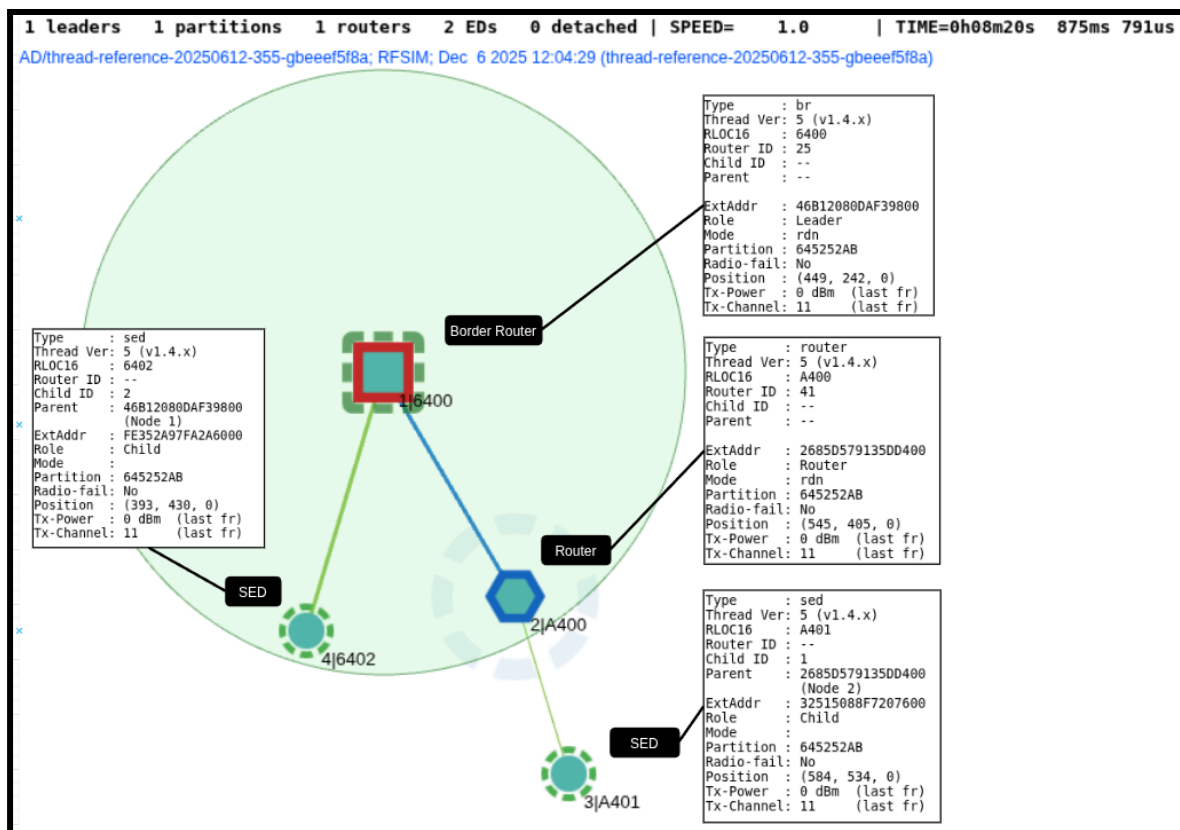
```
}  
  
table {  
  width: 100%;  
  border-collapse: collapse;  
}  
  
th, td {  
  padding: 10px;  
  text-align: left;  
}  
  
th {  
  width: 40%;  
  color: #333;  
}  
  
td {  
  color: #555;  
  background: #fff;  
  border-bottom: 1px solid #ddd;  
}
```

10.10 OTNS og Wireshark

Skal lægges ind

OTNS

OTNS (OpenThread Network Simulator) er et simuleringsværktøj udviklet til at visualisere og teste Thread-netværk uden brug af fysisk hardware. OTNS fungerer som en virtuel sandkasse, hvor udviklere kan oprette et komplet mesh-netværk bestående af både FTD- og MTD-noder og undersøge netværkets adfærd i realtid. Formålet er at give et kontrolleret miljø, hvor netværket kan analyseres, optimeres og fejlrettes, før det implementeres på faktiske enheder.



En central styrke ved OTNS er det visuelle overblik: alle noder vises i et interaktivt web-interface, hvor man kan observere netværkstopologien, ruter, link-kvalitet og ændringer i rollen mellem noder. Samtidig tildeles hver virtuel node sin egen CLI-instans, hvilket gør det muligt at køre OpenThread-kommandoer, aktivere datasæt, justere indstillinger eller simulere fejl direkte i simulationen.

OTNS giver også mulighed for at simulere radioforhold, f.eks. rækkevidde, interferens og forsinkelse, hvilket gør det lettere at evaluere netværkets robusthed. Ved at kunne slukke, flytte eller manipulere noder i simulationen kan man observere, hvordan mesh-protokollen automatisk genopbygger forbindelserne – et centralt aspekt i Thread-standardens selvhelende design.

I projektet anvendes OTNS som et udviklings- og analyseværktøj til at forstå, teste og optimere Thread-kommunikationen, før systemet implementeres på nRF-baserede enheder



og ESP32-S3/H2 Border Router-opsætningen. Dette reducerer kompleksiteten i udviklingsprocessen og minimerer fejl, da potentielle problemer kan identificeres tidligt. OTNS bidrager dermed til en mere effektiv udviklingscyklus og skaber et solidt grundlag for opbygningen af det endelige IoT-netværk.

Installering af OTNS <https://github.com/openthread/ot-ns/blob/main/GUIDE.md>

Kræver go 1.23 eller derover

Opsætning af OTNS til Wireshark

For at analysere kommunikationen i Thread-mesh-netværket uden fysisk hardware blev der anvendt en ren softwarebaseret simuleringspipeline bestående af OpenThread Network Simulator (OTNS) og Wireshark. Denne opsætning muliggør inspektion af simuleret radio- og netværkstrafik via PCAP-filer genereret direkte af simulatoren.

Projektet indeholder udelukkende Go-kildekode og kræver derfor en manuel kompilering af selve simulatoren. Den kørbare fil blev genereret via:

```
cd ~/otns  
go build -o otns ./cmd/otns
```

Den anvendte OTNS-version understøtter ikke ekstern logdirectory-konfiguration, men genererer i stedet PCAP direkte via flaget `-pcap`. Simulatoren startes fra roden af projektmappen:

```
cd ~/otns  
./otns -pcap wpan
```

Flaget `-pcap wpan` aktiverer logging af simulerede IEEE 802.15.4- og Thread-rammer. OTNS genererer automatisk filen: `/home/usr/otns/current.pcap`

PCAP-filen opdateres løbende under simulationen og overskrives ved hver ny session. Under simuleringen blev Thread-noder oprettet direkte via OTNS CLI: CMDS

Når simulatoren kører, registreres al trafik mellem disse virtuelle noder i den aktive PCAP-fil uden behov for yderligere konfiguration.

Wireshark blev installeret via Ubuntus officielle pakkerepositorier og åbnes direkte mod den af OTNS genererede PCAP-fil: `wireshark ~/otns/current.pcap`

Herfra kan den simulerede trafik analyseres på både MAC- og netværkslag. Relevante filtre i Wireshark omfatter bl.a.:

- `wpan` (IEEE 802.15.4 rammer)
- `mle` (Mesh Link Establishment)
- `icmpv6` (ping trafik)
- `coap` (applikationslag for Thread)

Denne metode gør det muligt at dokumentere routingadfærd, hop-struktur og pakkeflow i et Thread-mesh-netværk uden fysisk hardware.

Opsætningen med OTNS og Wireshark giver en fuldstændig softwarebaseret netværkstestplatform. Den muliggør realistisk simulering af Thread-mesh-kommunikation og generering af PCAP-filer til detaljeret analyse i Wireshark. Metoden kræver ingen fysisk radiohardware og er derfor velegnet som forundersøgelse til senere implementering på faktiske ESP32-S3 og H2 enheder.

Fuldt Wireshark capture af OTNS simulering:

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help				
Apply a display filter ... <Ctrl-/>				
No.	Source	Destination	Protocol	Length Info
1			IEEE 802.15.4	62 Reserved
2	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
3	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
4	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
5	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
6	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
7	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
8	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
9	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
10	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
11	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
12	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
13	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
14	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
15	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
16	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
17	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
18	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
19	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
20	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
21	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
22	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
23	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
24	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
25	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
26	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
27	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
28	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
29	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
30	fe80::88fa:b885:8431:1ee2	ff02::2	MLE	63
31	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
32	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
33	fe80::c03e:b01f:e2f:cd a0	ff02::2	MLE	63
34	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
35	fe80::280c:5d75:293c:24f3	ff02::1	MLE	75
36	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
37	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
38	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
39	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
40	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
41	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
42	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
43	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
44	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	75
45	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	75
46	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
47	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
48	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
49	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
50	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
51	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
52	fe80::280c:5d75:293c:24f3	ff02::1	MLE	69
53	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
54	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
55	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
56	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast

57	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
58	fe80::88fa:b885:8431:1ee2	fe80::f4ab:38b3:b00c:4a4a	MLE	113
59			IEEE 802.15.4	5 Ack
60	fe80::280c:5d75:293c:24f3	fe80::f4ab:38b3:b00c:4a4a	MLE	113
61			IEEE 802.15.4	5 Ack
62	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
63	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
64	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
65	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
66	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	91
67	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
68	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
69	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	69
70	fe80::280c:5d75:293c:24f3	ff02::2	MLE	63
71	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
72	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
73	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
74	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
75	fe80::88fa:b885:8431:1ee2	fe80::280c:5d75:293c:24f3	MLE	113
76			IEEE 802.15.4	5 Ack
77	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
78	9e:d8:26:ce:3a:23:08:2f	Broadcast	IEEE 802.15.4	78 Data, Src: 9e:d8:26:ce:3a:23:08:2f, Dst: Broadcast
79	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
80	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
81	fe80::88fa:b885:8431:1ee2	fe80::f4ab:38b3:b00c:4a4a	MLE	113
82			IEEE 802.15.4	5 Ack
83	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
84	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	118
85	fe80::c03e:b01f:e2f:cd:a0	ff02::2	MLE	63
86	fe80::280c:5d75:293c:24f3	fe80::88fa:b885:8431:1ee2	MLE	98
87			IEEE 802.15.4	5 Ack
88	8a:fa:b8:85:84:31:1e:e2	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	126 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: 2a:0c:5d:75:29:3c:24:f3
89			IEEE 802.15.4	5 Ack
90	8a:fa:b8:85:84:31:1e:e2	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	65 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: 2a:0c:5d:75:29:3c:24:f3
91			IEEE 802.15.4	5 Ack
92	fe80::88fa:b885:8431:1ee2	fe80::9cd8:26ce:3a23:82f	MLE	117
93			IEEE 802.15.4	5 Ack
94	fe80::f4ab:38b3:b00c:4a4a	ff02::2	MLE	63
95	fe80::9cd8:26ce:3a23:82f	ff02::2	MLE	63
96	fe80::c03e:b01f:e2f:cd:a0	ff02::2	MLE	63
97	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	69
98	fe80::88fa:b885:8431:1ee2	fe80::f4ab:38b3:b00c:4a4a	MLE	113
99			IEEE 802.15.4	5 Ack
100	fe80::88fa:b885:8431:1ee2	fe80::9cd8:26ce:3a23:82f	MLE	117
101			IEEE 802.15.4	5 Ack
102	fe80::280c:5d75:293c:24f3	fe80::f4ab:38b3:b00c:4a4a	MLE	113
103			IEEE 802.15.4	5 Ack
104	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	121 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
105	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	65 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
106	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	117
107			IEEE 802.15.4	5 Ack
108	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	121 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
109	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	86 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
110	fe80::f4ab:38b3:b00c:4a4a	fe80::88fa:b885:8431:1ee2	MLE	98
111			IEEE 802.15.4	5 Ack
112	8a:fa:b8:85:84:31:1e:e2	f6:ab:38:b3:b0:0c:4a:4a	IEEE 802.15.4	126 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: f6:ab:38:b3:b0:0c:4a:4a
113			IEEE 802.15.4	5 Ack
114	8a:fa:b8:85:84:31:1e:e2	f6:ab:38:b3:b0:0c:4a:4a	IEEE 802.15.4	108 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: f6:ab:38:b3:b0:0c:4a:4a
115			IEEE 802.15.4	5 Ack
116	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	121 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
117	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	96 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
118	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	102
119			IEEE 802.15.4	5 Ack
120	0x0401	0x0400	IEEE 802.15.4	57 Data, Src: 0x0401, Dst: 0x0400
121			IEEE 802.15.4	5 Ack
122	0x0400	0x0401	IEEE 802.15.4	53 Data, Src: 0x0400, Dst: 0x0401
123			IEEE 802.15.4	5 Ack
124	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
125	fe80::c03e:b01f:e2f:cd:a0	ff02::2	MLE	63
126	9e:d8:26:ce:3a:23:08:2f	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	34 Data Request
127			IEEE 802.15.4	5 Ack
128	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	126 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
129			IEEE 802.15.4	5 Ack
130	9e:d8:26:ce:3a:23:08:2f	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	34 Data Request
131			IEEE 802.15.4	5 Ack
132	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	126 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
133			IEEE 802.15.4	5 Ack
134	9e:d8:26:ce:3a:23:08:2f	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	34 Data Request
135			IEEE 802.15.4	5 Ack
136	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	116 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
137			IEEE 802.15.4	5 Ack
138	9e:d8:26:ce:3a:23:08:2f	2a:0c:5d:75:29:3c:24:f3	IEEE 802.15.4	34 Data Request
139			IEEE 802.15.4	5 Ack
140	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
141	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
142	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95
143			IEEE 802.15.4	5 Ack
144	0x0001	0x0000	IEEE 802.15.4	22 Data Request
145			IEEE 802.15.4	5 Ack
146	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	126 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
147			IEEE 802.15.4	5 Ack
148	0x0001	0x0000	IEEE 802.15.4	22 Data Request
149			IEEE 802.15.4	5 Ack
150	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	105 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
151			IEEE 802.15.4	5 Ack
152	0x0001	0x0000	IEEE 802.15.4	22 Data Request
153			IEEE 802.15.4	5 Ack
154	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89
155			IEEE 802.15.4	5 Ack
156	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	62
157			IEEE 802.15.4	5 Ack
158	fe80::f4ab:38b3:b00c:4a4a	fe80::c03e:b01f:e2f:cd:a0	MLE	117
159			IEEE 802.15.4	5 Ack
160	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
161	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
162	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
163	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
164	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78 Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast

165	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
166	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
167	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
168	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
169	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
170	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
171	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
172	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
173	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
174	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
175	c2:3e:b0:1f:0e:2f:cd:a0	Broadcast	IEEE 802.15.4	78	Data, Src: c2:3e:b0:1f:0e:2f:cd:a0, Dst: Broadcast
176	fe80::c03e:b01f:e2f:cda0	fe80::f4ab:38b3:b00c:4a4a	MLE	102	
177			IEEE 802.15.4	5	Ack
178	0x0402	0x0400	IEEE 802.15.4	57	Data, Src: 0x0402, Dst: 0x0400
179			IEEE 802.15.4	5	Ack
180	0x0400	0x0402	IEEE 802.15.4	53	Data, Src: 0x0400, Dst: 0x0402
181			IEEE 802.15.4	5	Ack
182	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
183	c2:3e:b0:1f:0e:2f:cd:a0	f6:ab:38:b3:b0:0c:4a:4a	IEEE 802.15.4	34	Data Request
184			IEEE 802.15.4	5	Ack
185	f6:ab:38:b3:b0:0c:4a:4a	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	126	Data, Src: f6:ab:38:b3:b0:0c:4a:4a, Dst: c2:3e:b0:1f:0e:2f:cd:a0
186			IEEE 802.15.4	5	Ack
187	c2:3e:b0:1f:0e:2f:cd:a0	f6:ab:38:b3:b0:0c:4a:4a	IEEE 802.15.4	34	Data Request
188			IEEE 802.15.4	5	Ack
189	f6:ab:38:b3:b0:0c:4a:4a	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	126	Data, Src: f6:ab:38:b3:b0:0c:4a:4a, Dst: c2:3e:b0:1f:0e:2f:cd:a0
190			IEEE 802.15.4	5	Ack
191	c2:3e:b0:1f:0e:2f:cd:a0	f6:ab:38:b3:b0:0c:4a:4a	IEEE 802.15.4	34	Data Request
192			IEEE 802.15.4	5	Ack
193	f6:ab:38:b3:b0:0c:4a:4a	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	116	Data, Src: f6:ab:38:b3:b0:0c:4a:4a, Dst: c2:3e:b0:1f:0e:2f:cd:a0
194			IEEE 802.15.4	5	Ack
195	fe80::c03e:b01f:e2f:cda0	fe80::f4ab:38b3:b00c:4a4a	MLE	95	
196			IEEE 802.15.4	5	Ack
197	0x3001	0x3000	IEEE 802.15.4	22	Data Request
198			IEEE 802.15.4	5	Ack
199	fe80::f4ab:38b3:b00c:4a4a	fe80::c03e:b01f:e2f:cda0	MLE	89	
200			IEEE 802.15.4	5	Ack
201	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
202	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
203	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
204	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
205	fe80::f4ab:38b3:b00c:4a4a	fe80::280c:5d75:293c:24f3	MLE	79	
206			IEEE 802.15.4	5	Ack
207	fe80::280c:5d75:293c:24f3	fe80::f4ab:38b3:b00c:4a4a	MLE	104	
208			IEEE 802.15.4	5	Ack
209	fe80::f4ab:38b3:b00c:4a4a	fe80::280c:5d75:293c:24f3	MLE	91	
210			IEEE 802.15.4	5	Ack
211	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
212	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
213	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
214	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
215	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	121	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
216	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	96	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
217	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	121	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
218	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	96	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
219	f6:ab:38:b3:b0:0c:4a:4a	Broadcast	IEEE 802.15.4	121	Data, Src: f6:ab:38:b3:b0:0c:4a:4a, Dst: Broadcast
220	f6:ab:38:b3:b0:0c:4a:4a	Broadcast	IEEE 802.15.4	96	Data, Src: f6:ab:38:b3:b0:0c:4a:4a, Dst: Broadcast
221	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
222	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
223	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
224	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
225	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
226	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
227	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
228	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
229	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
230	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
231	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
232	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
233	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
234	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
235	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
236	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
237	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
238	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
239	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
240	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
241	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
242	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
243	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
244	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
245	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
246	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
247	fe80::f4ab:38b3:b00c:4a4a	ff02::1	MLE	71	
248	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
249	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
250	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
251	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
252	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
253	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
254	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
255	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
256	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
257	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
258	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
259	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
260	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71	
261	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71	
262	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95	
263			IEEE 802.15.4	5	Ack
264	0x0001	0x0000	IEEE 802.15.4	22	Data Request
265			IEEE 802.15.4	5	Ack
266	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	126	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
267			IEEE 802.15.4	5	Ack
268	0x0001	0x0000	IEEE 802.15.4	22	Data Request
269			IEEE 802.15.4	5	Ack
270	2a:0c:5d:75:29:3c:24:f3	9e:d8:26:ce:3a:23:08:2f	IEEE 802.15.4	105	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: 9e:d8:26:ce:3a:23:08:2f
271			IEEE 802.15.4	5	Ack
272	0x0001	0x0000	IEEE 802.15.4	22	Data Request

273			IEEE 802.15.4	5 Ack
274	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89
275			IEEE 802.15.4	5 Ack
276	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	62
277			IEEE 802.15.4	5 Ack
278	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95
279			IEEE 802.15.4	5 Ack
280	0x0001	0x0000	IEEE 802.15.4	22 Data Request
281			IEEE 802.15.4	5 Ack
282	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89
283			IEEE 802.15.4	5 Ack
284	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
285	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
286	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
287	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
288	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
289	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
290	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
291	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
292	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
293	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
294	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
295	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
296	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
297	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
298	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
299	fe80::c03e:b01f:e2f:cd0	fe80::f4ab:38b3:b00c:4a4a	MLE	95
300	0x3001	0x3000	IEEE 802.15.4	22 Data Request
301	0x3001	0x3000	IEEE 802.15.4	22 Data Request
302	0x3001	0x3000	IEEE 802.15.4	22 Data Request
303	0x3001	0x3000	IEEE 802.15.4	22 Data Request
304	0x3001	0x3000	IEEE 802.15.4	22 Data Request
305	0x3001	0x3000	IEEE 802.15.4	22 Data Request
306	0x3001	0x3000	IEEE 802.15.4	22 Data Request
307	0x3001	0x3000	IEEE 802.15.4	22 Data Request
308	0x3001	0x3000	IEEE 802.15.4	22 Data Request
309	0x3001	0x3000	IEEE 802.15.4	22 Data Request
310	0x3001	0x3000	IEEE 802.15.4	22 Data Request
311	0x3001	0x3000	IEEE 802.15.4	22 Data Request
312	0x3001	0x3000	IEEE 802.15.4	22 Data Request
313	0x3001	0x3000	IEEE 802.15.4	22 Data Request
314	0x3001	0x3000	IEEE 802.15.4	22 Data Request
315	0x3001	0x3000	IEEE 802.15.4	22 Data Request
316	0x3001	0x3000	IEEE 802.15.4	22 Data Request
317	0x3001	0x3000	IEEE 802.15.4	22 Data Request
318	0x3001	0x3000	IEEE 802.15.4	22 Data Request
319	0x3001	0x3000	IEEE 802.15.4	22 Data Request
320	0x3001	0x3000	IEEE 802.15.4	22 Data Request
321	0x3001	0x3000	IEEE 802.15.4	22 Data Request
322	0x3001	0x3000	IEEE 802.15.4	22 Data Request
323	0x3001	0x3000	IEEE 802.15.4	22 Data Request
324	0x3001	0x3000	IEEE 802.15.4	22 Data Request
325	0x3001	0x3000	IEEE 802.15.4	22 Data Request
326	0x3001	0x3000	IEEE 802.15.4	22 Data Request
326	0x3001	0x3000	IEEE 802.15.4	22 Data Request
327	0x3001	0x3000	IEEE 802.15.4	22 Data Request
328	0x3001	0x3000	IEEE 802.15.4	22 Data Request
329	0x3001	0x3000	IEEE 802.15.4	22 Data Request
330	0x3001	0x3000	IEEE 802.15.4	22 Data Request
331	0x3001	0x3000	IEEE 802.15.4	22 Data Request
332	0x3001	0x3000	IEEE 802.15.4	22 Data Request
333	0x3001	0x3000	IEEE 802.15.4	22 Data Request
334	0x3001	0x3000	IEEE 802.15.4	22 Data Request
335	0x3001	0x3000	IEEE 802.15.4	22 Data Request
336	0x3001	0x3000	IEEE 802.15.4	22 Data Request
337	0x3001	0x3000	IEEE 802.15.4	22 Data Request
338	0x3001	0x3000	IEEE 802.15.4	22 Data Request
339	0x3001	0x3000	IEEE 802.15.4	22 Data Request
340	0x3001	0x3000	IEEE 802.15.4	22 Data Request
341	0x3001	0x3000	IEEE 802.15.4	22 Data Request
342	0x3001	0x3000	IEEE 802.15.4	22 Data Request
343	0x3001	0x3000	IEEE 802.15.4	22 Data Request
344	0x3001	0x3000	IEEE 802.15.4	22 Data Request
345	0x3001	0x3000	IEEE 802.15.4	22 Data Request
346	0x3001	0x3000	IEEE 802.15.4	22 Data Request
347	0x3001	0x3000	IEEE 802.15.4	22 Data Request
348	fe80::c03e:b01f:e2f:cd0	ff02::2	MLE	63
349	fe80::280c:5d75:293c:24f3	fe80::c03e:b01f:e2f:cd0	MLE	117
350			IEEE 802.15.4	5 Ack
351	fe80::88fa:b885:8431:1ee2	fe80::c03e:b01f:e2f:cd0	MLE	117
352			IEEE 802.15.4	5 Ack
353	fe80::c03e:b01f:e2f:cd0	fe80::88fa:b885:8431:1ee2	MLE	121
354			IEEE 802.15.4	5 Ack
355	c2:3e:b0:1f:0e:2f:cd:a0	8a:fa:b8:85:84:31:1e:e2	IEEE 802.15.4	34 Data Request
356			IEEE 802.15.4	5 Ack
357	8a:fa:b8:85:84:31:1e:e2	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	126 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: c2:3e:b0:1f:0e:2f:cd:a0
358			IEEE 802.15.4	5 Ack
359	c2:3e:b0:1f:0e:2f:cd:a0	8a:fa:b8:85:84:31:1e:e2	IEEE 802.15.4	34 Data Request
360			IEEE 802.15.4	5 Ack
361	8a:fa:b8:85:84:31:1e:e2	c2:3e:b0:1f:0e:2f:cd:a0	IEEE 802.15.4	121 Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: c2:3e:b0:1f:0e:2f:cd:a0
362			IEEE 802.15.4	5 Ack
363	0x0403	0x0400	IEEE 802.15.4	34 Data, Src: 0x0403, Dst: 0x0400
364			IEEE 802.15.4	5 Ack
365	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
366	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
367	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
368	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
369	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
370	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
371	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
372	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
373	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
374	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
375	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
376	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
377	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
378	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
379	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000
380	0x0400	0x3000	IEEE 802.15.4	33 Data, Src: 0x0400, Dst: 0x3000

Side 125 af 138

488	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
489	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
490	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
491	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
492	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
493	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
494	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
495	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
496	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
497	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
498	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
499	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
500	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
501	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
502	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
503	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
504	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
505	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
506	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
507	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
508	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
509	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
510	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
511	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
512	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
513	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	71
514	fe80::280c:5d75:293c:24f3	ff02::1	MLE	71
515	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95
516				
517	0x0001	0x0000	IEEE 802.15.4	5 Ack
518			IEEE 802.15.4	22 Data Request
519	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89
520			IEEE 802.15.4	5 Ack
521	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
522	fe80::c03e:b01f:e2f:cda0	fe80::88fa:b885:8431:1ee2	MLE	95
523			IEEE 802.15.4	5 Ack
524	0x0403	0x0400	IEEE 802.15.4	22 Data Request
525			IEEE 802.15.4	5 Ack
526	fe80::88fa:b885:8431:1ee2	fe80::c03e:b01f:e2f:cda0	MLE	89
527			IEEE 802.15.4	5 Ack
528	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
529	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
530	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
531	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
532	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
533	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
534	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
535	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
536	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
537	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
538	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
539	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
540	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
541	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
542	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
543	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
544	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
545	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
546	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
547	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
548	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
549	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
550	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
551	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
552	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
553	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
554	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
555	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
556	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
557	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
558	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
559	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
560	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
561	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
562	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
563	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95
564			IEEE 802.15.4	5 Ack
565	0x0001	0x0000	IEEE 802.15.4	22 Data Request
566			IEEE 802.15.4	5 Ack
567	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89
568			IEEE 802.15.4	5 Ack
569	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
570	fe80::c03e:b01f:e2f:cda0	fe80::88fa:b885:8431:1ee2	MLE	95
571			IEEE 802.15.4	5 Ack
572	0x0403	0x0400	IEEE 802.15.4	22 Data Request
573			IEEE 802.15.4	5 Ack
574	fe80::88fa:b885:8431:1ee2	fe80::c03e:b01f:e2f:cda0	MLE	89
575			IEEE 802.15.4	5 Ack
576	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
577	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
578	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
579	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
580	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
581	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
582	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
583	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
584	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
585	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
586	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
587	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
588	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
589	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
590	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
591	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78 Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
592	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70
593	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70
594	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70

594	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
595	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
596	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
597	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
598	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
599	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
600	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
601	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
602	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
603	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
604	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
605	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
606	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
607	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
608	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
609	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
610	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
611	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
612	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
613	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
614	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
615	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
616	fe80::9cd8:26ce:3a23:82f	fe80::280c:5d75:293c:24f3	MLE	95	
617			IEEE 802.15.4	5	Ack
618	0x0001	0x0000	IEEE 802.15.4	22	Data Request
619			IEEE 802.15.4	5	Ack
620	fe80::280c:5d75:293c:24f3	fe80::9cd8:26ce:3a23:82f	MLE	89	
621			IEEE 802.15.4	5	Ack
622	fe80::c03e:b01f:e2f:cda0	fe80::88fa:b885:8431:1ee2	MLE	95	
623			IEEE 802.15.4	5	Ack
624	0x0403	0x0400	IEEE 802.15.4	22	Data Request
625			IEEE 802.15.4	5	Ack
626	fe80::88fa:b885:8431:1ee2	fe80::c03e:b01f:e2f:cda0	MLE	89	
627			IEEE 802.15.4	5	Ack
628	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
629	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
630	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
631	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
632	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
633	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
634	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
635	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
636	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
637	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
638	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
639	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
640	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
641	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast
642	2a:0c:5d:75:29:3c:24:f3	Broadcast	IEEE 802.15.4	78	Data, Src: 2a:0c:5d:75:29:3c:24:f3, Dst: Broadcast
643	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
644	fe80::88fa:b885:8431:1ee2	ff02::1	MLE	70	
645	fe80::280c:5d75:293c:24f3	ff02::1	MLE	70	
646	8a:fa:b8:85:84:31:1e:e2	Broadcast	IEEE 802.15.4	78	Data, Src: 8a:fa:b8:85:84:31:1e:e2, Dst: Broadcast

10.11 Bellman-Ford Algoritmen og table routing

Algoritmen findes her: <https://www.geeksforgeeks.org/dsa/bellman-ford-algorithm-dp-23/>

I OpenThread stacken findes den tillempede AF logik her:

esp-idf/components/openthread/openthread/src/core/thread/router_table.cpp

 router_table.cpp

```
{
    uint8_t curCost = router->GetCost() + GetLinkCost(*nextHop);
    uint8_t newCost = cost + linkCostToNeighbor;

    if (newCost < curCost)
    {
        router->SetNextHopAndCost(aNeighborId, cost);
        SignalTableChanged();
    }
}
```

10.12 Opsætning af sniffer og indstilling af dekryptering:

En Thread-, Bluetooth- og Wi-Fi-sniffer er et værktøj, der opfanger og analyserer datapakker fra disse specifikke trådløse kommunikationsprotokoller, hvilket gør det muligt for udviklere og netværksadministratorer at fejlfinde, overvåge og analysere netværk. Den fungerer ved at opfange trafikken næsten i realtid og vise den — ofte i et større netværksanalyseværktøj som Wireshark — for at give indsigt i pakkeaktivitet, sikkerhedsindstillinger og dataflow. Ifølge Nordic Semiconductor kan disse værktøjer være dedikeret hardware, udviklingskits eller software, der kører på kompatibel hardware som f.eks. en nRF52840-dongle eller en nRF52840DK.

Sådan fungerer de:

- **Pakkeopsamling:** En sniffer konfigureres til at lytte på trådløs trafik på bestemte kanaler.
- **Datafangst:** Den opfanger de rå datapakker, mens de sendes mellem enheder
- **Pakkeanalyse:** De opsamlede data bliver derefter dekoderet og vist i et læsbart format, ofte med oplysninger om protokol, kilde- og destinationsadresse samt datapayload.
- **Dekryptering:** For krypterede netværk skal snifferen have de korrekte nøgler for at kunne dekryptere pakkerne og vise deres indhold, som det ofte er tilfældet i moderne Thread- og Bluetooth-netværk.
- **Visning:** De dekodede pakker vises typisk i et værktøj som Wireshark, der kan bruges til at analysere trafikken og fejlfinde problemer.

nRF Sniffer-softwaren til 802.15.4 sender kommandoer til nRF-sniffer-hardwaren gennem den serielle port og læser de indfangede frames. Softwaren kan installeres som et eksternt capture-plugin i Wireshark. Man behøver kun at installere plugin'et, hvis man planlægger at bruge nRF Sniffer som en Wireshark-capture-grænseflade.

Sådan installerer man nRF Sniffer capture-plugin'et:

Installer pySerial-modulet:

Åbn et kommandovindue.

Installer pySerial-modulet ved at skrive:

På Windows: `pip install pyserial`

På Linux eller macOS: `sudo pip install pyserial`

Luk kommandovinduet.

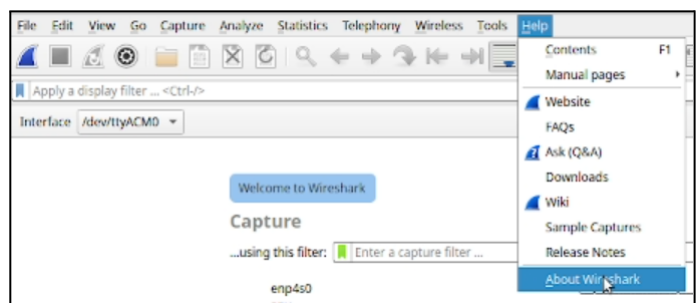
Kopiér sniffer-plugin'et ind i Wiresharks mappe til eksterne capture-plugins:

Åbn Wireshark.

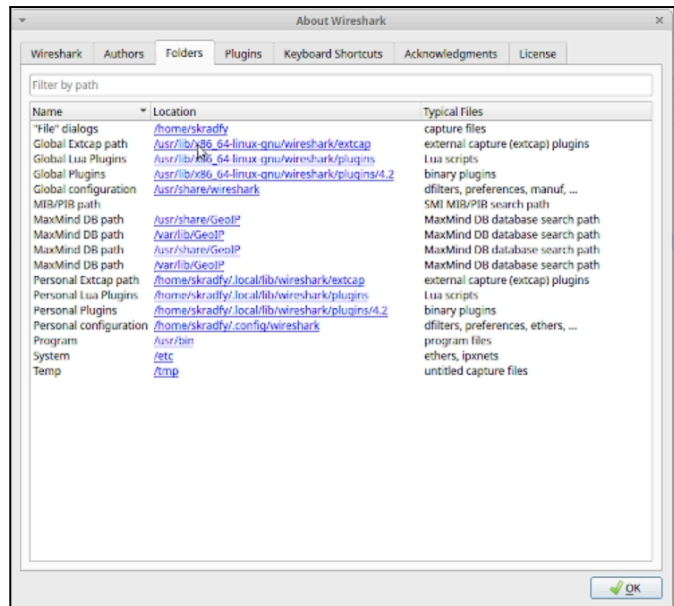
Gå til Hjælp > Om Wireshark (på Windows eller Linux)

eller Wireshark > Om Wireshark (på macOS).

Vælg fanen Mapper (Folders).



Åbn plugin-mappen ved at dobbeltklikke på placeringen for Global Extcap path. Denne mappe indeholder plugins, som er tilgængelige for alle brugere. Plugins, der kopieres til Personal Extcap path, installeres kun for én bruger. Kopiér følgende filer fra mappen Sniffer_Software/nrf802154_sniffer/ til denne mappe: På alle styresystemer: nrf802154_sniffer.py Derudover på Windows: nrf802154_sniffer.bat Nrf802154_sniffer.py og nrf802154_sniffer.bat kan hentes på:



\$ git clone https://github.com/NordicSemiconductor/nRF-Sniffer-for-802.15.4.git

Kontrollér at nRF-sniffer-filerne kører korrekt:

Åbn et kommandovindue i Wiresharks mappe til eksterne capture-plugins.

Kør sniffer-værktøjet for at vise tilgængelige interfaces:

På Windows: nrf802154_sniffer.bat --extcap-interfaces

På macOS eller Linux:

/usr/lib/x86_64-linux-gnu/wireshark/extcap/nrf802154_sniffer.py --extcap-interfaces

Man bør se en række tekstlinjer:

```
skradfy@skradfy: /usr/lib/x86_64-linux-gnu/wireshark/extcap$ /usr/lib/x86_64-linux-gnu/wireshark/extcap/nrf802154_sniffer.py --extcap-interfaces
extcap {version=0.8.0}{help=https://github.com/NordicSemiconductor/nRF-Sniffer-for-802.15.4}{display=nRF Sniffer for 802.15.4}
interface {value=/dev/ttyACM0}{display=nRF Sniffer for 802.15.4}
control {number=6}{type=button}{role=logger}{display=Log}{tooltip=Show capture log}
```

Hvis man får en fejl i forrige trin, skal du kontrollere at Python 3 er tilgængelig:

På Windows: skriv python --version

På macOS eller Linux: skriv python3

Hvis kommandoen ikke findes, eller versionen er forkert, skal du sikre dig, at Python v3.10 eller nyere ligger i din PATH, og at det er den første Python-version i rækkefølgen.

For macOS eller Linux:

Kontrollér at filen nrf802154_sniffer.py har eksekveringsrettigheder (x).

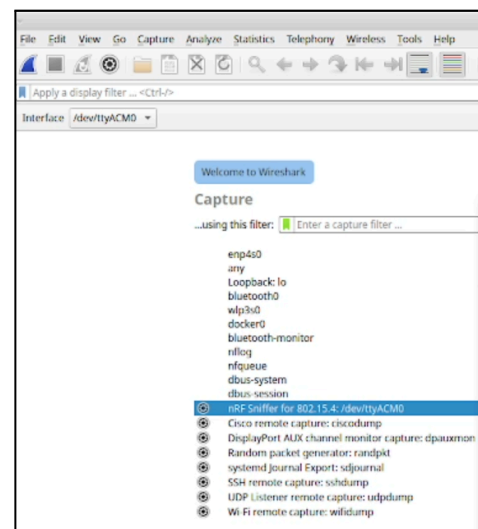
Hvis ikke, tilføj dem med:

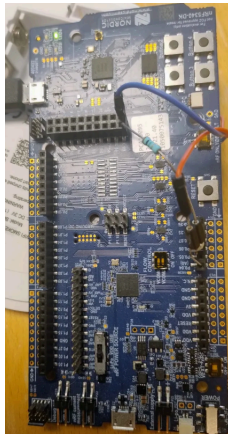
chmod +x nrf802154_sniffer.py

Opdater interfaces i Wireshark:

Vælg Capture > Refresh Interfaces, eller tryk på F5.

Derefter bør nRF Sniffer for 802.15.4 blive vist som et af interfaces på startskærmen i Wireshark.





OBS: Bruger man et udviklingskit (DK) med en nRF Universal Serial Bus (USB)-forbindelse tilgængelig, anbefales det at bruge nRF USB-forbindelsen i stedet for den virtuelle COM-port på det indbyggede interface-MCU. Dette skyldes, at den virtuelle COM-port ikke kan matche den højere Bluetooth LE-gennemstrømning, som nRF USB-forbindelsen tilbyder.

Derefter er det muligt at se trafik:

304	2304.974169			IEEE 802.15.4	31 Ack
305	2311.946833	fe80::1029:185b:3875:d707	ff02::1	MLE	95
306	2314.972142	0x3401	0x3400	IEEE 802.15.4	48 Data Request
307	2314.973236			IEEE 802.15.4	31 Ack
308	2314.986849	0x3400	0x3401	IEEE 802.15.4	75 Data, Dst: 0x3401, Src: 0x3400
309	2314.988801			IEEE 802.15.4	31 Ack
310	2322.096694	fe80::1029:185b:3875:d707	ff02::1	MLE	95
311	2324.971856	0x3401	0x3400	IEEE 802.15.4	48 Data Request
312	2324.972951			IEEE 802.15.4	31 Ack
313	2334.886471	fe80::1029:185b:3875:d707	ff02::1	MLE	95
314	2334.971562	0x3401	0x3400	IEEE 802.15.4	48 Data Request
315	2334.972657			IEEE 802.15.4	31 Ack
316	2344.971271	0x3401	0x3400	IEEE 802.15.4	48 Data Request
317	2344.972366			IEEE 802.15.4	31 Ack
318	2345.806290	fe80::1029:185b:3875:d707	ff02::1	MLE	95
319	2354.969889	0x3401	0x3400	IEEE 802.15.4	48 Data Request
320	2354.970984			IEEE 802.15.4	31 Ack
321	2361.366051	fe80::1029:185b:3875:d707	ff02::1	MLE	95
322	2364.970949	0x3401	0x3400	IEEE 802.15.4	48 Data Request
323	2364.971143			IEEE 802.15.4	31 Ack
324	2368.365904	fe80::1029:185b:3875:d707	ff02::1	MLE	95
325	2374.970078	0x3401	0x3400	IEEE 802.15.4	48 Data Request
326	2374.971173			IEEE 802.15.4	31 Ack
327	2384.485680	fe80::1029:185b:3875:d707	ff02::1	MLE	95
328	2384.969146	0x3401	0x3400	IEEE 802.15.4	48 Data Request
329	2384.970230			IEEE 802.15.4	31 Ack
330	2384.995709	0x3400	0x3401	IEEE 802.15.4	94 Data, Dst: 0x3401, Src: 0x3400
331	2384.998270			IEEE 802.15.4	31 Ack
332	2392.015539	fe80::1029:185b:3875:d707	ff02::1	MLE	95
333	2394.968633	0x3401	0x3400	IEEE 802.15.4	48 Data Request
334	2394.969728			IEEE 802.15.4	31 Ack

Wireshark-listen viser mange IEEE 802.15.4-rammer: MLE, Data Request, Ack, Data. Det er normalt Thread-trafik (MAC + MLE + 6LoWPAN/IPv6/UDP). ot-ctl-output viser Border Router / nodes IPv6-adresser og en child i child table (RLOC16 0x3401). UART-log (UDP received: hi) viser at applikationen inden for enhedens stack faktisk modtager/viser UDP-payload — altså at applikationen (eller border router) ser plaintext. MEN det betyder ikke nødvendigvis at radiokapturen er ukrypteret på luften.

Krypteringsstatus:

▼ Auxiliary Security Header
▼ Security Control Field: 0x0d, Security Level: Encryption with 32-bit Message Integrity Code, Key Identifier Mode: Indexed Key using the Default Key Source
... 101 = Security Level: Encryption with 32-bit Message Integrity Code (0x5)
... 1... = Key Identifier Mode: Indexed Key using the Default Key Source (0x1)
..0... = Frame Counter Suppression: False
.0... = ASN in Nonce: False
0... = Reserved: 0x0
Frame Counter: 300009

Security Level = Encryption + MIC 32 betyder, at denne frame er krypteret med AES-CCM på link-laget.

▼ [Expert Info (Warning/Undecoded): No encryption key set - can't decrypt]
[No encryption key set - can't decrypt]
[Severity level: Warning]
[Group: Undecoded]

Man kan på nuværende tidspunkt ikke læse UDP beskederne.

Sådan opsætter man dekryptering i wireshark:

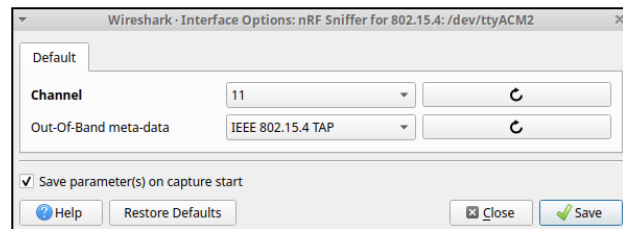
For at kunne se den krypterede trafik i wireshark er det vigtigt at kende følgende: *Netværks kanalen*, *IEEE 802.14.5* Networkkey til thread netværket, *DTLS*, som man skal generere en lokal nøgle for at få adgang til og den anvendte UDP port i *CoAP*.

Netværks kanalen indstilles i wireshark ved at klikke på tandhjulet:



 **nRF Sniffer for 802.15.4: /dev/ttyACM2**

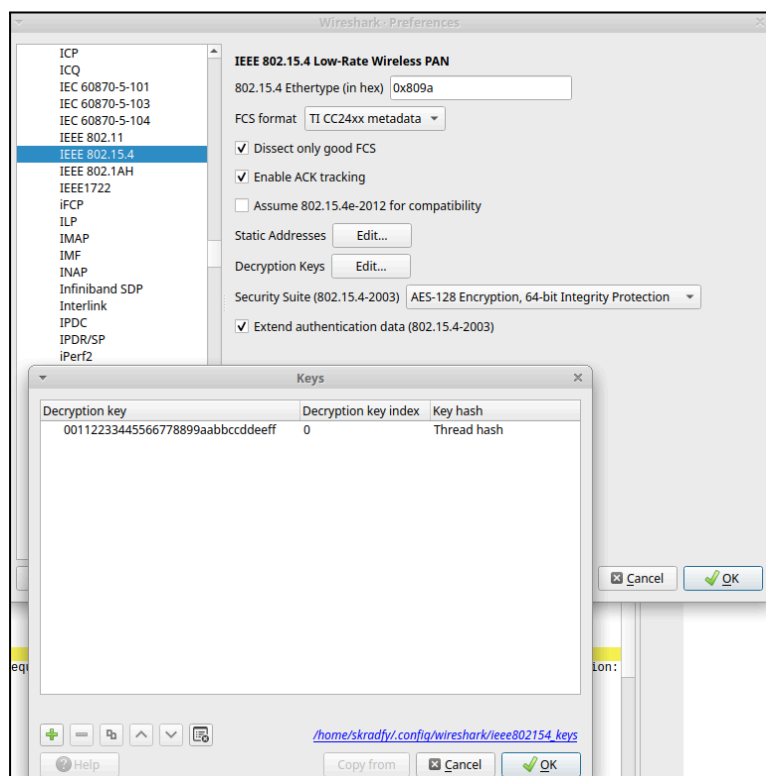
hvorefter man kan indstille kanal.



De næste tre indstillinger det kræver man har adgang til skal indstilles i Edit > Preferences Herefter skal man folder Protocols ud:

IEEE 802.14.5 standarden skal indstilles med networkkey fra thread netværket. kommandoen til dette er “dataset active” på border routeren og indstille i protokollen under Decryption keys med det grønne plus og sætte et thread hash på.

Network Key: 00112233445566778899aabbccddeeff



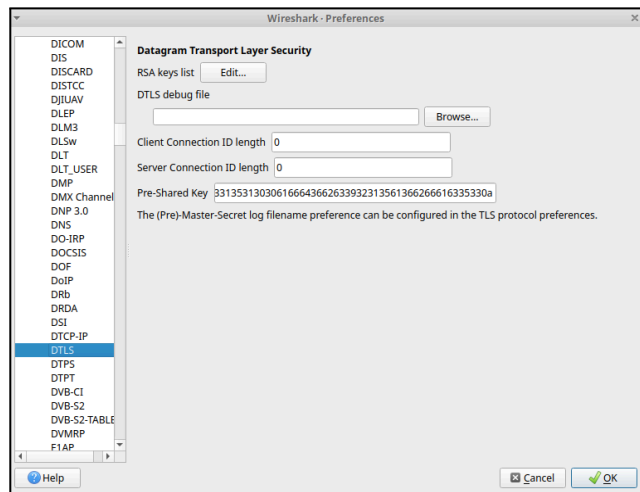
DTLS skal man generere en pre-shared key (psk) for at gøre dette skal man have fat i sin prehex pre shared key, som findes på border routeren ved at anvende thread kommandoen pskc.

```
> state
leader
Done
> pskc
104810e2315100afd6bc9215a6bfac53
Done
```

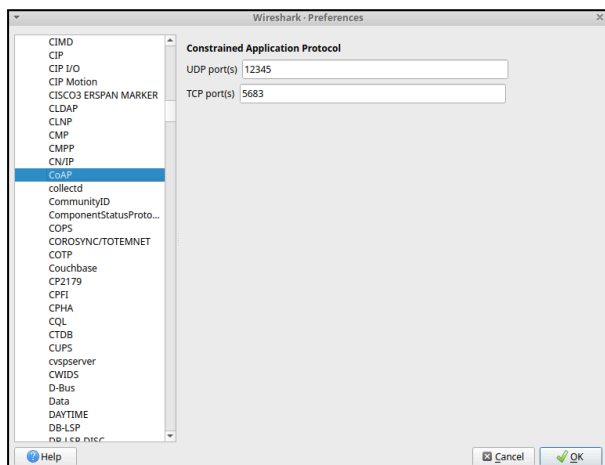
Denne nøgle skal omdannes til en HEX nøgle ved at skrive kommandoen med "psk" før den kan anvendes i wireshark: `$ echo "104810e2315100afd6bc9215a6bfac53" | tr -d '/n' | xxd -ps -c 200`

```
skradfy@skradfy:~$ echo "104810e2315100afd6bc9215a6bfac53" | tr -d '/n' | xxd -ps -c 200
31303438313065323331353130306166643662633932313561366266616335330a
skradfy@skradfy:~$
```

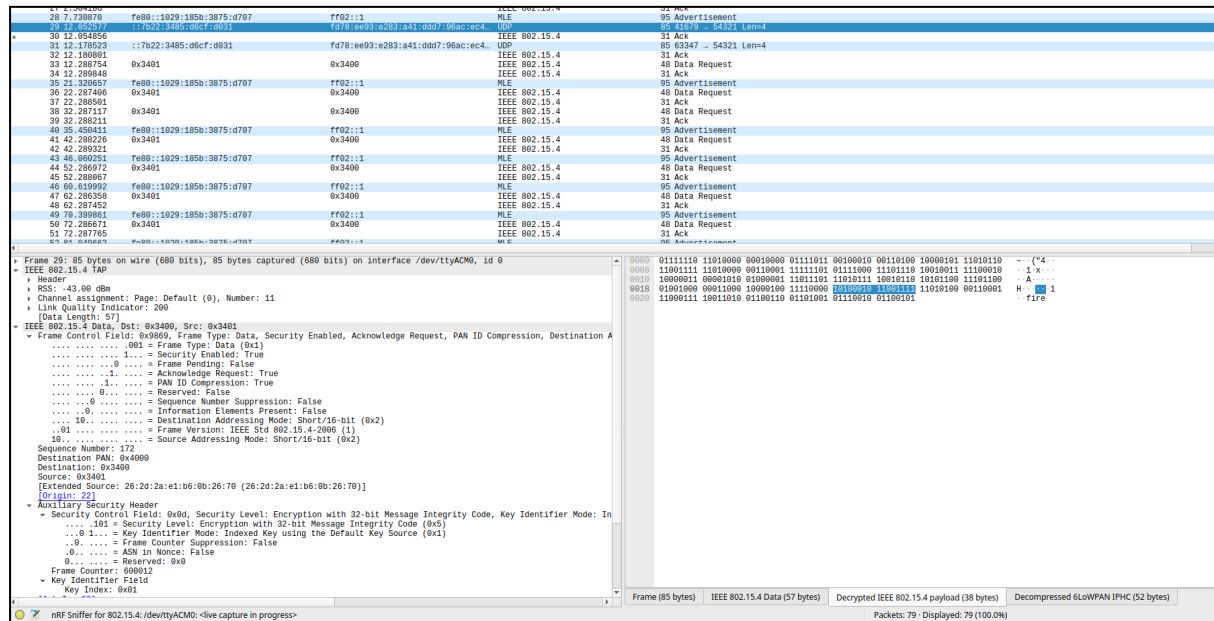
Inde i wireshark skal man finde DTLS under protocols og indsætte sin Hex genererede Pre Shared Key



Derefter skal man finde CoAP i protocols for at sætte UDP porten.



Man kan nu se den krypterede trafik ukrypteret på Wireshark.



ESP32-S3-familien har en dedikeret hardware-AES-enhed, mappet i SoC'ens register-område omkring. 0x6003A000 – 0x6003AFFF (AES hardware peripheral). Her skrives data og nøgle fra /home/usr/esp-idf/components/mbdts/mbdts/library/ ind i AES-registersættet, og chippen udfører selve 128-bit-operationen i hardware, når der flashes firmware.

Kan læses med kommandoen: xtensa-esp32s3-elf-nm build/esp_ot_br.elf | grep -i aes

```
4203ad60 T aes_128_cbc_decrypt
4203ad10 T aes_128_cbc_encrypt
3c153b00 d aes_128_cbc_info
3c1539f4 d aes_128_ccm_info
3c1539dc d aes_128_ccm_star_no_tag_info
3c153ae8 d aes_128_cfb128_info
3c153ab8 d aes_128_ctr_info
3c153b18 d aes_128_ecb_info
3c153a38 d aes_128_gcm_info
3c153ad0 d aes_128_ofb_info
3c153a74 d aes_128_xts_info
3c153af8 d aes_192_cbc_info
3c1539ec d aes_192_ccm_info
3c1539d4 d aes_192_ccm_star_no_tag_info
3c153ae0 d aes_192_cfb128_info
3c153ab0 d aes_192_ctr_info
3c153b10 d aes_192_ecb_info
3c153a30 d aes_192_gcm_info
3c153ac8 d aes_192_ofb_info
3c153af0 d aes_256_cbc_info
3c1539e4 d aes_256_ccm_info
```

AES128 for Zephyr ligger i

/home/usr/ECT_OT_UDVIKLING/nrf_sed/nrf54l15udp/build/nrf54l15udp/zephyr

Det er filerne:

build/nrf54l15udp/zephyr/zephyr.elf

build/nrf54l15udp/zephyr/zephyr_pre0.elf

build/nrf54l15udp/zephyr/zephyr.bin



De kan ikke læses, men de er eksekverbare og man kan få indholdet ved at anvende kommandoen: `$ nm zephyr.elf | grep -i -E "aes|ccm|cc3|mbedtls"`

```
00000001 A CONFIG_OPENTHREAD_MBEDTLS_LIB_NAME
00000001 A CONFIG_PSA_ACCEL_CCM_AES_128
00000001 A CONFIG_PSA_ACCEL_CCM_AES_192
00000001 A CONFIG_PSA_ACCEL_CCM_AES_256
00000001 A CONFIG_PSA_ACCEL_CMAC_AES_128
00000001 A CONFIG_PSA_ACCEL_CMAC_AES_192
00000001 A CONFIG_PSA_ACCEL_CMAC_AES_256
00000001 A CONFIG_PSA_ACCEL_ECB_NO_PADDING_AES_128
00000001 A CONFIG_PSA_ACCEL_ECB_NO_PADDING_AES_192
00000001 A CONFIG_PSA_ACCEL_ECB_NO_PADDING_AES_256
00000001 A CONFIG_PSA_ACCEL_GCM_AES_128
00000001 A CONFIG_PSA_ACCEL_GCM_AES_192
00000001 A CONFIG_PSA_ACCEL_GCM_AES_256
00000001 A CONFIG_PSA_NEED_CRACEN_CCM_AES
00000001 A CONFIG_PSA_NEED_CRACEN_ECB_NO_PADDING_AES
00000001 A CONFIG_PSA_NEED_CRACEN_GCM_AES
00000001 A CONFIG_PSA_WANT_AES_KEY_SIZE_128
00000001 A CONFIG_PSA_WANT_AES_KEY_SIZE_192
00000001 A CONFIG_PSA_WANT_AES_KEY_SIZE_256
00000001 A CONFIG_PSA_WANT_ALG_CCM
00000001 A CONFIG_PSA_WANT_ALG_PBKDF2_AES_CMAC_PRF_128
00000001 A CONFIG_PSA_WANT_KEY_TYPE_AES
00000001 A CONFIG_ZEPHYR_MBEDTLS_MODULE
```

De otte tal foran hvert symbol (fx 0005385d) er adresser i firmwares hukommelsesbillede — altså hvor funktionen eller variablen ligger i den kompilerede binærfil (zephyr.elf).

Symbol	Betydning
CONFIG_PSA_ACCEL_CCM_AES_128	Hardware-acceleration (PSA layer) for AES-CCM 128-bit er aktiveret. CCM = Counter with CBC-MAC → bruges til Thread/DTLS.
CONFIG_PSA_ACCEL_ECB_NO_PADDING_AES_128	Hardware-accelereret AES-ECB 128 (basisblok som CCM/GCM bruger).
CONFIG_PSA_WANT_KEY_TYPE_AES	PSA Crypto API har AES som nøgletype til rådighed.
CONFIG_PSA_WANT_AES_KEY_SIZE_128	Angiver at 128-bit nøglelængde understøttes og bruges.
CONFIG_PSA_ACCEL_GCM_AES_128	(Valgfrit) Hardware-acceleration for GCM, men CCM er den der bruges i Thread.
CONFIG_PSA_NEED_CRACEN_CCM_AES	Din build bruger Nordic CRACEN-engine – det er den interne AES-hardware i nRF54-familien.
create_aead_ccmheader, feed_singlepart_ccm_aad	Funktionssymboler i AES-CCM-databehandling. Det er softwaren, der forbereder AAD/data til HW-encrypt/decrypt.



10.13 Angrebsmetoder på AES

I forundersøgelsen er flere relevante angrebsvektorer mod AES-implementeringer blevet identificeret og teknisk kortlagt ud fra følgende relevante metoder:

Power analysis attacks:

<https://raelize.com/blog/espressif-systems-esp32-breaking-hw-aes-with-power-analysis/>

Side-channel attacks - med ChipWhisperer

[http://wiki.newae.com/V4:Tutorial_B5_Breaking_AES_\(Straightforward\)](http://wiki.newae.com/V4:Tutorial_B5_Breaking_AES_(Straightforward))

Cache timing attacks:

<https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>

Electromagnetic attacks:

<https://eprint.iacr.org/2018/076.pdf>

Disse angrebstyper udnytter fysisk eller tidsmæssig information fra enhedens implementering - eksempelvis strømprofiler, elektromagnetiske emissioner eller mikroarkitektur-timing - for at opnå information om den hemmelige nøgle-værdi uden at angribe selve algoritmen. Erfaringen viste, at reproduktion og systematisk evaluering af disse angreb kræver specialiseret måleudstyr, reproducerbare testopstillinger og betydelige ressourcer, hvorfor praktisk eksperimentering med disse teknikker ikke er udført i den nuværende udviklingsfase.

Der er også foretaget en indledende vurdering af muligheden for at anvende dedikeret forskningsudstyr. Adgang til sådant udstyr vurderes dog at være begrænset, og der foreligger derfor i denne rapport ingen intern, dokumenteret proces for eksperimentel udtrækning af AES-nøgler. Offentligt tilgængeligt demonstrationsmateriale og instruktionsvideoer er blevet gennemgået som teoretisk reference, men dette materiale er anvendt udelukkende til at forstå angrebsprincipperne - ikke til at udføre operationelle tests.

Som konsekvens af denne afgrænsning lægges der op til, at fremtidige udviklingsfaser indeholder en målrettet plan for side-channel-validering og -hærdning før produktionssætning. Eksempler på anbefalede modforanstaltninger omfatter: implementering af constant-time- og maskerings-teknikker i software, microarchitectural hardening, indførelse af hardwarebaserede tamper-detektioner og sikre nøgleopbevaringsmekanismer samt etablering af logging og hændelseshåndteringsprocedurer ved mistanke om uautoriseret fysisk adgang. Endvidere anbefales det, at adgang til specialudstyr og etablering af kontrollerede testfaciliteter prioriteres, så effekt og sikkerhed af de valgte mitigationer kan verificeres eksperimentelt i en senere valideringsfase.



10.14 Konvertering til HTTPS

Først skal der oprettes et *lokalt CA* og et *gyldigt servercertifikaten* med en openssl-sekvens for at kunne anvende webserveren som https. Mapper laves med:

```
$ mkdir -p ~/esp_ca && cd ~/esp_ca
```

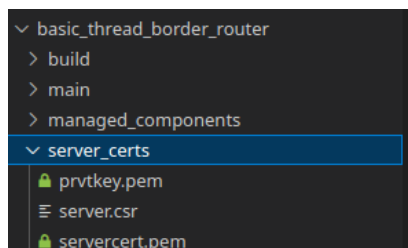
Lav CA privat nøgle:

```
$ openssl genrsa -out ca.key 2048
```

Lav CA certifikat:

```
$ openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt \
-subj "/C=DK/ST=Aarhus/L=Aarhus/O=ECT_CA/OU=Thread_CA/CN=ECT_Local_CA"
```

I denne folder: server_certs



Oprettes ESP32 nøgle og CSR (certifikat-reqiest) med cmd:

ESP32 privat nøgle:

```
$ openssl genrsa -out prvtkey.pem 2048
```

ESP32 CSR (Certificate Signing Request):

```
$ openssl req -new -key prvtkey.pem -out server.csr \
-subj "/C=DK/ST=Aarhus/L=Aarhus/O=ECT_BorderRouter/OU=ESP32/CN=10.0.0.84"
```

CN=10.0.0.84 skal matche IP'en for at tilgå siden

Herefter skal certifikatet signeres og filerne servercert.pem, prvtkey.pem og ca.crt genereres:

```
"openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
-out servercert.pem -days 365 -sha256 \
-extfile <(printf "subjectAltName=IP:10.0.0.84,DNS:esp32.local")"
```

Importer herefter CA til lokal browser:

Settings → Security → Manage certificates → Import

importer ca.crt

genstart browser

Tilføj includes:

```
#include "esp_https_server.h"
```

HTTPS understøttende kode i esp_br_web.c:

```
extern const unsigned char servercert_pem_start[] asm("_binary_servercert_pem_start");
extern const unsigned char servercert_pem_end[]   asm("_binary_servercert_pem_end");
extern const unsigned char prvtkey_pem_start[]    asm("_binary_prvtkey_pem_start");
extern const unsigned char prvtkey_pem_end[]      asm("_binary_prvtkey_pem_end");

#define SERVER_CERT      servercert_pem_start
```



```
#define SERVER_CERT_LEN  (servercert_pem_end - servercert_pem_start)
#define SERVER_KEY        prvtkey_pem_start
#define SERVER_KEY_LEN    (prvtkey_pem_end - prvtkey_pem_start)
```

HTTPD handle og port 443 skal inkluderes:

```
static httpd_handle_t *start_esp_br_https_server(const char *base_path, const char *host_ip)
{
    ESP_RETURN_ON_FALSE(base_path, NULL, WEB_TAG, "Invalid https server path");
    strcpy(s_server.ip, host_ip);
    strncpy(s_server.data.base_path, base_path, ESP_VFS_PATH_MAX + 1);
    list_spiffs_files(base_path);

    httpd_ssl_config_t conf = HTTPD_SSL_CONFIG_DEFAULT();
    conf.port_secure = 443;
    conf.servercert = SERVER_CERT;
    conf.servercert_len = SERVER_CERT_LEN;
    conf.prvtkey_pem = SERVER_KEY;
    conf.prvtkey_len = SERVER_KEY_LEN;

    // Allow wildcard URI matching (so "/*" works)
    conf.httpd.uri_match_fn = httpd_uri_match_wildcard;

    // Start HTTPS-server
    esp_err_t ret = httpd_ssl_start(&s_server.handle, &conf);
    if (ret != ESP_OK) {
        ESP_LOGE(WEB_TAG, "Failed to start HTTPS server: %s", esp_err_to_name(ret));
        return NULL;
    }

    // Registrér URI-handlers
    httpd_uri_t default_uris_get = {
        .uri = "/*",
        .method = HTTP_GET,
        .handler = default_urls_get_handler,
        .user_ctx = &s_server.data
    };
    for (int i = 0; i < sizeof(s_resource_handlers) / sizeof(httpd_uri_t); i++) {
        httpd_register_uri_handler(s_server.handle, &s_resource_handlers[i]);
    }
    httpd_register_uri_handler(s_server.handle, &default_uris_get);

    // Info i log
    ESP_LOGI(WEB_TAG, "%s\r\n", "<=====HTTPS server start=====>");
    ESP_LOGI(WEB_TAG, "https://%s:%d/index.html\r\n", s_server.ip, 443);
    ESP_LOGI(WEB_TAG, "%s\r\n", "<=====>");

    return s_server.handle;
}
```



Opsætning af HTTPS authenticering

For at opsætte en authenticering af https, blev denne kode indsat i esp_br_web.c, det er en simpel kode, der anvender base64 metoden:

```
#define AUTH_STRING "Basic YWRtaW46dGVzdDEyMw==" // "admin:test123" med base64
static const char *AUTH_TAG = "auth";

static esp_err_t check_auth(httpd_req_t *req)
{
    char auth_value[128];
    size_t auth_len = httpd_req_get_hdr_value_len(req, "Authorization") + 1;

    if (auth_len <= 1) {
        httpd_resp_set_hdr(req, "WWW-Authenticate", "Basic realm=\"Secure\"");
        httpd_resp_send_err(req, HTTPD_401_UNAUTHORIZED, "Unauthorized");
        ESP_LOGW(AUTH_TAG, "Access denied: no credentials");
        return ESP_FAIL;
    }

    httpd_req_get_hdr_value_str(req, "Authorization", auth_value, sizeof(auth_value));

    if (strcmp(auth_value, AUTH_STRING) != 0) {
        httpd_resp_set_hdr(req, "WWW-Authenticate", "Basic realm=\"Secure\"");
        httpd_resp_send_err(req, HTTPD_401_UNAUTHORIZED, "Unauthorized");
        ESP_LOGW(AUTH_TAG, "Access denied: wrong credentials");
        return ESP_FAIL;
    }

    ESP_LOGI(AUTH_TAG, "Access granted");
    return ESP_OK;
}
```

i static esp_err_t index_html_get_handler(httpd_req_t *req, char *path) funktionen, skal der indsættes et autoriserings-tjek:

```
if (check_auth(req) != ESP_OK) {
    return ESP_FAIL;
}
```



10.15 OTNS tabeller

```
node 1> router table
```

ID	RLOC16	Next Hop	Path Cost	LQ In	LQ Out	Age	Extended MAC	Link
0	0x0000	12	2	3	3	1	2a0c5d75293c24f3	1
1	0x0400	63	0	0	0	0	8afab88584311ee2	0
12	0x3000	0	2	2	2	2	f6ab38b3b00c4a4a	1

```
node 1> neighbor table
```

Role	RLOC16	Age	Avg RSSI	Last RSSI	LQ In	R D N	Extended MAC	Version
R	0x0000	1	-77	-78	3	1 1 1	2a0c5d75293c24f3	5
R	0x3000	2	-84	-84	2	1 1 1	f6ab38b3b00c4a4a	5

```
node 2> router table
```

ID	RLOC16	Next Hop	Path Cost	LQ In	LQ Out	Age	Extended MAC	Link
0	0x0000	63	0	0	0	0	2a0c5d75293c24f3	0
1	0x0400	12	2	3	3	0	8afab88584311ee2	1
12	0x3000	1	2	2	2	2	f6ab38b3b00c4a4a	1

```
node 2> neighbor table
```

Role	RLOC16	Age	Avg RSSI	Last RSSI	LQ In	R D N	Extended MAC	Version
C	0x0001	112	-78	-78	3	0 0 1	9ed826ce3a23082f	5
R	0x0400	0	-77	-78	3	1 1 1	8afab88584311ee2	5
R	0x3000	2	-85	-84	2	1 1 1	f6ab38b3b00c4a4a	5

```
node 2> child table
```

ID	RLOC16	Timeout	Age	LQ In	C_VN	R D N	Ver	CSL	QMsgCnt	Suprvsn	Extended MAC
1	0x0001	240	112	3	88	0 0 1	5	0	1	129	9ed826ce3a23082f

```
node 3> router table
```

ID	RLOC16	Next Hop	Path Cost	LQ In	LQ Out	Age	Extended MAC	Link
0	0x0000	1	1	2	2	1	2a0c5d75293c24f3	1
1	0x0400	0	1	2	2	0	8afab88584311ee2	1
12	0x3000	63	0	0	0	0	f6ab38b3b00c4a4a	0

```
node 3> neighbor table
```

Role	RLOC16	Age	Avg RSSI	Last RSSI	LQ In	R D N	Extended MAC	Version
C	0x3001	110	-66	-66	3	0 0 1	c23eb01f0e2fcda0	5
R	0x0000	1	-85	-84	2	1 1 1	2a0c5d75293c24f3	5
R	0x0400	0	-84	-84	2	1 1 1	8afab88584311ee2	5

```
node 3> child table
```

ID	RLOC16	Timeout	Age	LQ In	C_VN	R D N	Ver	CSL	QMsgCnt	Suprvsn	Extended MAC
1	0x3001	240	110	3	88	0 0 1	5	0	1	129	c23eb01f0e2fcda0